



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sistema Inteligente para la Identificación y Seguimiento de los derechos de los NNA por feminicidios

Trabajo Terminal no. 2026-A135

Presentan:

Morales Martinez Hector Alberto

Directores:

M. en C. Ulises Saldaña Velez

M. en C. Gabriel Hurtado Avilés

Ciudad de México, a 4 de junio de 2026



0.1. Glosario y conceptos técnicos del sistema	1
1. Introducción	4
1.1. Presentación del proyecto	4
1.2. Planteamiento del problema	5
1.2.1. La inviabilidad del monitoreo manual	5
1.3. Objetivos del Proyecto	5
1.3.1. Objetivo General	5
1.3.2. Objetivos Específicos	5
2. Justificación y alcance del proyecto	6
2.1. Justificación social	6
2.2. Justificación tecnológica	7
2.3. Justificación ética y moral: Protección de la privacidad de los NNA	8
2.3.1. El interés superior del niño	8
2.3.2. Naturaleza de la información procesada	8
2.3.3. ¿Por qué el sistema no proporciona datos completos de NNA?	8
2.4. Alcance del sistema	9
2.4.1. Alcance funcional	9
2.4.2. Alcance geográfico y temporal	9
2.5. Limitaciones y exclusiones	9
2.5.1. Exclusiones explícitas del alcance	10
2.5.2. Limitaciones técnicas	10
3. Marco Teórico	11
3.1. Procesamiento de lenguaje natural y representación lingüística	11
3.1.1. Tokenización y el algoritmo WordPiece	11
3.1.2. Lematización y normalización lexicológica	12
3.1.3. Modelo de espacio vectorial: TF-IDF	12
3.1.4. Arquitectura Transformer y mecanismo de auto-atención	13
3.1.5. BETO: representaciones bidireccionales para el español	13
3.2. Paradigmas de inferencia y clasificación semántica	14

3.2.1.	Clasificación zero-shot por similitud coseno (modo fallback)	14
3.2.2.	Ajuste fino supervisado (<i>Fine-Tuning</i>) con <i>Linear Probing</i>	14
3.2.3.	Acumulación de Gradientes (<i>Gradient Accumulation</i>)	15
3.3.	Agrupamiento semántico y reducción de dimensionalidad	15
3.3.1.	UMAP: Proyección Topológica Preservadora de Estructura	15
3.3.2.	HDBSCAN: Agrupamiento Jerárquico por Densidad Variable	16
3.3.3.	c-TF-IDF: Relevancia de Términos por Clúster	17
3.4.	Técnicas de Deduplicación <i>Cross-Site</i>	17
3.4.1.	Hash Exacto MD5	17
3.4.2.	Similitud de Jaccard sobre Conjuntos de Tokens	17
3.4.3.	SimHash: Huellas Digitales Localmente Sensibles	17
3.4.4.	Similitud Coseno sobre Vectores TF-IDF	18
3.5.	Tecnologías de Extracción de Datos y Persistencia	18
3.5.1.	Protocolos de Sindicación: RSS y Atom	18
3.5.2.	Evasión de WAF: <i>StealthSession</i> y TLS/JA3 Fingerprinting	18
3.5.3.	Propiedades ACID de postgresql	19
3.5.4.	Índices GIN y Full-Text Search en Español	19
3.6.	Sistemas Neuro-Simbólicos y Clasificación Escalonada v7.1	20
3.6.1.	Arquitectura de <i>Scoring Escalonado</i> v7.1: las cuatro capas	20
3.7.	Modelos de Lenguaje de Gran Escala y el Módulo <i>DeepInvestigator</i>	21
3.7.1.	Síntesis estructurada mediante Gemini Flash	21
3.7.2.	Agentes de búsqueda y el patrón OSINT	22
4.	Estado del Arte	23
4.1.	Sistemas de monitoreo de medios tradicionales	23
4.2.	NLP Aplicado a la Detección de Violencia de Género	23
4.3.	Herramientas y esfuerzos existentes en México	24
4.3.1.	Red por los Derechos de la Infancia en México (REDIM)	24
4.3.2.	El Mapa de los Feminicidios en México	24
4.3.3.	Fundación Futuro con Derechos	24
4.4.	Análisis comparativo y brecha identificada	24
4.5.	Técnicas de extracción de datos	25
4.5.1.	Extracción Estática (HTML Parsing con BeautifulSoup)	25
4.5.2.	Automatización de Navegador (Selenium / Playwright)	25
4.5.3.	Agregación Estructurada (RSS/Atom) y Stealth Sessions	26
4.5.4.	Tabla Comparativa de Técnicas de Extracción	26
4.6.	Evolución del Almacenamiento: de Archivos Planos a PostgreSQL	27
4.6.1.	Archivos Planos (CSV): Ventajas, Limitaciones y el Techo del TT1	27
4.6.2.	PostgreSQL 16: Justificación de la Selección	28
4.7.	Evolución de los Modelos de Representación Lingüística para NLP	29
4.7.1.	Representaciones Vectoriales Estáticas: Word2Vec y GloVe	29
4.7.2.	Modelos Recurrentes: LSTM y GRU	29
4.7.3.	Arquitectura Transformer y Atención Bidireccional: BERT	30
4.7.4.	Justificación de BETO frente a Modelos Multilingües (mBERT)	30
4.7.5.	El Límite del <i>Zero-Shot</i> y la Transición al Sistema de Clasificación Escalonada	31
4.7.6.	Soberanía de Datos y Rechazo de APIs Privativas	32
4.7.7.	Investigación Profunda Bajo Demanda: <i>DeepInvestigator</i>	32
4.8.	Técnicas de Agrupamiento Semántico y Organización Temática	33

4.8.1. Métodos Tradicionales: K-Means y su Inadecuación para Datos de Alta Dimensionalidad	33
4.8.2. DBSCAN: Ventajas y Limitaciones Residuales	34
4.8.3. BERTopic: Arquitectura Modular para Descubrimiento de Tópicos Semánticos	34
4.8.4. Evolución Comparativa TT1 → TT2: de Léxico a Semántica	36
4.9. Conclusión del Capítulo: La Convergencia Tecnológica al Servicio de los Derechos de la Infancia	37
5. Metodología de desarrollo	39
5.1. Metodología de desarrollo evolutivo	39
5.1.1. Cronograma general de evolución del proyecto	39
5.2. Evolución metodológica de los prototipos	40
5.2.1. Fase TT-1 Prototipos exploratorios e ingesta sintáctica (P0-P2)	40
5.2.2. Fase TT-2 Arquitectura de aprendizaje profundo y clasificador escalonado (P3-P4)	41
5.3. Evolución del Stack Tecnológico (TT-1 → TT-2)	41
5.4. Arquitectura de Microservicios, Contenerización y Gestión de Recursos	42
5.4.1. Contenedor de base de datos nna-postgres	42
5.4.2. Contenedor del Backend nna-analyzer	43
5.4.3. Contenedor de interfaz analítica y dashboard web nna-webapp	43
5.5. Especificación de requerimientos	43
6. Requerimientos del sistema	46
6.1. Perfiles de usuario	46
6.2. Requerimientos Funcionales	47
6.3. Requerimientos No Funcionales	49
6.4. Modelado de Casos de Uso	51
6.5. CU-01 Ejecutar flujo de recolección y procesamiento de noticias	52
6.5.1. Descripción completa	52
6.5.2. Atributos importantes	52
6.5.3. Trayectorias del Caso de Uso	53
6.6. CU-02 Gestión del catálogo de fuentes de información	54
6.6.1. Descripción completa	54
6.6.2. Atributos importantes	54
6.6.3. Trayectorias del Caso de Uso	55
6.7. CU-03 Investigación profunda bajo demanda	57
6.7.1. Descripción completa	57
6.7.2. Atributos importantes	57
6.7.3. Trayectorias del Caso de Uso	57
6.8. CU-04 Identificación por enlace externo	59
6.8.1. Descripción completa	59
6.8.2. Atributos importantes	59
6.8.3. Trayectorias del Caso de Uso	59
6.9. CU-05 Gestión y exportación de seguimiento	61
6.9.1. Descripción completa	61
6.9.2. Atributos importantes	61
6.9.3. Trayectorias del Caso de Uso	61
6.10. CU-06 Iniciar sesión en el sistema	63
6.10.1. Descripción completa	63
6.10.2. Atributos importantes	63
6.10.3. Trayectorias del Caso de Uso	64

6.11. CU-07 Búsqueda y Filtrado de Noticias	65
6.11.1. Descripción completa	65
6.11.2. Atributos importantes	65
6.11.3. Trayectorias del Caso de Uso	66
6.12. Interfaces de Usuario	67
6.13. IU-01 Dashboard principal de noticias	67
6.13.1. Objetivo	67
6.13.2. Diseño	67
6.13.3. Salidas	68
6.13.4. Entradas	68
6.13.5. Comandos	68
6.14. IU-02 Gestión de Fuentes de Información	68
6.14.1. Objetivo	68
6.14.2. Diseño	68
6.14.3. Salidas	69
6.14.4. Entradas	69
6.14.5. Comandos	69
6.15. IU-03 Módulo de investigación profunda	70
6.15.1. Objetivo	70
6.15.2. Diseño	70
6.15.3. Salidas	70
6.15.4. Entradas	71
6.15.5. Comandos	71
6.16. IU-04 Seguimiento de casos y exportación	71
6.16.1. Objetivo	71
6.16.2. Diseño	71
6.16.3. Salidas	71
6.16.4. Entradas	71
6.16.5. Comandos	71
6.17. IU-05 Pantalla de inicio de sesión	72
6.17.1. Objetivo	72
6.17.2. Diseño	72
6.17.3. Salidas	72
6.17.4. Entradas	73
6.17.5. Comandos	73
7. Desarrollo de Trabajo Terminal I	74
7.1. Prototipo 0 Scraping directo sobre HTML	74
7.1.1. Premisa y enfoque	74
7.1.2. Resultado	74
7.1.3. Decisión Técnica	75
7.2. Prototipo 1 RSS + K-Means	75
7.2.1. Arquitectura implementada	75
7.2.2. Limitaciones identificadas	75
7.3. Prototipo 2 Triple Fuente + DBSCAN + FeminicideDetector	76
7.3.1. Evolución del sistema	76
7.4. Limitaciones identificadas	76
7.4.1. Falsos positivos	76

7.4.2. Clustering léxico	77
7.4.3. La poca escalabilidad del almacenamiento en CSV	77
7.4.4. Síntesis	77
8. Desarrollo de Trabajo Terminal II	78
8.1. Contexto: Limitaciones del Prototipo 2	78
8.2. Patrones arquitectónicos del flujo analítico	79
8.2.1. Patrón <i>Repository</i> para la capa de persistencia	79
8.2.2. Patrón <i>Pipeline</i> para el flujo analítico	79
8.3. Prototipo 3 Motor semántico BETO y migración de persistencia	79
8.3.1. Migración de CSV a PostgreSQL	80
8.3.2. Arquitectura del motor BETO	80
8.3.3. Clasificación <i>Zero-Shot</i> y <i>Temperature Scaling</i>	81
8.3.4. Evolución del recolector	81
8.3.5. Deduplicación en cascada de cuatro fases	83
8.4. Prototipo 4 Arquitectura híbrida de clasificación	83
8.4.1. Por qué la IA sola no basta	85
8.4.2. Investigador profundo	85
8.5. Sistema de clasificación escalonada	88
9. Resultados y evaluación	90
9.1. Resultados de la recolección y deduplicación	90
9.1.1. Volumen de ingesta	90
9.1.2. Cascada de deduplicación	90
9.2. Resultados de la clasificación semántica escalonada	91
9.2.1. Distribución del embudo analítico	91
9.2.2. Comportamiento por modo de inferencia	91
9.3. Resultados del Agrupamiento Semántico (BERTopic)	92
9.4. Resultados del Módulo de Investigación Profunda (<i>DeepInvestigator</i>)	92
9.5. Evaluación comparativa TT1/TT2 y cumplimiento de metas	92

Índice de figuras

5.1. Cronograma metodológico de evolución y transición de los prototipos del sistema.	40
5.2. Arquitectura del sistema.	44
6.1. IU-01 Dashboard principal de noticias.	67
6.2. IU-02 Pantalla de Gestión de Fuentes.	69
6.3. IU-03 Modal de Resultados del DeepInvestigator.	70
6.4. IU-04 Pantalla de Seguimiento de Casos Activos.	72
6.5. IU-05 Pantalla de Inicio de Sesión.	73
8.1. Arquitectura del motor BETO y clasificación <i>Zero-Shot</i> con <i>Temperature Scaling</i>	82
8.2. Flujo de deduplicación en cascada.	83
8.3. Flujo de investigación profunda	87
8.4. Arquitectura del Sistema de Clasificación Escalonada	89
9.1. Visualización jerárquica de noticias de Alta relevancia en el Dashboard, con etiquetas de clasificación y porcentaje de score BETO por registro.	93

Índice de tablas

1. Información general y acta de constitución del proyecto	VIII
2.1. Distinción entre información procesada y excluida	8
4.1. Comparativa de técnicas de extracción de datos para monitoreo de medios	27
4.2. Comparativa de modelos de representación lingüística para clasificación semántica	31
4.3. Comparativa de algoritmos de agrupamiento semántico para corpus periodístico	37
5.1. Evolución del stack tecnológico	42
5.2. Matriz de trazabilidad general	44
7.1. Comparativa de métricas de TT1 contra metas del TT2	77
8.1. Casos de uso del módulo de investigación profunda	88
9.1. Distribución de duplicados eliminados por fase de la cascada de deduplicación	91
9.2. Distribución de noticia por etapa en la clasificación escalonada	91
9.3. Evaluación comparativa de métricas TT1/P2 vs. metas TT2 vs. resultados P4	93

Project Charter

Proyecto:	2026-A135: Sistema Inteligente para la Identificación y Seguimiento de NNA		
Responsable:	Héctor Alberto Morales Martínez, desarrollador principal		
Autoriza:	Comisión de Trabajos Terminales (CATT)		
Background/Contexto:	Miles de NNA quedan en orfandad por feminicidios anualmente; actualmente no existe un sistema centralizado de seguimiento para apoyarlos y visibilizar su situación.		
Beneficios esperados:	Visibilidad del fenómeno, reducción radical del trabajo manual de monitoreo para ONGs y estructuración de una base de datos histórica mediante PNL.		
Costo estimado:	Proyecto Académico de Investigación (TT1/TT2)		
Fecha de inicio:	Septiembre 2025	Fecha de término:	Junio 2026
Objetivo:	Desarrollar un sistema automatizado e inteligente de recolección, análisis PNL y visualización de eventos de orfandad por feminicidio en México.		
Entregables Principales			
	SIST-01	Sistema web orquestado en microservicios contene- rizados (Docker Compose)	
	DOC-01	Documentación Técnica (este documento)	
	COD-01	Código fuente modular en repositorio	
Alcance del proyecto			
Incluye:	• Recolección multi-fuente con mas de 50 feeds RSS y Google News API. • Clasificación semántica con el modelo BETO Fine-tuned localmente. • Eliminación de ruido con clasificación escalonada. • Modelado automático de tópicos con BERTopic. • Persistencia con PostgreSQL y dashboard web con modulo de investigación profunda.		
Excluye:	• Identificación, almacenamiento o filtrado de datos personales completos y sensibles de NNA. • Intervención social, apoyo psicológico directo o canalización jurídica oficial ante autoridades. • Validación pericial o legal de la veracidad jurídica de los hechos publicados por los medios.		
Metodología:	Desarrollo evolutivo por prototipos (P0 a P4).		
Datos de contacto			
Desarrollador:	Héctor Alberto Morales Martínez		
Directores:	M. en C. Ulises Saldaña Velez, M. en C. Gabriel Hurtado Avilés		
Organización de Apoyo:			
Riesgos y peligros:	• Bloqueos anti-scraping progresivos y cambios en la estructura de los portales web.		
Supuestos:	• Las fuentes periodísticas locales mantienen un marco de narrativa policiaca para crímenes de género. • Los servicios y APIs de agregación semántica de Google News permanecen accesibles.		

Tabla 1: Información general y acta de constitución del proyecto

0.1. Glosario y conceptos técnicos del sistema

En esta sección se definen los conceptos de ingeniería de software, aprendizaje profundo, seguridad y operación legal que fundamentan la arquitectura del sistema.

Active Learning (Aprendizaje Activo): Estrategia de entrenamiento iterativo donde el modelo de *machine learning* selecciona y solicita el etiquetado de los ejemplos más informativos o confusos lo que acelera la convergencia y mejora la precisión con un volumen reducido de datos etiquetados. Se identifica como línea de trabajo futuro para reducir la tasa de falsos positivos del clasificador BETO por debajo del 10 %.

Batch ID (Identificador de lote): Cadena de caracteres única (formato YYYYMMDD_HHMMSS) generada al inicio de cada ejecución del flujo de recolección y análisis. Todos los registros insertados en esa corrida reciben el mismo `batch_id`, permitiendo al analista filtrar el dashboard por lote específico para comparar corridas históricas y calcular métricas de evolución entre corridas.

BETO: Modelo de lenguaje masivo preentrenado basado en la arquitectura *BERT* bidireccional, especializado en la sintaxis y semántica del idioma español. Cuenta con 12 capas Transformer, 12 cabezas de atención, dimensión oculta de 768 y 110 millones de parámetros. En el sistema actúa como motor de la capa 2 de la arquitectura de clasificación escalonada en modo Fine-Tuned como prioridad.

BERTopic: Algoritmo neuro-estadístico para el modelado dinámico de tópicos. Encadena tres algoritmos independientes: UMAP para reducción de dimensionalidad, HDBSCAN para agrupamiento por densidad variable y c-TF-IDF para la representación léxica de cada tópico. En el sistema genera 15 tópicos semánticos sobre el corpus, con una tasa de outliers reducida del 81.4 % al 55 % mediante estrategias progresivas de reincorporación.

Circuit Breaker (Disyuntor de Dominio): Patrón de resiliencia implementado en *StealthSession* que tras 3 fallos consecutivos de scraping sobre el mismo dominio web este se “abre” y bloquea todas las peticiones hacia ese dominio durante 300 segundos (extensible a 600 en reincidencia) evitando un baneo permanente de IP por presión repetida sobre un WAF activo.

Deduplicación en Cascada: Flujo de limpieza de datos en cuatro fases secuenciales de complejidad creciente: (1) Hash MD5 exacto, (2) Similitud de Jaccard sobre tokens del título, (3) SimHash con distancia de Hamming y (4) Similitud coseno TF-IDF sobre el cuerpo completo. El diseño en cascada garantiza que los casos obvios (copia exacta) se resuelvan con el método más barato lo que reserva los algoritmos más costosos solo para los candidatos que superan los filtros anteriores.

DeepInvestigator (Investigador Profundo): Módulo de inteligencia artificial generativa activado bajo demanda explícita del analista sobre noticias de Alta relevancia. Ejecuta un flujo asíncrono de tres etapas: (1) generación de consulta de búsqueda optimizada mediante Gemini Flash, (2) búsqueda web multi-fuente con DuckDuckGo HTML/Lite y scraping de resultados mediante *StealthSession*, y (3) síntesis estructurada en JSON con siete campos (`ubicacion`, `victimas`, `niños_afectados`, `edades`, `situacion_actual`, `medios_contacto`, `resumen`) mediante Gemini Flash. La latencia media del flujo completo es de aproximadamente 47 segundos por caso.

Efecto péndulo: Fenómeno de colisión semántica diagnosticado en el prototipo 3 donde el clasificador Zero-Shot de BETO exhibía falsos positivos al confundir textos legislativos (por ejemplo Ley Monzón, iniciativas de tipificación del feminicidio) o informes estadísticos nacionales con casos de NNA en situación de orfandad, debido al alto solapamiento de tokens léxicos comunes (“víctima”, “menor”, “homicidio”). El fenómeno se caracterizaba por scores concentrados en el rango [0,40, 0,70] lo que resultaba en una capacidad de discriminación insuficiente entre las clases. Fue resuelto en el prototipo 4 mediante la capa 1 (escudo suave) y el fine-tuning de la cabeza de clasificación.

Fallback (Modo de Respaldo): Mecanismo de degradación controlada que activa automáticamente un subsistema alternativo ante el fallo del subsistema primario, garantizando la continuidad operativa. En el sistema se implementan dos niveles de fallback: (a) el clasificador Zero-Shot se activa si no existe un modelo fine-tuned guardado; y (b) la herramienta GoogleSearch nativa de Gemini se activa si DuckDuckGo bloquea las peticiones del DeepInvestigator.

Feminicidio: Muerte violenta de las mujeres por razones de género, tipificada en el artículo 325 del Código Penal Federal de México. Constituye el evento desencadenante principal que el flujo de recolección rastrea de forma automatizada en medios periodísticos nacionales y regionales.

Fine-Tuning (Ajuste fino supervisado): Proceso mediante el cual los pesos de un modelo preentrenado masivo se actualizan sobre un corpus etiquetado del dominio objetivo, especializando sus predicciones. En el sistema se implementa mediante *Linear Probing* donde se congelan los pesos del encoder BETO y únicamente se entrena la cabeza de clasificación lineal (solo 2,307 parámetros de ~110 millones) lo que reduce el consumo de memoria de gradientes.

FTS / Full-Text Search (Búsqueda de texto completo): Motor de recuperación de información implementado en PostgreSQL mediante los tipos de datos `tsvector` (representación preprocesada del documento) y `tsquery` (representación de la consulta), con índices GIN. Aplica lematización Snowball para español lo que reduce “*feminicidios*”, “*feminicida*” y “*feminicidio*” a la misma raíz léxica.

GIN (Generalized Inverted Index): Estructura de indexación de PostgreSQL que almacena un índice invertido de lemas para cada lexema único en el corpus, el índice GIN mantiene una lista de punteros directos a todas las filas que lo contienen. Resuelve consultas FTS en $O(\log n)$ (búsqueda en el árbol B del índice) en lugar del $O(n)$ de un escaneo secuencial. Se crea automáticamente sobre las columnas `tsvector` mediante un trigger SQL en cada INSERT/UPDATE.

HDBSCAN: Algoritmo de agrupamiento por densidad variable que elimina la dependencia de un umbral de densidad global ϵ mediante la construcción de una jerarquía completa de clústeres seleccionada por el criterio de Exceso de Masa (EoM). Las noticias no asignables a ningún clúster reciben la etiqueta de outlier (`topic_id = -1`), elevando su umbral de clasificación en BETO de 0.50 a 0.65.

JA3 Fingerprinting: Método de identificación criptográfica basado en la secuencia de parámetros anunciados por un cliente durante el saludo TLS (*handshake*) que son el suite de cifrado, extensiones TLS y curvas elípticas. El hash JA3 resultante identifica unívocamente la librería SSL del cliente (Python/requests vs. Chrome/Windows) lo que permite a los WAF de grado empresarial bloquear scrapers aunque declaren un User-Agent legítimo. StealthSession neutraliza este mecanismo mediante cloudscraper.

LLM (Large Language Model): Modelo de lenguaje de gran escala basado en arquitectura Transformer de muy alta capacidad (miles de millones a billones de parámetros) que es entrenado sobre corpus masivos de texto. A diferencia de BETO (*encoder-only*) los LLM son modelos *decoder* capaces de generar texto libre condicionado a una instrucción (*prompt*). En el sistema Gemini Flash (Google DeepMind) actúa como LLM del módulo DeepInvestigator.

NNA: Acrónimo de **Niñas, Niños y Adolescentes**. Sujeto focal de la extracción semántica del sistema pues la arquitectura está diseñada específicamente para detectar noticias donde los NNA son víctimas indirectas del feminicidio de su madre, quedando en situación de orfandad.

OSINT (Open Source Intelligence): Metodología de inteligencia basada en la recopilación y síntesis sistemática de información públicamente disponible en fuentes abiertas (prensa, registros públicos, redes sociales, bases de datos abiertas) para producir inteligencia accionable sobre un sujeto o evento. El DeepInvestigator sigue este

patrón que agrega información de múltiples fuentes periodísticas abiertas para construir la ficha de caso de un NNA en situación de orfandad incluyendo datos de contacto institucional (DIF municipal, Fiscalía).

Clasificación Escalonada: Arquitectura híbrida que evalúa la relevancia de una noticia mediante cuatro capas **secuenciales** que operan sobre un score acumulativo. La capa 0 (escudo léxico que aplica bloqueo determinista en $O(1)$ por dominio ajeno), la capa 1 (escudo suave que aplica penalización aditiva $\delta = -0,35$ por contenido legislativo o estadístico), la capa 2 (inferencia BETO Fine-Tuned) y la capa 3 (post-filtro de bonificaciones heurísticas por frases de orfandad explícita). El score final determina si la clasificación es Alta ($S \geq 0,80$), Media ($0,50 \leq S < 0,80$) o No relevante ($S < 0,50$). Resolvió el Efecto Péndulo del prototipo 3.

StealthSession Clase de sesión HTTP que elude las medidas anti-bot de los Web Application Firewalls empresariales (Cloudflare, Akamai) mediante la emulación de fingerprint TLS/JA3 idéntico a Chrome/Windows usando cloudscraper; delays entre peticiones con distribución log-normal (mediana $\approx 12,2$ s) para imitar lectura humana; y circuit breaker por dominio (3 fallos $\rightarrow$ cooldown 300 segundos). Respeta las directivas del protocolo robots.txt en cada dominio nuevo.

UMAP (Uniform Manifold Approximation and Projection): Algoritmo de reducción de dimensionalidad que proyecta los embeddings BETO de $\mathbb{R}^{768}$ a $\mathbb{R}^5$, preservando así la estructura topológica local y global mediante optimización sobre grafos de vecindad. La configuración `metric='cosine'` y `min_dist=0.0` maximiza la densidad de los clústeres para facilitar la separación posterior por HDBSCAN.

Víctima Indirecta: Persona dependiente económicamente o con lazos de primer grado de consanguinidad respecto a la víctima directa de un feminicidio. En el contexto del sistema los NNA hijos de la víctima constituyen las víctimas indirectas cuya detección e identificación es el objetivo central del flujo de análisis semántico.

WAF (Web Application Firewall): Sistema de seguridad perimetral que filtra y monitorea el tráfico HTTP/HTTPS entre un cliente e internet, bloqueando peticiones que identifica como automatizadas o maliciosas mediante técnicas de fingerprinting TLS (JA3), análisis de cabeceras HTTP, detección de patrones de tasa de petición y resolución de desafíos JavaScript (CAPTCHA). Los WAF de grado empresarial más comunes en medios periodísticos mexicanos son Cloudflare y Akamai.

El presente Trabajo Terminal registrado bajo el número de protocolo 2026-A135, constituye la segunda fase de un proyecto de investigación aplicada que aborda una problemática crítica en la intersección de los derechos humanos, la violencia de género y la ingeniería de sistemas inteligentes: la invisibilidad estadística y la ausencia de seguimiento sistemático de las Niñas, Niños y Adolescentes (NNA) en situación de orfandad por feminicidio en México. Mediante la conceptualización, diseño, desarrollo y despliegue de un sistema inteligente de monitoreo de medios, el proyecto propone una solución tecnológica para mitigar la dispersión de datos que obstaculiza la labor de las organizaciones civiles y de las instituciones gubernamentales en la procuración de justicia y la reparación integral del daño.

1.1. Presentación del proyecto

México atraviesa una crisis sistémica de violencia feminicida de proporciones documentadas. De acuerdo con los registros del Secretariado Ejecutivo del Sistema Nacional de Seguridad Pública (SESNSP) y la Secretaría de Gobernación, entre los años 2010 y 2023 se han contabilizado **más de 8,000 víctimas** de feminicidio en el territorio nacional. Esta cifra ya de por sí alarmante, oculta una dimensión adicional del daño: el impacto colateral sobre los hijos e hijas de las víctimas, quienes quedan en situación de orfandad repentina y traumática.

La Red por los Derechos de la Infancia en México (REDIM) estima que existen **al menos 3,500 Niñas, Niños y Adolescentes** que han perdido a su madre como consecuencia directa de un feminicidio. Pese a la gravedad de esta cifra no existe a nivel nacional un padrón oficial, un censo unificado ni un mecanismo de seguimiento estandarizado que documente el estado de estas víctimas indirectas. La Convención sobre los Derechos del Niño y la Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA) establecen el principio del *interés superior de la niñez* como eje que orienta, garantizando el derecho a la protección, la salud, la educación y la reparación integral. No obstante la ausencia de datos estructurados imposibilita la materialización efectiva de estos derechos.

El TT no. 2026-A135 surge como respuesta tecnológica a esta brecha. El sistema automatiza el ciclo completo de descubrimiento informativo desde la recolección de noticias en fuentes periodísticas de acceso público hasta la clasificación semántica y la persistencia relacional de los casos identificados, esto con el objetivo de que ningún NNA en situación de orfandad por feminicidio permanezca invisible para las instituciones que tienen la obligación legal de protegerlo.

1.2. Planteamiento del problema

La información sobre los NNA en situación de orfandad por feminicidio se encuentra fragmentada en el ecosistema informativo mexicano. Los datos relevantes sobre la existencia, el estado legal y las condiciones de resguardo de los hijos de una víctima suelen estar dispersos en notas periodísticas de medios locales, comunicados aislados de fiscalías estatales y registros internos de organizaciones no gubernamentales que no se interconectan entre sí. La información existe pero no es accesible de forma sistemática ni procesable a escala computacional.

1.2.1. La inviabilidad del monitoreo manual

La mitigación de esta invisibilidad estadística recae actualmente sobre organizaciones de la sociedad civil como la Fundación "Futuro con Derechos".^{el} proyecto cartográfico de la investigadora María Salguero y la REDIM. La metodología subyacente a este trabajo de documentación presenta un cuello de botella operativo: la **revisión manual y artesanal de fuentes hemerográficas**. El proceso exige que analistas y voluntarios dediquen muchas horas semanales a la revisión individual de portales de noticias, buscando menciones colaterales de hijos en narrativas de notas policíacas. Este enfoque es inviable frente al volumen de información que se genera diariamente en el país ya que México cuenta con más de 1,200 medios periodísticos digitales activos, produciendo decenas de miles de artículos diarios.

La latencia introducida por el factor humano limita drásticamente la capacidad de procesamiento, provocando que una enorme cantidad de casos pasen desapercibidos porque es imposible auditar concurrentemente todas las fuentes periodísticas nacionales y regionales. Además este proceso consume los recursos de talento especializado (psicólogos, abogados, trabajadores sociales), desviando su capacidad hacia labores de recolección de datos en lugar de la intervención directa con las víctimas.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Desarrollar un sistema inteligente que sea capaz de automatizar la recolección, clasificación y análisis de información relacionadas con casos de feminicidio, para facilitar la identificación de niños, niñas y adolescentes que se encuentren en una situación de orfandad en consecuencia de los crímenes de violencias. Mediante el uso de Web Scraping y Procesamiento de Lenguaje Natural

1.3.2. Objetivos Específicos

- OE-1** Identificar y seleccionar fuentes públicas confiables que contengan información relacionada con feminicidios y posibles víctimas indirectas.
- OE-2** Diseñar y desarrollar una herramienta automatizada de Web Scraping para la recolección sistemática de datos provenientes de dichas fuentes.
- OE-3** Implementar técnicas de Procesamiento de Lenguaje Natural (PLN) para extraer y estructurar información clave a partir de textos no estructurados.
- OE-4** Diseñar una base de datos estructurada que permita almacenar la información recolectada de manera eficiente y segura.

Justificación y alcance del proyecto

En este capítulo se presenta la justificación social, tecnológica y ética que sustenta el desarrollo del *Sistema Inteligente para la Identificación y Seguimiento de NNA* y se delimita el alcance del sistema, sus limitaciones y la alineación con el protocolo de Trabajo Terminal 2026-A135.

2.1. Justificación social

Entre 2010 y 2023 más de 8,000 mujeres fueron víctimas de feminicidio en México según datos de la Secretaría de Gobernación [?]. Detrás de estas cifras se encuentran miles de niñas, niños y adolescentes (NNA) que quedaron en condición de orfandad. La Red por los Derechos de la Infancia en México (REDIM) estima que al menos 3,500 NNA perdieron a su madre por feminicidio entre 2012 y 2022 [?] sin embargo no existe un censo oficial ni un sistema de seguimiento que permita dimensionar con precisión el alcance de esta crisis.

La información sobre los hijos e hijas de las víctimas se encuentra fragmentada y dispersa pues puede aparecer en una nota periodística local, en el comunicado de una fiscalía estatal o en los archivos internos de alguna organización civil. Sin un mecanismo que conecte estos fragmentos de información resulta prácticamente imposible rastrear qué ha sucedido con esos menores y de si recibieron atención psicológica, si mantienen acceso a educación, si se les garantizó reparación del daño o si fueron canalizados a instituciones como en el DIF.

Algunas de las consecuencias de esta dispersión son:

- **Para las organizaciones civiles:** Organizaciones como *Futuro con Derechos* dedican varias horas semanales a revisar manualmente decenas de sitios de noticias buscando casos donde se mencione a hijos de víctimas de feminicidio pero muchos casos pasan desapercibidos porque es imposible revisar todas las fuentes disponibles con recursos humanos limitados.
- **Para las autoridades:** Sin datos estructurados ni trazables las instituciones gubernamentales carecen de información para evaluar la efectividad de sus programas de atención a víctimas indirectas y para diseñar políticas públicas basadas en evidencia.
- **Para las familias:** Los familiares de las víctimas frecuentemente desconocen sus derechos, las instituciones de apoyo disponibles y los mecanismos de reparación del daño. La falta de un sistema de seguimiento dificulta la coordinación interinstitucional.

Un sistema automatizado que actúe como filtro inicial que procese cientos de noticias, ue identifique las potencialmente relevantes y presentándolas estructuradas permite que las organizaciones dediquen su tiempo al seguimiento de casos concretos en lugar de a la búsqueda manual de información.

2.2. Justificación tecnológica

La revisión manual de medios de comunicación no es viable a escala nacional pues México cuenta con cientos de medios digitales (nacionales, estatales y locales) que publican diariamente noticias relacionadas con feminicidios. La cobertura periodística de cada caso varía significativamente pues algunos eventos son cubiertos por múltiples medios con redacciones diferentes, mientras que otros aparecen exclusivamente en medios regionales de bajo alcance.

Durante el desarrollo del Trabajo Terminal I se probaron tres enfoques de recolección que mostraron la complejidad técnica del problema:

1. **Scraping directo (Prototipo 0):** Fracasó completamente. Los medios modernos emplean protecciones como cloudflare, contenido renderizado con JavaScript y bloqueos HTTP 403, haciendo complicada la extracción directa del HTML.
2. **RSS básico (Prototipo 1):** Funcionó parcialmente con 8 fuentes RSS, recolectando aproximadamente 96 noticias en dos semanas. La cuestión era que la cobertura se limitaba a medios nacionales con feeds públicos.
3. **Triple fuente (Prototipo 2):** Combinó 10 RSS, Google News y scraping histórico, alcanzando 250 noticias únicas. Este enfoque hizo ver que la viabilidad de la recolección multi-fuente existía pero aún mantenía limitaciones en la detección semántica (más del 60 % de falsos positivos con expresiones regulares).

En Trabajo Terminal II se representa una evolución significativa de estas estrategias mediante la integración de tecnologías de procesamiento de lenguaje natural (PLN):

BETO Fine-Tuned (clasificación escalonada): El corazón del sistema es una arquitectura neuro-simbólica de cuatro capas que combina reglas simbólicas explícitas con el modelo Transformer BETO (*bert-base-spanish-wwm-cased*, 110 millones de parámetros). Las capas operan secuencialmente sobre un score acumulativo: la Capa 0 (Escudo Léxico) descarta dominios ajenos en $O(1)$; la Capa 1 (Escudo Suave) penaliza contenido legislativo o estadístico en $\delta = -0,35$; la Capa 2 ejecuta la inferencia BETO Fine-Tuned como modo prioritario; y la Capa 3 bonifica la presencia de frases de orfandad explícita. Esta arquitectura eliminó el *efecto péndulo* identificado en el Prototipo 3.

Investigación profunda (Gemini Flash + DuckDuckGo): Módulo de investigación bajo demanda que ante una noticia de Alta relevancia este genera una ficha de caso estructurada (JSON con 7 campos que son la ubicación, víctimas, NNA afectados, edades, situación actual, medios de contacto y resumen) mediante búsqueda web (DuckDuckGo) y síntesis con el LLM Gemini Flash. El diseño es modular y se activa bajo demanda explícita del analista para evitar el costo de API en el corpus completo.

BERTopic: Sistema de agrupamiento semántico que combina embeddings BETO (de 768 dimensiones), reducción dimensional UMAP (de 5 dimensiones) y clustering de densidad variable HDBSCAN que agrupa las noticias por similitud de significado.

Deduplicación en cascada: Flujo de cuatro algoritmos secuenciales (Hash MD5, Similitud Jaccard, SimHash distancia Hamming, similitud coseno TF-IDF) que eliminó cerca del 25 % del corpus como redundancia periodística.

PostgreSQL con FTS e índices GIN: Base de datos relacional que reemplaza el almacenamiento en archivos CSV lo que habilita transacciones ACID, integridad referencial, búsqueda de texto completo.

Gracias a esta arquitectura el texto periodístico no estructurado se transforma en datos consultables y jerarquizados por lo que el analista recibe una lista priorizada de casos confirmados (Alta relevancia) con su posterior ficha de caso generada por IA, en lugar de un volumen de texto sin procesar que requeriría decenas de horas de revisión manual.

2.3. Justificación ética y moral: Protección de la privacidad de los NNA

2.3.1. El interés superior del niño

El desarrollo de este sistema se enmarca en un principio ético fundamental: **el interés superior del niño**, establecido en el artículo 3 de la Convención sobre los Derechos del Niño de las Naciones Unidas [?] y en el artículo 2 de la Ley General de los Derechos de Niñas, Niños y Adolescentes de México [?]. Este principio establece que cualquier acción que involucre a menores de edad debe priorizar su bienestar, protección y dignidad por encima de cualquier otro interés.

En base a este principio, el sistema ha sido diseñado desde su inicio con una restricción no negociable: **priorizar el manejo cuidadoso y confidencial de la información obtenida de los NNA.**

2.3.2. Naturaleza de la información procesada

Es importante aclarar la naturaleza de los datos que el sistema procesa y la distinción entre lo que el sistema *identifica* y lo que *no identifica*:

Tabla 2.1: Distinción entre información procesada y excluida

El sistema SÍ identifica	El sistema NO identifica
Noticias que mencionan feminicidios	Nombres completos de los NNA afectados
Menciones genéricas a NNA como víctimas indirectas (ej. "dejó 3 hijos menores")	Datos personales de los menores (edad exacta, CURP, domicilio)
Contexto del caso: ubicación geográfica, fecha, instituciones involucradas	Identidad verificada de ningún menor
Nombres de víctimas adultas (cuando aparecen en la nota periodística)	Fotografías ni datos biométricos de NNA
Patrones y tendencias estadísticas	Información no publicada en medios

2.3.3. ¿Por qué el sistema no proporciona datos completos de NNA?

La ausencia de datos completos de los NNA en los resultados del sistema **no es una limitación técnica, sino una decisión ética respaldada por un marco normativo y periodístico sólido:**

1. **Los medios de comunicación protegen la identidad de los menores.** El Código de Ética Periodística de la UNESCO y los códigos editoriales de los principales medios mexicanos establecen que la identidad de los NNA víctimas (directas o indirectas) debe ser resguardada. Por esta razón las notas periodísticas que constituyen la fuente primaria del sistema rara vez incluyen los datos completos de los menores. En su lugar, utilizan referencias genéricas: "sus tres hijos menores de edad", "una niña de 5 años", "los menores quedaron bajo custodia de la abuela". Esta práctica periodística responsable se refleja necesariamente en los datos que el sistema puede extraer.

2. **La Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA).** El artículo 76 de la LGDNNA prohíbe la difusión de datos personales de NNA que permitan su identificación cuando estos se encuentran en situación de vulnerabilidad. Los NNA que han perdido a su madre por feminicidio se encuentran en una de las situaciones de mayor vulnerabilidad contempladas por la ley. Cualquier sistema que pretendiera identificarlos nominalmente estaría en violación directa de esta disposición legal.

2.4. Alcance del sistema

2.4.1. Alcance funcional

El sistema desarrollado abarca las siguientes funcionalidades que fueron implementadas progresivamente a lo largo de cuatro prototipos (P1-P4):

- **Recolección automatizada multi-fuente (CU-01):** Extracción de noticias desde fuentes RSS nacionales y de género, consultas a Google News y scraping HTML complementario mediante *StealthSession* (emulación de fingerprint TLS, delays log-normales y *Circuit Breaker* por dominio).
- **Deduplicación en cascada (CU-01):** Eliminación automática de noticias duplicadas o cuasi-duplicadas mediante un flujo de cuatro fases (Hash MD5, Jaccard, SimHash Hamming y Coseno TF-IDF).
- **Clasificación semántica escalonada (CU-01):** Arquitectura neuro-simbólica de cuatro capas (escudo léxico, escudo suave, inferencia BETO Fine-Tuned, post-filtro de bonificaciones) que produce clasificaciones alta, media o no relevante.
- **Agrupamiento semántico BERTopic (CU-01):** Agrupación del corpus por similitud de significado mediante UMAP + HDBSCAN + c-TF-IDF sobre embeddings BETO de 768 dimensiones reducidos a 5D.
- **Investigación profunda bajo demanda (CU-03):** Módulo *DeepInvestigator* que genera una ficha de caso estructurada (JSON con 7 campos) sobre cualquier noticia de Alta relevancia mediante búsqueda web y síntesis con Gemini Flash.
- **Detección manual por enlace externo (CU-04):** El analista puede introducir una URL periodística no capturada por el flujo automático, el sistema extrae su contenido, valida el cuerpo periodístico con Gemini Flash y genera la ficha de caso correspondiente.
- **Almacenamiento relacional con FTS (CU-01):** Persistencia en PostgreSQL con esquema normalizado, transacciones ACID, búsqueda de texto completo con índices GIN y trazabilidad por lote de análisis.
- **Dashboard interactivo con autenticación (CU-06, CU-07):** Interfaz web Flask con inicio de sesión (Flask-Login), gestión completa de fuentes periodísticas (CRUD + sondeo técnico previo), búsqueda FTS, cuatro gráficas analíticas (temporal, relevancia, geográfica y por fuente), paginación y exportación CSV.
- **Seguimiento y exportación de casos (CU-05):** Lista de monitoreo activo de casos investigados que es exportable con los campos del JSON de investigación en columnas individuales.

2.4.2. Alcance geográfico y temporal

Geográfico: Cobertura nacional con énfasis en medios mexicanos. Incluye medios nacionales (La Jornada, Proceso, Animal Político, El Universal, Milenio, entre otros), medios especializados en género y derechos humanos (CIMAC Noticias, Luchadoras) y medios regionales (El Sol de Toluca, Diario de Xalapa, Noroeste, entre otros).

Temporal: El sistema procesa noticias desde enero de 2025 hasta la fecha actual (mayo 2026) con capacidad de extensión mediante el módulo de scraping histórico.

2.5. Limitaciones y exclusiones

2.5.1. Exclusiones explícitas del alcance

El sistema no contempla las siguientes funcionalidades:

- No identifica datos completos personales de NNA (justificación ética detallada en la Sección 2.3).
- No realiza intervención social directa.
- No accede a bases de datos gubernamentales cerradas (fiscalías, DIF, registros civiles). Toda la información proviene de fuentes periodísticas públicas.
- No valida la veracidad de las noticias. El sistema asume que las fuentes periodísticas monitoreadas mantienen estándares editoriales mínimos pero no verifica la veracidad de cada nota.
- No implementa Named Entity Recognition (NER) completo con fine-tuning. Esta funcionalidad se identifica como trabajo futuro.

2.5.2. Limitaciones técnicas

1. **Dependencia de fuentes periodísticas:** La cobertura del sistema está limitada a lo que los medios publican. Por esta razón los feminicidios no cubiertos por la prensa o que ocurren en zonas con escasa cobertura mediática no serán detectados.
2. **Precisión del modelo semántico:** El sistema opera en modo *Fine-Tuned* como prioridad (entrenado sobre el corpus etiquetado del dominio) y en modo *Zero-Shot* como fallback ante ausencia del modelo guardado. Bajo el modo Fine-Tuned con la arquitectura clasificación escalonada, se estima una tasa de falsos positivos de ~20 %, frente al 60 % del prototipo 2 (regex) y al rango del 40-50 % del prototipo 3 (Zero-Shot sin clasificación escalonada). La reducción de falsos positivos por debajo del 10 % se identifica como línea de trabajo futuro mediante un ciclo de aprendizaje activo.
3. **Protecciones anti-scraping:** Algunos medios implementan protecciones agresivas (CAPTCHAs, rate limiting estricto, paywalls) que pueden impedir la recolección de sus contenidos.
4. **Latencia en tiempo real:** El sistema opera con ciclos de recolección programados (cada 6-24 horas) no en tiempo real por lo que existe un desfase entre la publicación de una noticia y su procesamiento.
5. **Idioma:** El sistema está optimizado exclusivamente para noticias en español. Noticias en lenguas indígenas o en inglés no son procesadas.

En este capítulo se establece el fundamento matemático y conceptual que sustenta la arquitectura del Sistema Inteligente para la Identificación y Seguimiento de NNA. Cada sección describe una tecnología implementada en el sistema y justifica su adopción frente a las alternativas disponibles, trazando así una línea directa entre los principios teóricos formales y las decisiones de implementación documentadas.

3.1. Procesamiento de lenguaje natural y representación lingüística

El Procesamiento de lenguaje natural (PLN) comprende el conjunto de técnicas computacionales orientadas a la comprensión, generación y manipulación del lenguaje humano en formato textual. El dominio específico de este proyecto; que son noticias periodísticas mexicanas en español sobre violencia feminicida, exige herramientas de PLN que capturen la semántica contextual del lenguaje, trascendiendo la mera relación de palabras.

3.1.1. Tokenización y el algoritmo WordPiece

La tokenización es el proceso mediante el cual una cadena de texto se divide en unidades elementales de representación (*tokens*) que el modelo puede procesar. Los enfoques clásicos segmentan el texto a nivel de palabra completa lo que genera vocabularios de tamaño inmanejable (más de 500,000 tipos) y deja fuera de vocabulario cualquier término no visto durante el entrenamiento.

El modelo BETO al igual que toda la familia BERT, emplea el algoritmo **WordPiece** [?], el cual es un esquema de segmentación sub-léxica que descompone las palabras en secuencias de sub-unidades frecuentes. El algoritmo construye el vocabulario de forma incremental: inicializa el conjunto con todos los caracteres individuales del corpus y fusiona iterativamente los pares de sub-unidades que maximizan la verosimilitud del corpus de entrenamiento bajo un modelo de lenguaje:

$$\text{Puntuación}(u_1, u_2) = \frac{P(u_1 u_2)}{P(u_1) \cdot P(u_2)} \quad (3.1)$$

Donde u_1 y u_2 son sub-unidades adyacentes y $P(\cdot)$ denota la probabilidad del sub-token en el corpus. Las fusiones con mayor puntuación se aplican repetidamente hasta alcanzar el tamaño de vocabulario objetivo (30,000 tokens para

BETO). Los sub-tokens que continúan una palabra se prefijan con ## para distinguirlos de tokens iniciales, por ejemplo: "feminicidio" se tokeniza como fem##ini##cid##io.

Este esquema tiene dos consecuencias técnicas que son fundamentales para el dominio del sistema. Primero, las palabras desconocidas como neologismos o nombres propios se descomponen en sub-tokens conocidos en lugar de ser descartadas, para así preservar parcialmente su información. Segundo, el entrenamiento con *Whole Word Masking* (WWM) garantiza que todos los sub-tokens de una misma palabra se enmascaren conjuntamente, forzando al modelo a aprender representaciones semánticamente coherentes de términos especializados de múltiples sub-unidades.

Además BETO utiliza dos tokens especiales de estructura: [CLS] (primer token de toda secuencia, cuyo vector de salida representa a la secuencia completa para tareas como clasificación) y [SEP] (que es mas como un separador entre dos segmentos de texto distintos).

3.1.2. Lematización y normalización lexicológica

La lematización es el proceso de reducir una forma flexionada de una palabra a su lema canónico (forma base del diccionario), considerando el contexto morfosintáctico. Se distingue de la *stemming* en que produce formas lingüísticamente válidas: "feminicidios" → "feminicidio" (lematización), no "feminicid" (stemming).

En el sistema, la lematización se emplea en la capa de preprocesamiento del módulo de búsqueda por sinónimos (SynonymDictionary) y en el vectorizador TF-IDF de deduplicación, donde la normalización morfológica reduce la dispersión léxica del vocabulario y mejora la robustez de las métricas de similitud ante variaciones de tiempo verbal, número y género gramaticales comunes en el periodismo mexicano. PostgreSQL aplica su propio proceso de lematización para español (diccionario Snowball) al momento de construir el tsvector para la búsqueda de texto completo, cubriendo el caso de recuperación de información en el dashboard.

3.1.3. Modelo de espacio vectorial: TF-IDF

La representación vectorial clásica de documentos de texto se articula mediante el modelo *Term Frequency - Inverse Document Frequency* (TF-IDF) [?]. El peso asignado a un término t en un documento d dentro de una colección D se calcula como el producto de dos factores:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (3.2)$$

Donde la frecuencia de término $\text{TF}(t, d)$ mide la proporción de ocurrencias del término en el documento:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (3.3)$$

Y la frecuencia inversa de documento $\text{IDF}(t, D)$ penaliza los términos ubicuos en la colección (que aportan poca información discriminativa) y premia los términos raros:

$$\text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}| + 1} \quad (3.4)$$

Este sistema implementa una variante de TF-IDF que multiplica el peso de términos del glosario de dominio (por ejemplo: "orfandad", "DIF", "resguardo") por un factor amplificador $\beta > 1$ que incrementa artificialmente su IDF efectivo. Esta modificación operacionaliza el conocimiento experto del dominio en el espacio vectorial sin requerir reentrenamiento. TF-IDF se emplea en el sistema exclusivamente para las fases de deduplicación y el módulo de sinónimos; la clasificación de relevancia utiliza exclusivamente los embeddings BETO.

3.1.4. Arquitectura Transformer y mecanismo de auto-atención

La arquitectura *Transformer* [?] constituye el fundamento de todos los modelos de lenguaje de última generación. A diferencia de las Redes Neuronales Recurrentes (RNN) y las LSTM, que procesan la secuencia de tokens de forma estrictamente lineal, generando un estado oculto h_t en función del estado h_{t-1} y el token actual, el Transformer prescinde de la recurrencia. Todas las posiciones de la secuencia se procesan en paralelo mediante el mecanismo de auto-atención multi-cabeza (*Multi-Head Self-Attention*).

Para cada token x_i de la secuencia, el mecanismo de atención computa tres vectores: una consulta $Q_i = x_i W^Q$, una clave $K_i = x_i W^K$ y un valor $V_i = x_i W^V$, donde $W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$ son matrices de proyección aprendidas. La relevancia del token j para el token i se calcula como el producto punto de sus vectores de consulta y clave, escalado para evitar saturación del gradiente en la función softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.5)$$

Donde d_k es la dimensión de las claves. La salida para el token i es una suma ponderada de todos los vectores V_j de la secuencia, con pesos determinados por la similitud de atención. Esto permite que "menor" en la posición i reciba contribuciones directas de resguardo del DIF. en la posición j , sin importar la distancia lineal entre ambas posiciones.

La auto-atención multi-cabeza ejecuta h instancias paralelas del mecanismo de atención, cada una con sus propias proyecciones W_i^Q, W_i^K, W_i^V , y concatena sus salidas.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3.6)$$

Donde cada cabeza captura un tipo diferente de relación semántica o sintáctica. BERT-base emplea $h = 12$ cabezas en cada una de sus 12 capas de transformación.

3.1.5. BETO: representaciones bidireccionales para el español

BERT (*Bidirectional Encoder Representations from Transformers*) [?] extiende la arquitectura Transformer original mediante el preentrenamiento bidireccional: en lugar de predecir el siguiente token (modelado de lenguaje causal como GPT) BERT se entrena con el objetivo de **Modelado de Lenguaje Enmascarado** (MLM). En cada secuencia de entrenamiento el 15 % de los tokens se enmascara aleatoriamente y el modelo debe predecir el token original usando el contexto *bidireccional completo* (los tokens a la izquierda y a la derecha de la máscara). Este objetivo fuerza al modelo a desarrollar representaciones profundamente contextuales.

BETO [?] (bert-base-spanish-wwm-cased) es la instancia monolingüe de BERT preentrenada exclusivamente en español, con la siguiente arquitectura base:

- 12 capas Transformer (encoders)
- 12 cabezas de atención por capa
- Dimensión del espacio oculto: $d_{model} = 768$
- 110 millones de parámetros entrenables
- Vocabulario WordPiece de 30,000 tokens en español

El corpus de preentrenamiento incluye wikipedia en español (aproximadamente 3 GB), OpenSubtitles, ParaCrawl, MultiUN (corpus OPUS) y fuentes de prensa latinoamericana. Esta composición es crítica para el dominio del sistema pues el periodismo latinoamericano posee un estilo de redacción y un léxico que difieren del español peninsular. Los corpus de prensa incluidos en el preentrenamiento de BETO sesgan sus representaciones hacia el vocabulario y las

construcciones sintácticas empleadas por los medios de comunicación mexicanos, lo cual mejora la calidad de los embeddings para el dominio específico del sistema.

La representación semántica de una noticia se extrae del *hidden state* del token especial [CLS] en la última capa del encoder que es un vector de 768 dimensiones que condensa el significado global de la secuencia completa. Este vector se normaliza mediante norma L2 para habilitar la comparación por similitud coseno:

$$\hat{e} = \frac{e}{\|e\|_2}, \quad e = \text{BETO}(\text{texto})[0, :] \quad (3.7)$$

Donde $e \in \mathbb{R}^{768}$ es el embedding crudo del token [CLS] y $\hat{e}$ es su versión normalizada empleada en la clasificación zero-shot.

3.2. Paradigmas de inferencia y clasificación semántica

3.2.1. Clasificación zero-shot por similitud coseno (modo fallback)

El paradigma de **clasificación zero-shot** [?] permite categorizar documentos en clases definidas en lenguaje natural sin requerir ejemplos etiquetados de entrenamiento para dichas clases. En el sistema, este paradigma opera como **modo de fallback**: se activa automáticamente cuando el clasificador ajustado (Fine-Tuned) no está disponible, garantizando así la continuidad operativa del pipeline en su fase inicial o ante la ausencia del archivo de modelo guardado.

La implementación adoptada es la clasificación por similitud coseno entre embeddings. Se precomputan representaciones BETO para descriptores textuales de las categorías (*“noticia que reporta un caso concreto de feminicidio donde los hijos de la víctima quedaron en orfandad o bajo resguardo institucional”* para la clase *relevante*; *“notas estadísticas, políticas públicas o reporte donde el menor es el agresor”* para la clase *no relevante*). En tiempo de inferencia, el embedding del texto $\hat{e}_{\text{texto}}$ se compara contra los embeddings de cada categoría $\hat{e}_c$ mediante el producto punto (equivalente a similitud coseno, dado que los vectores están normalizados en ℓ_2):

$$s_c = \hat{e}_{\text{texto}} \cdot \hat{e}_c = \cos(\hat{e}_{\text{texto}}, \hat{e}_c) \quad (3.8)$$

La ventaja principal de este paradigma es que el conocimiento experto del dominio se codifica en lenguaje natural auditable y modificable sin reentrenamiento. Su limitación, identificada empíricamente durante el Prototipo 3 como el *Efecto Péndulo*, es que los scores tienden a concentrarse en el rango [0,40,0,70], reduciendo la discriminabilidad del clasificador entre noticias legislativas y casos fácticos de orfandad con vocabulario solapado.

3.2.2. Ajuste fino supervisado (Fine-Tuning) con Linear Probing

El **ajuste fino** (Fine-Tuning) [?] de un modelo preentrenado consiste en continuar el proceso de entrenamiento sobre un corpus etiquetado del dominio objetivo, actualizando los pesos del modelo para especializar sus representaciones hacia las características semánticas específicas de la tarea. En el sistema, esta es la modalidad de inferencia **prioritaria**: el módulo `semantic_detector.py` verifica al inicializarse si existe un clasificador guardado y, de haberlo, lo carga como motor principal; en caso contrario, activa el modo zero-shot como fallback.

Dado que el reentrenamiento completo de los 110 millones de parámetros de BETO exige capacidades de cómputo (GPU de alto rendimiento, memoria de video de al menos 16 GB) incompatibles con el entorno de desarrollo del proyecto (CPU con 8 GB de RAM), el sistema implementa la estrategia de **Linear Probing**: se congelan (`requires_grad = False`) todos los bloques Transformer del encoder BETO y únicamente se entrena la capa lineal de clasificación superpuesta (la *cabeza de clasificación*):

$$\hat{y} = \text{softmax}(W_{clf} \cdot \hat{e}_{[\text{CLS}]} + b_{clf}) \quad (3.9)$$

Donde $W_{clf} \in \mathbb{R}^{C \times 768}$ es la matriz de pesos de la cabeza de clasificación, $b_{clf} \in \mathbb{R}^C$ es el vector de sesgo, C es el número de clases (Alta, Media, No relevante) y $\hat{e}_{[CLS]} \in \mathbb{R}^{768}$ es el embedding del token [CLS] extraído de la última capa del encoder congelado. El número de parámetros entrenables se reduce de 110 millones a aproximadamente $768 \times C + C \approx 2,307$ parámetros, reduciendo el consumo de memoria de gradientes en $\sim 99.99\%$.

3.2.3. Acumulación de Gradientes (*Gradient Accumulation*)

El **Gradient Accumulation** [?] es una técnica que permite simular el entrenamiento con lotes de tamaño grande (necesario para estabilidad del descenso del gradiente) en equipos con memoria RAM limitada, sin realizar el paso de actualización de pesos (`optimizer.step()`) en cada lote individual. El algoritmo acumula los gradientes de k lotes consecutivos antes de aplicar una sola actualización de pesos:

$$\nabla_{\theta}^{\text{acum}} = \frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \mathcal{L}(x_i, y_i) \quad (3.10)$$

Donde $\nabla_{\theta} \mathcal{L}(x_i, y_i)$ es el gradiente de la función de pérdida para el i -ésimo mini-lote y k es el factor de acumulación. El efecto es matemáticamente equivalente a entrenar con un lote de tamaño efectivo $k \times \text{batch_size}$, usando solo la memoria de un lote de tamaño `batch_size` en cada paso.

En el sistema, se emplea `batch_size = 4` con un factor de acumulación $k = 8$, dando un lote efectivo de 32 muestras por actualización, valor estándar para el Fine-Tuning estable de modelos BERT. El proceso de entrenamiento incorpora adicional una estrategia de detención temprana (*Early Stopping*) basada en el F1-score macro sobre el conjunto de validación: si el F1 no mejora en 3 épocas consecutivas (`patience = 3`), el entrenamiento se detiene y se recuperan los pesos del mejor epoch, previniendo el sobreajuste ante el tamaño reducido del corpus etiquetado.

3.3. Agrupamiento semántico y reducción de dimensionalidad

El agrupamiento semántico de documentos periodísticos tiene como objetivo identificar grupos de noticias que pertenecen al mismo tema independientemente del vocabulario empleado por cada medio. El sistema adopta **BERTopic** [?] como motor de agrupamiento, el cual su arquitectura modular encadena tres algoritmos independientes: UMAP para reducción de dimensionalidad, HDBSCAN para detección de clústeres de densidad variable y c-TF-IDF para la representación léxica de cada tópico descubierto.

3.3.1. UMAP: Proyección Topológica Preservadora de Estructura

Los *embeddings* generados por BETO están en $\mathbb{R}^{768}$. Si se aplica directamente algoritmos de agrupamiento sobre espacios de esta dimensionalidad, se produce resultados de baja calidad debido a la **maldición de la dimensionalidad** [?]: en espacios de alta dimensión, la razón entre la distancia al vecino más lejano y al vecino más cercano tiende a 1 asintóticamente, haciendo que las distancias euclidianas pierdan poder discriminatorio. En el límite, todos los puntos son aproximadamente equidistantes entre sí, privando a los algoritmos de agrupamiento de la señal de proximidad.

UMAP (*Uniform Manifold Approximation and Projection*) [?] proyecta los datos de alta dimensión a un espacio de dimensión reducida ($\mathbb{R}^5$ en el sistema) conservando la estructura topológica local y global. El algoritmo se fundamenta en tres principios de geometría Riemanniana y topología algebraica:

1. **Construcción del grafo topológico en alta dimensión.** Para cada punto x_i , se construye un grafo de vecindad ponderado G^{HD} donde el peso de la arista (x_i, x_j) representa la probabilidad de que x_j sea un vecino de x_i bajo una métrica Riemanniana local calibrada por la distancia al k -ésimo vecino más cercano de x_i . El parámetro `n_neighbors=15` controla este radio de vecindad local.

2. **Construcción del grafo en baja dimensión.** Se inicializa un grafo G^{LD} análogo en $\mathbb{R}^5$, donde las aristas se ponderan mediante una función de distribución de densidad de probabilidad diferente (familia t -Student para evitar el crowding problem de t-SNE).
3. **Optimización por entropía cruzada.** Los pesos de G^{LD} se optimizan para minimizar la entropía cruzada difusa respecto a G^{HD} :

$$\mathcal{L} = \sum_{(i,j)} \left[w_{ij}^{HD} \log \frac{w_{ij}^{HD}}{w_{ij}^{LD}} + (1 - w_{ij}^{HD}) \log \frac{1 - w_{ij}^{HD}}{1 - w_{ij}^{LD}} \right] \quad (3.11)$$

La minimización de $\mathcal{L}$ mediante descenso de gradiente estocástico garantiza que la disposición en el espacio reducido preserve las relaciones de vecindad del espacio original.

La configuración `metric='cosine'` es crítica para el dominio de texto: los embeddings de lenguaje tienen magnitudes variables, y la similitud semántica se mide por el ángulo entre vectores, no por su distancia euclidiana. El parámetro `min_dist=0.0` comprime al máximo los clústeres, aumentando la densidad local y facilitando la separación de fronteras por HDBSCAN.

3.3.2. HDBSCAN: Agrupamiento Jerárquico por Densidad Variable

HDBSCAN (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) [?] extiende DBSCAN mediante la construcción de una jerarquía completa de clústeres, eliminando la dependencia de un parámetro global de densidad ϵ . El algoritmo opera en cuatro etapas:

1. **Distancia de alcanzabilidad mutua.** Para un parámetro `min_samples = m`, la distancia de alcanzabilidad mutua entre dos puntos x_i y x_j se define como:

$$d_{mreach}(x_i, x_j) = \max(\text{core}_m(x_i), \text{core}_m(x_j), d(x_i, x_j)) \quad (3.12)$$

donde $\text{core}_m(x_i)$ es la distancia del punto x_i a su m -ésimo vecino más cercano. Esta transformación suaviza regiones de baja densidad, separando los clústeres reales del ruido.

2. **Árbol de expansión mínima (MST).** Se construye el MST del grafo ponderado por distancias de alcanzabilidad mutua entre todos los pares de puntos.
3. **Jerarquía de clústeres.** El MST se convierte en una jerarquía de clústeres eliminando aristas en orden decreciente de peso (equivalente a reducir el umbral de densidad progresivamente). El dendrograma resultante codifica la estructura de clústeres a todas las escalas de densidad.
4. **Selección por Exceso de Masa (EoM).** El método *Excess of Mass* selecciona el subconjunto de clústeres en la jerarquía que maximiza la “persistencia” (estabilidad topológica medida como la diferencia entre el nivel de densidad de aparición y desaparición del clúster). Los puntos que no pertenecen a ningún clúster persistente reciben la etiqueta de ruido: `topic_id = -1`.

La etiqueta de ruido es semánticamente valiosa para el sistema: las noticias *outlier* que no comparten coherencia temática con ningún clúster del corpus son tratadas con mayor escepticismo en la lógica de decisión neuro-simbólica, elevando su umbral de clasificación en BETO de 0.50 a 0.65.

3.3.3. c-TF-IDF: Relevancia de Términos por Clúster

Una vez agrupados los documentos, BERTopic extrae la representación léxica de cada tópico mediante *Class-based TF-IDF* (c-TF-IDF) [?]. El algoritmo trata cada clúster como un “mega-documento de clase” formado por la concatenación de todos sus documentos miembro, y calcula la importancia de cada término t en la clase c frente al corpus completo:

$$W_{t,c} = TF_{t,c} \times \log\left(1 + \frac{A}{TF_t}\right) \quad (3.13)$$

Donde $TF_{t,c}$ es la frecuencia del término t en el mega-documento del clúster c , TF_t es la frecuencia total del término en todos los clústeres, y A es el número promedio de palabras por clúster (factor de normalización de escala). Los términos con $W_{t,c}$ más alto son frecuentes en el clúster pero infrecuentes en el resto del corpus: son los que mejor caracterizan semánticamente ese tópico.

El vectorizador configurado con `ngram_range=(1, 3)` permite que las representaciones incluyan bigramas y trigramas, capturando expresiones compuestas de alta especificidad como “*hijos resguardados DIF*” o “*víctima de feminicidio*” que los unigramas individuales no pueden representar. La representación final por tópico mediante KeyBERTInspired refina los términos seleccionados usando similitud coseno entre el embedding del n-grama y el embedding promedio del clúster, garantizando representatividad semántica en lugar de meramente estadística.

3.4. Técnicas de Deduplicación Cross-Site

La arquitectura multi-fuente del sistema (45 RSS + Google News + scraping HTML) genera redundancia mediática: distintos medios cubren el mismo evento con redacciones parcialmente distintas. Sin deduplicación, el corpus infla artificialmente las estadísticas y puede generar alertas duplicadas en el dashboard. El sistema implementa una cascada de cuatro algoritmos de complejidad creciente, donde los casos más obvios se resuelven con el método más barato.

3.4.1. Hash Exacto MD5

La primera barrera es la detección de copias exactas. Se emplea MD5 (*Message-Digest Algorithm 5*) [?], una función de hash criptográfico que proyecta una cadena de longitud arbitraria a un valor de 128 bits. El efecto avalancha garantiza que cualquier diferencia mínima en el texto de entrada altera completamente el hash resultante. La comparación de hashes tiene complejidad $O(1)$ por par, haciendo de esta fase la más eficiente de la cascada.

3.4.2. Similitud de Jaccard sobre Conjuntos de Tokens

Para detectar reescrituras con sustituciones léxicas simples (“*fue asesinada*” ↔ “*perdió la vida*”), se emplea el coeficiente de Jaccard [?]. Dado que los títulos de las noticias se tokenizan en conjuntos de palabras A y B , la similitud se define como:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1] \quad (3.14)$$

Un umbral $J \geq 0,70$ marca a los pares como cuasi-duplicados. La complejidad por par es $O(|A| + |B|)$, lineal en el tamaño del vocabulario de los títulos.

3.4.3. SimHash: Huellas Digitales Localmente Sensibles

SimHash [?] es un *Locality-Sensitive Hash* (LSH) que genera una huella digital de 64 bits por documento tal que documentos similares producen hashes con **distancia de Hamming** baja (número de bits diferentes). El algoritmo

proyecta los pesos TF-IDF de los tokens del documento en $\{+1, -1\}^{64}$ mediante 64 funciones de hash independientes, y el bit b_k del SimHash se determina por el signo del promedio ponderado de las proyecciones:

$$b_k = \text{sgn}\left(\sum_{t \in d} w_t \cdot h_k(t)\right), \quad k = 1, \dots, 64 \quad (3.15)$$

Dos documentos con distancia de Hamming ≤ 6 bits se consideran cuasi-duplicados. SimHash escala a millones de documentos en $O(n)$ sin comparaciones par-a-par, siendo el algoritmo adecuado para la fase de detección de paráfrasis en corpora de gran volumen.

3.4.4. Similitud Coseno sobre Vectores TF-IDF

La fase final de la cascada aplica similitud coseno sobre los vectores TF-IDF completos del contenido de las noticias, capturando casos donde la redacción difiere sustancialmente pero el contenido fáctico es el mismo. Dado que los vectores TF-IDF $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{|V|}$ son inherentemente normalizados en ℓ_2 , la similitud coseno se reduce al producto punto:

$$\cos(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2} = \sum_{i=1}^{|V|} \hat{A}_i \hat{B}_i \in [-1, 1] \quad (3.16)$$

Un umbral $\cos \geq 0,85$ marca el par como duplicado. Este paso es $O(n^2)$ en el peor caso pero, gracias al pre-filtrado de las fases anteriores, solo se aplica a un subconjunto pequeño de pares candidatos.

3.5. Tecnologías de Extracción de Datos y Persistencia

3.5.1. Protocolos de Sindicación: RSS y Atom

RSS (*Really Simple Syndication*) [?] y Atom son estándares de sindicación web basados en XML que permiten a los sitios publicar resúmenes estructurados de su contenido más reciente para su consumo por agentes automatizados. Un feed RSS 2.0 típico encapsula 20–50 artículos con sus metadatos (título, enlace, resumen, fecha, autor) en un documento XML de < 50 KB, en contraste con los 500 KB–2 MB de una página HTML completa con recursos estáticos.

La estructura del elemento raíz `<channel>` y sus `<item>` hijos es un estándar inmutable: el esquema XML no cambia ante actualizaciones del diseño del portal, eliminando la fragilidad de los selectores CSS del HTML scraping. Los endpoints RSS son recursos públicos destinados explícitamente a la ingestión automatizada; los WAF no aplican *fingerprinting* TLS sobre ellos, reduciendo la fricción de recolección a prácticamente cero para las 45 fuentes RSS del sistema.

3.5.2. Evasión de WAF: StealthSession y TLS/JA3 Fingerprinting

Los *Web Application Firewalls* (WAF) de grado empresarial —Cloudflare, Akamai, PerimeterX— identifican y bloquean scrapers mediante **TLS/JA3 fingerprinting**: durante el saludo TLS (*TLS handshake*), el cliente anuncia sus capacidades de cifrado (cipher suites, extensiones TLS, curvas elípticas) en un orden determinístico que identifica unívocamente la librería SSL utilizada. El hash JA3 de este saludo es diferente para Python/requests versus Chrome/Windows, aunque ambos declaren el mismo User-Agent. Un WAF moderno detecta la asimetría y emite HTTP 403 sin revelar el motivo.

StealthSession v7.0 resuelve este problema mediante *ccloudscraper*, una librería que reemplaza la pila TLS de Python con una que genera hashes JA3 idénticos a los de Chrome/Windows, haciendo el handshake criptográficamente indistinguible de un navegador real. Complementariamente, el sistema implementa:

- **Delays log-normales.** Los intervalos entre requests siguen una distribución log-normal $\mathcal{LN}(\mu = 2,5, \sigma = 0,7)$ con mediana $\approx 12,2$ s y cola derecha larga, imitando los patrones de lectura humana (distribución asimétrica con pausas ocasionales largas) frente a la distribución uniforme trivialmente detectable.
- **Circuit Breaker por dominio.** Después de 3 fallos consecutivos sobre un dominio, el circuito se “abre” y rechaza requests por 300 s, evitando el ban permanente de IP al no presionar repetidamente un sitio que está bloqueando activamente la sesión.
- **Protocolo robots.txt.** El sistema respeta las directivas del estándar [?], consultando el archivo antes de cada dominio nuevo y omitiendo los paths restringidos para mantener la extracción dentro del marco ético del OSINT.

3.5.3. Propiedades ACID de postgresql

PostgreSQL [?] implementa el modelo transaccional ACID (*Atomicity, Consistency, Isolation, Durability*) mediante el protocolo de Control de Concurrencia Multiversión (MVCC):

Atomicidad. Toda transacción se ejecuta de forma atómica: o se aplican todas las operaciones que la componen (INSERT, UPDATE, DELETE) o ninguna. Una falla en mitad de una inserción por lote no corrompe los datos previamente persistidos.

Consistencia. La base de datos transiciona de un estado válido a otro estado válido. Las restricciones de integridad referencial (FOREIGN KEY) y las restricciones de dominio (NOT NULL, UNIQUE) se verifican al completar cada transacción.

Aislamiento. MVCC permite que múltiples transacciones concurrentes operen sin bloquearse mutuamente: cada transacción opera sobre su propia instantánea (*snapshot*) consistente de la base de datos. El pipeline analítico y el dashboard Flask pueden leer y escribir simultáneamente sin condiciones de carrera.

Durabilidad. Una vez que una transacción es confirmada (COMMIT), los cambios se persisten en el *Write-Ahead Log* (WAL) antes de retornar al cliente, garantizando la persistencia ante fallos de sistema o reinicio del servidor.

3.5.4. Índices GIN y Full-Text Search en Español

La búsqueda de texto completo (FTS) en PostgreSQL opera mediante dos tipos de datos especializados: *tsvector* (representación preprocesada del documento) y *tsquery* (representación preprocesada de la consulta del usuario). El proceso de conversión a *tsvector* aplica, en orden:

1. **Tokenización** del texto en lexemas (unidades léxicas significativas).
2. **Eliminación de *stop words*** en español (artículos, preposiciones, conjunciones frecuentes).
3. **Stemming** mediante el algoritmo Snowball para español: reduce “*feminicidios*”, “*feminicida*” y “*feminicidio*” a la misma raíz “*feminicid*”, garantizando que una búsqueda por cualquiera de las formas recupere documentos con las demás.
4. **Asignación de posición léxica** a cada lexema para el cálculo posterior de *ts.rank* (relevancia por proximidad de términos).

Los índices de tipo **GIN** (*Generalized Inverted Index*) [?] almacenan un índice invertido de los lexemas: para cada lexema único en el corpus, el índice GIN mantiene una lista de punteros directos a todas las filas que contienen ese lexema. Una consulta FTS de la forma `to_tsvector('spanish', titulo) @@ to_tsquery('spanish',`

'feminicidio') es resuelta en $O(\log n)$ (búsqueda en el árbol B del índice GIN) en lugar del $O(n)$ de un escaneo secuencial con LIKE '%feminicidio%', donde n es el número de registros en la tabla.

El sistema crea automáticamente el tsvector de cada noticia mediante un *trigger* SQL que se activa en cada INSERT y UPDATE sobre la tabla noticias, concatenando título (peso A, mayor relevancia) y contenido (peso B, menor relevancia) para el ranking de resultados.

3.6. Sistemas Neuro-Simbólicos y Clasificación Escalonada v7.1

La arquitectura neuro-simbólica [?] es un paradigma de inteligencia artificial que combina dos tradiciones complementarias: el razonamiento **neuronal** (aprendizaje de representaciones probabilísticas a partir de datos, generalización por interpolación en espacios de embeddings) y el razonamiento **simbólico** (manipulación explícita de reglas lógicas formales, conocimiento estructurado del dominio, razonamiento determinista y auditable).

Los sistemas neuro-simbólicos surgen como respuesta a las limitaciones individuales de cada paradigma. Los sistemas puramente neuronales son poderosos en generalización pero producen *alucinaciones* en distribuciones fuera del rango de entrenamiento. Los sistemas puramente simbólicos son auditables y correctos por construcción, pero no escalan ante vocabulario abierto y lenguaje natural no estructurado.

En el contexto del sistema, el componente neuronal (BETO Fine-Tuned) provee la comprensión semántica del lenguaje periodístico, mientras que el componente simbólico (las cuatro capas de la arquitectura de *Scoring Escalonado* v7.1) garantiza que ninguna predicción viole el conocimiento experto del dominio.

3.6.1. Arquitectura de *Scoring Escalonado* v7.1: las cuatro capas

La arquitectura v7.1 organiza la clasificación semántica en cuatro capas secuenciales que operan sobre un **score acumulativo** $S \in \mathbb{R}$, inicializado en cero al comienzo de la inferencia sobre cada noticia:

Capa 0 — Escudo Léxico (bloqueo absoluto). Evaluación determinista mediante un conjunto congelado de tokens de dominio ajeno (frozenset), con complejidad $O(1)$ por consulta. Si el texto contiene *exclusivamente* términos de dominios fuera del alcance del sistema (deportes, economía, política exterior), la noticia es descartada inmediatamente sin continuar a las capas siguientes. Esta es la capa de mayor especificidad simbólica.

Capa 1 — Escudo Suave (penalización aditiva). Evaluación semántica sobre dominios limítrofes: contenido legislativo (reformas, iniciativas de ley), estadístico (porcentajes nacionales sin caso concreto) o de alcance geográfico extranacional. Si el texto cae en alguna de estas categorías, se aplica una penalización aditiva $\delta = -0,35$ al score acumulativo:

$$S \leftarrow S + \delta, \quad \delta = -0,35 \quad (3.17)$$

A diferencia de la Capa 0, esta penalización no es una exclusión binaria: una noticia legislativa que simultáneamente describe un caso fáctico concreto puede recuperar el score en las capas siguientes.

Capa 2 — Inferencia BETO (score semántico probabilístico). El texto de la noticia se procesa por el clasificador BETO Fine-Tuned (o Zero-Shot como fallback). El logit de la clase positiva, transformado por softmax al rango $[0, 1]$, se añade directamente al score acumulativo:

$$S \leftarrow S + P(\text{relevante} \mid \text{texto}) \quad (3.18)$$

Para que la noticia avance al post-filtro, se requiere $S > 0,50$ al concluir esta capa.

Capa 3 — Post-filtro de validación (bonificaciones heurísticas). Se buscan en el texto un conjunto de patrones lingüísticos de alta especificidad que evidencian orfandad fáctica: expresiones del tipo “*quedó al cuidado de*”, “*el menor*

presenció”, “DIF resguardó”, “hijos quedaron huérfanos”, etc. Cada patrón encontrado añade una bonificación positiva al score:

$$S \leftarrow S + \sum_{p \in P_{enc}} b_p \quad (3.19)$$

donde P_{enc} es el conjunto de patrones encontrados y $b_p > 0$ es el peso de cada bonificación. El score final determina la clasificación definitiva: $S \geq 0,80 \rightarrow$ Alta; $0,50 \leq S < 0,80 \rightarrow$ Media; $S < 0,50 \rightarrow$ No relevante.

Esta arquitectura opera bajo el principio de *corrección por capas*: los filtros simbólicos de certeza máxima (Capa 0) tienen prioridad absoluta; los filtros de penalización (Capa 1) actúan como restricciones suaves; y solo en los casos genuinamente ambiguos el sistema delega la decisión al componente neuronal (Capa 2), cuyo resultado puede ser confirmado o reforzado por evidencia explícita de orfandad (Capa 3).

3.7. Modelos de Lenguaje de Gran Escala y el Módulo DeepInvestigator

Los **Modelos de Lenguaje de Gran Escala** (*Large Language Models*, LLM) [?] son sistemas de IA generativa basados en arquitecturas Transformer de muy alta capacidad (desde miles de millones hasta billones de parámetros) entrenados sobre corpus de texto de escala masiva (cientos de miles de millones de tokens). A diferencia de BETO, que produce únicamente representaciones vectoriales (*encoder-only*), los LLM son modelos *decoder* (o *encoder-decoder*) capaces de generar texto libre condicionado a una instrucción o contexto (*prompt*).

La capacidad de razonamiento de los LLM emerge a partir de cierta escala de parámetros y datos de entrenamiento, una propiedad denominada *emergencia* [?]: modelos de mayor capacidad exhiben comportamientos cualitativamente diferentes a los de modelos más pequeños, incluyendo seguimiento de instrucciones complejas, razonamiento en múltiples pasos (*Chain-of-Thought*) y síntesis estructurada de información de múltiples fuentes.

3.7.1. Síntesis estructurada mediante Gemini Flash

El módulo DeepInvestigator emplea **Gemini Flash** (Google DeepMind) [?], un LLM de alta eficiencia diseñado para tareas de inferencia de latencia reducida. El módulo lo utiliza en dos tareas distintas:

1. **Generación de consulta de búsqueda.** Dado el título y el fragmento inicial del artículo periodístico, el LLM genera una consulta de búsqueda altamente específica (máximo 6 palabras) que maximiza la probabilidad de recuperar fuentes complementarias sobre el mismo caso fáctico. Esta tarea requiere compresión semántica y desambiguación de nombres propios.
2. **Síntesis estructurada en JSON.** Dados los textos multi-fuente recolectados (artículo original + fuentes complementarias scrapeadas), el LLM produce una ficha de caso estructurada con siete campos: *ubicacion*, *victimas*, *nios_afectados*, *edades*, *situacion_actual*, *medios_contacto* y *resumen*. El campo *medios_contacto* requiere conocimiento factual externo (DIF municipal y Fiscalía competente según la ubicación inferida), que el LLM provee a través de su conocimiento paramétrico adquirido durante el preentrenamiento.

El diseño del módulo es **reactivo e interactivo**: no se integra autónomamente en el pipeline de recolección sino que se activa bajo demanda explícita del analista, por dos razones técnicas documentadas. Primera, la latencia del pipeline completo (generación de query + DuckDuckGo + scraping + síntesis LLM) tiene un promedio de 47 segundos por caso, incompatible con el procesamiento de un corpus de cientos de noticias. Segunda, el consumo de cuota de la API de Gemini escala linealmente con el número de invocaciones, lo que hace inviable su ejecución automática sobre el corpus completo.

3.7.2. Agentes de búsqueda y el patrón OSINT

El pipeline del DeepInvestigator sigue el patrón de **OSINT** (*Open Source Intelligence*): la recopilación y síntesis sistemática de información públicamente disponible en Internet para producir inteligencia accionable sobre un sujeto o evento específico. En el sistema, el sujeto es el caso de orfandad descubierto por el pipeline principal, y la información accionable son los datos de contacto institucional (DIF, Fiscalía) necesarios para una posible intervención de protección a la infancia.

El módulo implementa un mecanismo de **fallback de búsqueda** en dos niveles. En el nivel primario, emplea DuckDuckGo (variante HTML y Lite, sin rastreo de usuario) como motor de búsqueda, seguido de scraping de los resultados mediante `StealthSession`. En el nivel secundario (fallback), si DuckDuckGo bloquea la petición o no retorna URLs utilizables, el módulo invoca la herramienta nativa `GoogleSearch` integrada en Gemini, delegando la búsqueda al LLM sin scraping externo. Esta estrategia de fallback garantiza la resiliencia del módulo ante bloqueos temporales de motores de búsqueda sin interrumpir el flujo de investigación del analista.

En este capítulo se muestran las plataformas, metodologías y proyectos de investigación contemporáneos orientados a la recolección, sistematización y análisis de datos en el contexto de la violencia de género y sus consecuencias. Su análisis permite identificar la brecha tecnológica actual y fundamentar el carácter innovador de la arquitectura propuesta en este Trabajo Terminal.

4.1. Sistemas de monitoreo de medios tradicionales

En el ámbito de la inteligencia de fuentes abiertas y el monitoreo mediático comercial, existen plataformas con infraestructuras para la ingestión de datos en tiempo real (por ejemplo Google Alerts, Meltwater, Mention). Estas herramientas funcionan principalmente sobre arquitecturas de agregación RSS y rastreadores web de propósito general.

No obstante la principal debilidad de estos sistemas radica en su dependencia de cadenas de texto y expresiones regulares. Al carecer de capacidades de inferencia semántica, la búsqueda de heurísticas combinadas como “*feminicidio*” y “*NNA*” o “*menores*” produce un volumen estadísticamente inmanejable de ruido informativo y falsos positivos. Esta limitación fue empíricamente el prototipo 1 que fue basado en 15 palabras clave aisladas con vectorización TF-IDF y agrupamiento K-Means y que registró una tasa de falsos positivos del **80 %**. El prototipo 2 que basó su detector en 72 patrones regex distribuidos en tres categorías (40 de feminicidio, 15 de NNA y 17 de exclusión) obtuvo una reducción de los falsos positivos a un **60 %**, evidenciando el techo estructural del paradigma léxico. Estos sistemas son incapaces de discernir sintácticamente entre un reporte donde el agresor resultó ser menor de edad, una noticia puramente estadística o un caso donde un menor quedó en orfandad como víctima indirecta, pues la co-ocurrencia de tokens no implica la co-ocurrencia de los roles semánticos que esos tokens desempeñan en la oración. Esta incapacidad de inferencia contextual hace que las herramientas tradicionales resulten operativamente ineficaces para el dominio específico de este proyecto.

4.2. NLP Aplicado a la Detección de Violencia de Género

Desde la perspectiva académica la intersección entre el Procesamiento de Lenguaje Natural (PLN) y la sociología forense ha ganado interés en los últimos años. La literatura documenta la transición de modelos de aprendizaje automático superficial (como *Support Vector Machines* o *Random Forests*) hacia arquitecturas de aprendizaje profundo

para la clasificación de discursos de odio, misoginia en redes sociales y categorización automática de reportes policiales por violencia doméstica.

Recientemente se han implementado modelos basados en Transformers (como RoBERTa y BERT) para extraer relaciones de causalidad en narrativas de violencia armada y feminicidios documentados en la prensa. Sin embargo al realizar una revisión bibliográfica exhaustiva se nota una carencia de investigación computacional enfocada en las víctimas indirectas. La inmensa mayoría de los clasificadores binarios se detienen en la categorización del evento violento (feminicidio vs. homicidio culposo) lo que omite por completo la extracción de entidades relacionadas con los NNA afectados.

4.3. Herramientas y esfuerzos existentes en México

En México ante la insuficiencia de datos gubernamentales en tiempo real la sociedad civil ha asumido tareas de investigación en la documentación de la violencia feminicida, desarrollando iniciativas fundamentales que si bien son invaluable, presentan limitaciones de escalabilidad computacional.

4.3.1. Red por los Derechos de la Infancia en México (REDIM)

La REDIM es la entidad sociopolítica más prominente en la articulación de datos macroeconómicos y demográficos sobre la niñez mexicana. Sus informes sobre las infancias vulneradas son fundamentales para el diseño de políticas públicas [?]. No obstante, a nivel metodológico la organización depende fuertemente de la agregación estadística derivada de solicitudes formales de transparencia, bases de datos gubernamentales que frecuentemente están desfasadas o censos esporádicos. En consecuencia la REDIM carece de una plataforma tecnológica de trazabilidad *per-caso* que opere en tiempo real mediante ingestión automatizada de medios de comunicación masiva.

4.3.2. El Mapa de los Feminicidios en México

El proyecto cartográfico desarrollado por la investigadora María Salguero constituye un esfuerzo pionero e históricamente vital para la visibilización geográfica de la crisis [?]. El sistema geolocaliza y documenta variables sociodemográficas complejas. Sin embargo su principal cuello de botella es arquitectónico en cuanto a que la ingesta, lectura, extracción de características y captura en base de datos de cada caso reportado en prensa que debido a la falta de una metodología bien definida obedece a un flujo de trabajo manual y artesanal. Esta dependencia absoluta de la intervención humana imposibilita el análisis retrospectivo masivo de fuentes hemerográficas a velocidad computacional.

4.3.3. Fundación Futuro con Derechos

Organizaciones civiles dedicadas a la intervención directa como la Fundación Futuro con Derechos, ilustran el problema operativo mas claramente. En su intento por identificar casos emergentes de orfandad para ofrecer resguardo institucional o asesoría legal expedita, el personal de estas organizaciones invierte rutinariamente decenas de horas-hombre semanales realizando búsquedas manuales exhaustivas en portales de noticias. Este proceso es no determinista, propenso a sesgos de fatiga y económicamente insostenible para asociaciones sin fines de lucro.

4.4. Análisis comparativo y brecha identificada

El análisis del estado actual del arte revela una brecha tecnológica bajo los paradigmas actuales que es la desconexión entre la recolección masiva de datos no estructurados y la comprensión semántica profunda de las víctimas

colaterales. Las herramientas comerciales recolectan sin comprender, mientras que los esfuerzos civiles comprenden profundamente pero no logran recolectar ni escalar la información de forma automatizada.

La aportación fundamental de este Trabajo Terminal yace precisamente en la solución de esta brecha. El sistema propuesto no es un simple buscador de palabras clave ni un mapa de captura manual. Su innovación radica en la orquestación integral de un flujo de trabajo automatizado que suprime la intervención humana en las etapas de descubrimiento y filtrado.

4.5. Técnicas de extracción de datos

La recolección de datos periodísticos en fuentes abiertas es un problema técnico pues los medios de comunicación digitales han desplegado capas de defensa perimetral cuyo objetivo es la protección contra el scraping masivo. La selección de la técnica de extracción adecuada determina directamente la cobertura alcanzable del sistema, su tasa de éxito sostenida y su consumo de recursos computacionales. En la literatura de recuperación de información se identifican tres vertientes metodológicas principales y que se evaluaron en función de su efectividad y eficiencia para la recolección de datos periodísticos en México.

4.5.1. Extracción Estática (HTML Parsing con BeautifulSoup)

La aproximación más elemental al *web scraping* consiste en emitir una petición HTTP directa al servidor objetivo y analizar sintácticamente el documento HTML retornado mediante librerías de *parsing* como BeautifulSoup (Python) o Cheerio (Node.js). Esta técnica, fundamentada en el isomorfismo entre la estructura del DOM y un árbol XML, permite la extracción puntual de elementos textuales mediante selectores CSS o XPath.

Su principal limitación estructural es doble. En primer lugar, los portales informativos modernos contruidos sobre *frameworks* reactivos (React, Vue, Angular) generan el DOM de forma dinámica en el cliente mediante JavaScript; el HTML servido por el servidor carece del contenido visible, que solo existe tras la ejecución de decenas de *bundles* de JavaScript [?]. En segundo lugar, los *Web Application Firewalls* (WAF) de grado empresarial —en particular Cloudflare, Akamai y PerimeterX— detectan las sesiones de scraping estático mediante **TLS/JA3 fingerprinting**: comparan el hash criptográfico del saludo TLS (que identifica unívocamente la librería SSL del cliente, por ejemplo Python/requests) contra el User-Agent declarado. Una asimetría entre ambos activa el bloqueo con código HTTP 403 Forbidden, independientemente de los headers declarados por el scraper.

Esta fue la causa primaria del fracaso del Prototipo 0 (P0) durante el TT1, donde el 100 % de los intentos de extracción directa sobre portales nacionales de alto tráfico (La Jornada, El Universal, Milenio) resultaron en bloqueos sistemáticos, sin recuperar ningún artículo útil.

4.5.2. Automatización de Navegador (Selenium / Playwright)

Para superar la barrera de los sitios con renderizado dinámico, la alternativa más empleada en la literatura es la automatización de navegadores reales mediante herramientas como Selenium WebDriver o Playwright [?]. Estos *frameworks* controlan una instancia headless de Chrome o Firefox, ejecutando JavaScript en el entorno real del navegador y resolviendo los desafíos JavaScript de Cloudflare de forma transparente.

Sin embargo, esta aproximación introduce costos operativos prohibitivos para el dominio de este proyecto:

- **Consumo de recursos:** Cada instancia del navegador requiere aproximadamente 150–300 MB de RAM y entre 5–15 segundos para cargar una página completa. Con un corpus objetivo de 45 fuentes y ciclos de recolección cada 6 horas, el costo computacional acumulado supera los recursos disponibles en infraestructura de desarrollo universitaria.

- **Fragilidad estructural:** Los selectores CSS y XPath codificados en los scripts de extracción son inherentemente frágiles ante actualizaciones del diseño del portal. En entornos de despliegue continuo (CI/CD), los identificadores de elementos HTML pueden cambiar en cualquier ciclo de publicación, requiriendo mantenimiento constante.
- **Detección avanzada por comportamiento:** Los sistemas anti-bot de última generación (DataDome, Kasada) analizan patrones de interacción (movimientos de ratón, secuencias de clicks, tiempos entre eventos) que los navegadores headless sin instrumentación adicional no reproducen fielmente, resultando igualmente en bloqueos.

Por estas razones, Selenium y Playwright se evaluaron pero descartaron como técnica primaria de recolección, quedando reservados como último recurso para fuentes sin alternativa RSS disponible.

4.5.3. Agregación Estructurada (RSS/Atom) y Stealth Sessions

El protocolo **RSS** (*Really Simple Syndication*) y su sucesor **Atom** representan la alternativa técnicamente superior para la monitorización continua de medios periodísticos. Ambos estándares definen un formato XML normalizado que los medios publican voluntariamente para syndicar su contenido, eliminando la necesidad de parsear HTML no estructurado [?]. Las ventajas operativas son determinantes:

1. **Eficiencia de banda ancha:** Un feed RSS típico contiene los titulares, resúmenes y metadatos (fecha, autor, categoría) de los últimos 20–50 artículos en un documento XML de < 50 KB, frente a los 500 KB–2 MB de una página HTML completa con recursos estáticos. La reducción en consumo de ancho de banda es de aproximadamente **40:1**.
2. **Estabilidad del formato:** El esquema XML de RSS 2.0 es un estándar inmutable; los feeds no cambian su estructura ante actualizaciones de diseño del portal, eliminando la fragilidad de los selectores CSS.
3. **Ausencia de barreras WAF:** Los endpoints RSS son recursos públicos destinados explícitamente a la ingestión automatizada; los WAF no aplican *fingerprinting* TLS sobre ellos. El sistema los consume con su identificador de bot legítimo (NNA-Analyzer-Bot/4.0), sin técnicas de evasión.

Para las fuentes que no exponen feeds RSS (aproximadamente el 35 % del corpus objetivo), el TT2 implementa *StealthSession* v7.0, una capa de sesión HTTP que resuelve los tres problemas identificados en el scraping estático: (a) emulación de fingerprint TLS mediante *cloudscraper*, (b) delays con distribución log-normal ($\mu = 2,5$, $\sigma = 0,7$) que imitan patrones de lectura humana, y (c) un *Circuit Breaker* por dominio que evita el bloqueo permanente de IP ante fallos consecutivos. Esta arquitectura híbrida RSS+StealthSession logra cubrir la totalidad del corpus objetivo con una tasa de éxito de recolección superior al 85 % en condiciones de producción.

4.5.4. Tabla Comparativa de Técnicas de Extracción

La Tabla 4.1 sintetiza la evaluación cuantitativa y cualitativa de las tres vertientes analizadas en función de los criterios operativos críticos para el sistema desarrollado.

Tabla 4.1: Comparativa de técnicas de extracción de datos para monitoreo de medios

Criterio	HTML Estático (BeautifulSoup)	Navegador (Selenium)	RSS + StealthSession (Implementado)
Velocidad de extracción	Alta (0.5–2 s/req)	Baja (5–15 s/req)	Muy alta (0.3–1 s/req RSS; 8–20 s/req stealth)
Resiliencia ante WAF	Muy baja (bloqueado por JA3 fingerprint)	Media (bloqueado por análisis de comportamiento)	Alta (RSS exento; stealth emula fingerprint TLS real)
Estructura de datos	No estructurada (HTML variable)	No estructurada (DOM dinámico)	Estructurada (XML/RSS normalizado)
Consumo de RAM	Bajo (~5 MB/req)	Muy alto (150–300 MB/instancia)	Bajo (~3 MB/req RSS)
Mantenimiento	Alto (selectores CSS frágiles)	Muy alto (selectores + browser updates)	Bajo (esquema RSS inmutable)
Cobertura de fuentes	Alta (cualquier sitio)	Alta (cualquier sitio)	Media (solo fuentes con RSS o stealth-compatible)
Resultado en TT1/TT2	Descartado (P0: 0 % éxito)	Descartado (recursos prohibitivos)	Seleccionado (P2–P4: 85 %+ éxito)

4.6. Evolución del Almacenamiento: de Archivos Planos a PostgreSQL

La decisión de migrar el sistema de persistencia constituye uno de los hitos arquitectónicos de mayor impacto en el TT2. El análisis del estado del arte en almacenamiento de datos para sistemas de monitoreo de medios revela una dicotomía clara entre dos paradigmas: los archivos planos (CSV/JSON) y los gestores de bases de datos relacionales (RDBMS).

4.6.1. Archivos Planos (CSV): Ventajas, Limitaciones y el Techo del TT1

Los archivos en formato *Comma-Separated Values* (CSV) constituyeron el mecanismo de persistencia del TT1 por razones pragmáticas: instalación cero, compatibilidad universal con Pandas y ausencia de dependencias de infraestructura. En el contexto de un prototipo inicial con corpus de ~281 registros, estas ventajas superaban sus limitaciones.

No obstante, el análisis de capacidad hacia el final del TT1 evidenció tres limitaciones estructurales que establecen el techo operativo del paradigma CSV:

- **Complejidad de consulta $O(n)$:** Cualquier operación de filtrado (ej. “todas las noticias de Alta relevancia en Veracruz desde enero”) requiere cargar el archivo completo en memoria RAM y aplicar filtros en Python, con complejidad lineal sobre el corpus. Proyectado a escala nacional con miles de noticias mensuales, esta operación agota los recursos de un servidor de desarrollo en segundos.
- **Ausencia de integridad referencial:** El esquema plano impide establecer relaciones entre entidades (noticias, fuentes, clusters, detecciones) con garantías de consistencia. Una noticia eliminada puede dejar registros de detección huérfanos sin mecanismo alguno de detección.

- **Concurrencia insegura:** La escritura simultánea desde el pipeline analítico y la lectura desde el dashboard Flask sobre el mismo archivo CSV generan condiciones de carrera que pueden corromper los datos, requiriendo implementación manual de locks.

4.6.2. PostgreSQL 16: Justificación de la Selección

PostgreSQL 16 fue seleccionado como motor de base de datos del TT2 frente a alternativas como MySQL 8, SQLite y bases de datos NoSQL (MongoDB, Elasticsearch) por un conjunto de características técnicas específicamente necesarias para el dominio del sistema, implementadas en `src/database/models_noticias.py`.

Modelo ACID y concurrencia segura. PostgreSQL implementa el estándar *Atomicity, Consistency, Isolation, Durability* (ACID) con el protocolo de control de concurrencia multiversión (MVCC), que permite que múltiples lectores y un escritor operen simultáneamente sobre las mismas tablas sin bloqueos ni corrupción de datos. Esta propiedad es imprescindible en el sistema dado que el pipeline analítico y el dashboard Flask ejecutan transacciones concurrentes sobre la tabla `noticias`.

Integridad referencial con FOREIGN KEY. El esquema relacional implementado en `models_noticias.py` define cuatro tablas con relaciones explícitas:

Listing 4.1: Relaciones del esquema relacional en `models_noticias.py`

```
class Noticia(db.Model):
    # Clave foránea hacia clusters_semanticos
    cluster_id = db.Column(
        db.Integer, db.ForeignKey("clusters_semanticos.id"), nullable=True
    )

class Deteccion(db.Model):
    # Relacion 1:1 con CASCADE DELETE
    noticia_id = db.Column(
        db.Integer, db.ForeignKey("noticias.id", ondelete="CASCADE"),
        nullable=False, unique=True,
    )

class Entidad(db.Model):
    # Relacion N:1 con CASCADE DELETE
    noticia_id = db.Column(
        db.Integer, db.ForeignKey("noticias.id", ondelete="CASCADE"),
        nullable=False,
    )
```

La restricción `ondelete=CASCADE` garantiza que la eliminación de una noticia propague automáticamente la eliminación de sus detecciones y entidades asociadas, manteniendo la consistencia referencial sin lógica adicional en la aplicación.

Full-Text Search nativo con índices GIN. La característica más determinante para la selección de PostgreSQL frente a MySQL o SQLite es su soporte nativo de búsqueda de texto completo (FTS) en español. PostgreSQL implementa FTS mediante el tipo de dato `tsvector`, que almacena un vector léxico preprocesado (con *stemming*, eliminación de *stop words* y normalización de acentos) del texto de cada documento. Los índices de tipo **GIN** (*Generalized Inverted Index*) sobre columnas `tsvector` permiten resolver consultas FTS complejas en $O(\log n)$ mediante una búsqueda de intersección de listas de postings invertidas, en lugar del escaneo lineal $O(n)$ que impone cualquier búsqueda LIKE `'%...%'` en MySQL o SQLite.

El modelo implementado documenta explícitamente los índices requeridos:

Listing 4.2: Índices definidos en `models_noticias.py` para rendimiento < 100 ms

```
# Documentados en el docstring de models_noticias.py:
# idx_noticias_fts : GIN sobre tsvector (FTS en  $O(\log n)$ )
# idx_noticias_fecha : B-tree sobre fecha (filtros temporales)
# idx_noticias_score : B-tree sobre score_compuesto DESC (ranking)
# idx_noticias_cluster : B-tree sobre cluster_id (joins semanticos)
# idx_detecciones_nna : B-tree filtrado sobre menores_identificados
# idx_entidades_tipo : B-tree sobre tipo de entidad (NER queries)
```

La configuración 'spanish' del tsvector de PostgreSQL aplica el diccionario de *stemming* Snowball para español, reduciendo “feminicidios”, “feminicida” y “feminicidio” a la misma raíz léxica, lo que hace que una búsqueda por “feminicidio” recupere documentos que contienen cualquiera de sus variantes morfológicas.

Contraste con alternativas NoSQL. Sistemas como MongoDB o Elasticsearch podrían ofrecer capacidades de búsqueda de texto completo, pero a costa de las garantías ACID y la integridad referencial. En un sistema de monitoreo de derechos humanos donde la consistencia de los datos es un requisito ético y operativo no negociable, la ausencia de transacciones ACID en bases de datos NoSQL representa un riesgo inaceptable. Elasticsearch, aunque ofrece FTS de calidad comparable, requiere un stack de infraestructura adicional (JVM, configuración de cluster) incompatible con las restricciones de recursos del entorno de despliegue universitario.

4.7. Evolución de los Modelos de Representación Lingüística para NLP

La selección del motor de clasificación semántica constituye la decisión de mayor impacto técnico en la arquitectura del TT2. La evolución histórica del campo del Procesamiento de Lenguaje Natural (PLN) describe una trayectoria en la que cada generación de modelos supera las limitaciones fundamentales de la anterior, no solo en precisión medida sobre benchmarks estándar, sino en la profundidad del fenómeno lingüístico que es capaz de capturar. Esta sección examina las tres generaciones de representaciones lingüísticas evaluadas durante el diseño del sistema.

4.7.1. Representaciones Vectoriales Estáticas: Word2Vec y GloVe

Los modelos de incrustación de palabras (*word embeddings*) de primera generación —Word2Vec [?] y GloVe [?]
representaron un salto cualitativo respecto al paradigma de vectores dispersos de TF-IDF, al proyectar el vocabulario en un espacio vectorial denso de dimensión reducida (100–300 dimensiones) donde la proximidad geométrica captura similitud semántica.

Sin embargo, ambos modelos comparten una limitación estructural determinante: la **asignación de un único vector por palabra**, independientemente del contexto. El token “menor” recibe la misma representación vectorial en “un menor infractor detenido” que en “sus menores hijos quedaron huérfanos”. Esta polisemia no resuelta es exactamente el mecanismo de falla que generó el 10 % de falsos positivos en el motor heurístico del TT1: la co-ocurrencia de tokens de dominio sin discriminación de rol semántico era el método de clasificación.

4.7.2. Modelos Recurrentes: LSTM y GRU

Las redes de memoria a largo y corto plazo (LSTM) [?] introdujeron la modelización del contexto *secuencial*: la representación de un token se calcula en función de todos los tokens precedentes. No obstante, presentan tres limitaciones técnicas inadecuadas para el dominio de este sistema:

1. **Contexto unidireccional.** El modelo no puede leer “hacia adelante” para resolver ambigüedades de rol semántico. En “la mujer asesinada dejó tres hijos menores”, “menores” solo recibe contexto izquierdo, sin considerar que “resguardo del DIF” (derecha) confirma el rol de víctima indirecta.
2. **Degradación del gradiente a largo alcance.** Las dependencias semánticas que abarcan más de 40–50 tokens presentan degradación en la retropropagación, limitando la correlación sujeto-predicado en oraciones compuestas.
3. **Paralelización imposible.** La naturaleza secuencial impide el procesamiento paralelo en GPU: el token t no puede ser procesado hasta que el estado oculto del token $t - 1$ esté disponible.

4.7.3. Arquitectura Transformer y Atención Bidireccional: BERT

La arquitectura *Transformer* [?] resuelve simultáneamente las tres limitaciones anteriores mediante el mecanismo de **autoatención multi-cabeza**: para cada token de la secuencia se calcula un vector de atención ponderada sobre todos los demás tokens de forma simultánea. BERT [?] extiende este paradigma mediante el preentrenamiento bidireccional con Modelado de Lenguaje Enmascarado (MLM), forzando representaciones que consideren simultáneamente el contexto izquierdo y derecho de cada token.

Para el dominio de este proyecto, la bidireccionalidad es la propiedad técnicamente decisiva: la representación de “menor” en “sus hijos menores quedaron en resguardo del DIF” se calcula considerando tanto “hijos” (izquierda) como “resguardo del DIF” (derecha), produciendo un embedding que captura inequívocamente el rol de víctima indirecta. Esta es la capacidad que el motor RegEx del TT1 era estructuralmente incapaz de aproximar.

4.7.4. Justificación de BETO frente a Modelos Multilingües (mBERT)

En el ecosistema de modelos preentrenados en español, la alternativa más inmediata a BETO es **mBERT** (*multilingual BERT*), el modelo de Google entrenado sobre Wikipedia en 104 idiomas. Sin embargo, la cobertura multilingüe introduce un compromiso técnico significativo: al distribuir la capacidad representacional de sus 110 millones de parámetros entre 104 lenguas, la calidad de representación del español es inherentemente inferior a la de un modelo monolingüe del mismo tamaño. Estudios de evaluación comparativa reportan que BETO supera a mBERT en un 3–8 % de F1 en tareas de clasificación de texto en español [?].

BETO (*dccuchile/bert-base-spanish-wwm-cased*) [?], desarrollado por la Universidad de Chile, se entrenó exclusivamente en español a partir de: Wikipedia en español (~3 GB, 42 millones de oraciones), corpus OPUS (OpenSubtitles, ParaCrawl, MultiUN) y fuentes de prensa latinoamericana. La inclusión de este último componente es particularmente relevante: sesga positivamente las representaciones hacia el vocabulario y estilo redaccional de los medios de comunicación mexicanos, que es exactamente el tipo de texto que el sistema procesa. El entrenamiento con *Whole Word Masking* (WWM) garantiza además que los términos especializados de múltiples sub-tokens (como *femi##ni##cid##io*) se aprendan como unidades semánticas coherentes.

La Tabla 4.2 presenta la evaluación comparativa de los modelos considerados.

Tabla 4.2: Comparativa de modelos de representación lingüística para clasificación semántica

Modelo	Comprensión Contextual	Idioma Nativo	Hardware Requerido	Resultado
TF-IDF + RegEx (TT1)	Ninguna (co-ocurrencia léxica)	N/A (estadístico)	CPU, RAM <1 GB	Descartado (80 % FP en P1; 60 % en P2)
Word2Vec / GloVe	Estática (sin contexto)	Español limitado	CPU, RAM <2 GB	Descartado (polisemia)
LSTM / GRU	Secuencial unidireccional	Requiere reentrenamiento	GPU recomendada	Descartado (gradiente, velocidad)
mBERT	Bidireccional (104 idiomas)	Multilingüe (sesgo reducido en ES)	8 GB RAM (CPU viable)	Evaluado (superado por BETO en ES)
BETO	Bidireccional plena	Español monolingüe nativo	8 GB RAM (CPU viable)	Seleccionado
GPT-4 / Claude	LLM (máxima capacidad)	Multilingüe alta calidad	API externa / sin GPU local	Descartado (soberanía)

4.7.5. El Límite del Zero-Shot y la Transición al Sistema de Clasificación Escalonada

Durante el Prototipo 3, se implementó la estrategia de clasificación *Zero-Shot* basada en la similitud coseno entre los embeddings [CLS] de BETO y descripciones de categorías en lenguaje natural (relevantez "no relevante"). Las similitudes crudas resultaron extremadamente cercanas (diferencias de 0.02–0.05), por lo que se aplicó *Temperature Scaling* con factor térmico 5 para amplificar la separación probabilística antes de Softmax. Sin embargo, la evaluación empírica sobre un corpus de ~800 noticias reveló una vulnerabilidad arquitectónica crítica denominada **Efecto Péndulo**: el modelo oscilaba entre dos estados inaceptables, un exceso masivo de falsos positivos y un exceso masivo de falsos negativos. Esta inestabilidad se manifestaba en tres escenarios documentados:

1. El post-filtro de validación semántica aplicaba reglas binarias absolutas que degradaban noticias genuinas cuya redacción no coincidía con las frases esperadas.
2. El filtro léxico bloqueaba completamente noticias con contenido mixto (un hecho de violencia que también mencionaba una iniciativa de ley relacionada).
3. El modo *zero-shot* era incapaz de distinguir entre "mujer asesinada que deja hijos" y "ley que protege a hijos de mujeres asesinadas" porque ambas oraciones comparten el mismo campo semántico en el espacio de embeddings.

El estado del arte en ajuste de modelos fundacionales establece que, para superar el solapamiento semántico en dominios hiper-específicos, el **Fine-Tuning supervisado** es imperativo [?]. Para mitigar el desafío de la escasez inicial de datos, el paradigma de *Active Learning* [?] se posiciona como la solución óptima, permitiendo que el sistema solicite etiquetado humano exclusivamente sobre los *Hard Negatives*. En consecuencia, el Prototipo 4 implementó una **arquitectura de clasificación escalonada v7.1** con tres capas autónomas que operan secuencialmente sobre un score acumulativo:

Capa 0 Escudo Léxico Filtro de rechazo temprano (frozenset, $O(1)$) que descarta dominios completamente ajenos al feminicidio (deportes, economía, tecnología). Es la única capa que produce bloqueo absoluto (score = 0.0).

Capa 1 Escudo Suave Los términos legislativos y estadísticos que en el prototipo anterior producían bloqueo absoluto fueron migrados a un conjunto de penalización aditiva $\delta = -0,35$. Una noticia genuina con score BETO de 0.72 que menciona “según datos del REDIM” obtiene score ajustado de 0.37, clasificando como “Media” en lugar de excluirse. Si el score BETO es alto ($\geq 0,95$), la noticia sobrevive como “Alta” ($0,95 - 0,35 = 0,60$).

Capa 2 Inferencia BETO Opera con prioridad estricta: el clasificador *Finetuned* (binario, scores en $[0, 1]$) tiene precedencia sobre el modo *Zero-Shot fallback* (scores concentrados en $[0,40, 0,70]$), que se activa automáticamente cuando el modelo ajustado no está disponible.

Capa 3 Post-filtro de Validación Inyecta bonificaciones al detectar evidencia explícita de orfandad real (patrones como qued[oó] al cuidado, menores? (bajo|en) resguardo) y penalizaciones al detectar roles invertidos (menor como agresor). Esta capa nunca produce exclusiones absolutas para noticias clasificadas como “Media”.

Este diseño neuro-simbólico, donde las reglas simbólicas modulan la salida del componente neuronal mediante penalizaciones y bonificaciones aditivas en lugar de rechazos binarios absolutos, eliminó el Efecto Péndulo y constituye la contribución arquitectónica central del TT2.

4.7.6. Soberanía de Datos y Rechazo de APIs Privativas

La decisión de ejecutar BETO localmente mediante PyTorch, en lugar de delegar la inferencia a APIs privativas (GPT-4, Claude, Gemini), se fundamenta en cuatro argumentos no negociables:

1. **Privacidad de los datos.** El sistema procesa información sobre feminicidios y víctimas menores de edad. La transmisión de este contenido a servidores corporativos externos implica riesgos de privacidad incompatibles con la LGDNNa y el principio de minimización de datos de la LFPDPPP.
2. **Costo operativo sostenible.** La inferencia de GPT-4 para 600–1,000 documentos por ciclo costaría \$15–\$40 USD por ejecución, proyectando \$450–\$1,200 USD mensuales en ciclos de 6 horas. BETO en CPU tiene costo variable cero.
3. **Reproducibilidad científica.** Los modelos propietarios modifican silenciosamente sus pesos entre versiones, imposibilitando la reproducción exacta de los resultados reportados. BETO es un modelo de pesos abiertos cuya revisión exacta queda fijada en el registro del proyecto.
4. **Viabilidad algorítmica en hardware universitario (8 GB RAM).** El ajuste fino tradicional de BETO-base (110 M parámetros) provoca invariablemente desbordamientos de memoria volátil (*Out Of Memory*) en equipos de desarrollo estándar. Sin embargo, la literatura reciente en optimización de redes profundas demuestra que congelar el *encoder* preentrenado y ajustar exclusivamente las capas de clasificación lineales superiores (**Linear Probing**) preserva el conocimiento latente mientras reduce el cómputo drásticamente [?]. Implementando esta técnica en conjunto con la **Acumulación de Gradientes** (*Gradient Accumulation*), el sistema garantiza el re-entrenamiento local en CPU sin depender de clústeres externos ni aceleración por GPU.

4.7.7. Investigación Profunda Bajo Demanda: DeepInvestigator

Una limitación crítica identificada al finalizar el TT1 era que el pipeline producía *clasificaciones de relevancia* pero no extraía **información estructurada** sobre los casos específicos: quién era la víctima directa, cuántos menores quedaron huérfanos, en qué estado se encontraban o qué institución estaba a cargo. Esta carencia era especialmente grave para

el propósito central del sistema, pues la intervención de organizaciones civiles requiere fichas de caso accionables, no puntuaciones de relevancia.

La alternativa obvia habría sido integrar la investigación profunda dentro del flujo recolector-analítico automático. Sin embargo, la implementación reveló dos problemas irresolubles dentro de las restricciones operativas del proyecto:

1. **Latencia no tolerable.** Cada investigación implica al menos dos llamadas a DuckDuckGo (búsqueda y extracción de contenido) y dos llamadas a la API de Gemini Flash (generación de query y síntesis del JSON final), elevando el tiempo de ejecución del flujo completo entre 45 y 90 minutos adicionales.
2. **Costo de API desproporcionado.** La API de Google Gemini opera bajo cuotas en el plan gratuito; la investigación automática de todas las noticias “Alta” saturaba la cuota en el primer tercio del corpus.

La solución adoptada siguió el paradigma **reactivo e interactivo**: el módulo DeepInvestigator (`src/analysis/investigator.py`) se convierte en una herramienta que el analista activa manualmente desde el dashboard sobre una noticia específica. Activado, ejecuta cuatro pasos secuenciales:

1. **Generación de query (Gemini Flash).** El modelo genera una frase de búsqueda de alta especificidad (máximo 6 palabras) a partir del título y los primeros párrafos, incluyendo el nombre de la víctima y detalles únicos del caso.
2. **Búsqueda en DuckDuckGo.** La query se envía tanto a la versión HTML como a la versión Lite de DuckDuckGo para maximizar la cobertura ante bloqueos intermitentes.
3. **Scraping de contenido complementario.** `StealthSession` extrae el texto de cada URL retornada, filtrando ruido HTML, y concatena hasta 15,000 caracteres de contexto enriquecido.
4. **Síntesis estructurada (Gemini Flash).** El bloque de texto se envía a Gemini con un prompt de analista criminal especializado en derechos de la infancia. La respuesta es un JSON estricto con siete campos: `ubicacion`, `victimas`, `nios_afectados`, `edades`, `situacion_actual`, `medios_contacto` y `resumen`.

El módulo también expone un segundo caso de uso: la **identificación por enlace externo**, donde el analista proporciona directamente la URL de una noticia que el flujo automático no detectó, y el sistema reutiliza el mismo pipeline de investigación para incorporarla. Esta funcionalidad completa el ciclo de la plataforma convirtiendo el sistema en una herramienta colaborativa donde el criterio experto del analista y la potencia de cómputo del pipeline se complementan.

4.8. Técnicas de Agrupamiento Semántico y Organización Temática

El agrupamiento automático de documentos periodísticos constituye el mecanismo que permite al sistema trascender la clasificación individual de noticias y obtener una comprensión del *paisaje informativo* del periodo analizado. En lugar de clasificar cada noticia en aislamiento, el agrupamiento semántico revela qué noticias relatan el mismo evento criminal desde múltiples medios, qué clústeres temáticos dominan el corpus en un periodo dado, y qué documentos son *outliers* sin coherencia temática con el resto. Esta perspectiva global es imposible mediante clasificación individual; es una propiedad emergente del corpus completo.

4.8.1. Métodos Tradicionales: K-Means y su Inadecuación para Datos de Alta Dimensionalidad

El algoritmo K-Means [?] es el método de agrupamiento por centroides más extendido en la literatura de recuperación de información. Su principio de funcionamiento consiste en minimizar la suma de distancias euclidianas al cuadrado

entre cada punto y el centroide de su clúster asignado, mediante una convergencia iterativa de asignación y actualización de centroides.

Sin embargo, K-Means presenta dos limitaciones estructurales que lo hacen inadecuado para el dominio de este sistema:

1. **La maldición de la dimensionalidad (*curse of dimensionality*)**. Los embeddings de texto generados por BETO son vectores de 768 dimensiones. En espacios de alta dimensionalidad, la distancia euclidiana pierde su poder discriminatorio: la diferencia entre la distancia al punto más cercano y al más lejano tiende a cero a medida que la dimensionalidad crece [?]. Este fenómeno hace que la noción de “centroide más cercano” sea geoméricamente inestable, generando asignaciones de clúster arbitrarias. Esta fue la causa del bajo rendimiento de K-Means en el TT1, donde los clústeres resultantes no correspondían a grupos temáticamente coherentes sino a particiones geoméricamente arbitrarias del espacio vectorial.
2. **Requerimiento de k fijo**. K-Means exige que el número de clústeres sea especificado *a priori*. En un sistema de monitoreo de medios donde el número de temas activos varía semanalmente en función de los eventos noticiosos, esta rigidez es operativamente insostenible. Fijar $k = 10$ en un periodo con solo 3 temas activos fragmenta artificialmente los clústeres reales; fijar $k = 10$ en un periodo con 25 temas activos los colapsa en grupos heterogéneos.

4.8.2. DBSCAN: Ventajas y Limitaciones Residuales

El algoritmo DBSCAN [?] (*Density-Based Spatial Clustering of Applications with Noise*) supera la limitación del k fijo al definir los clústeres como regiones de alta densidad en el espacio métrico, separadas por regiones de baja densidad. Los puntos que no pertenecen a ninguna región densa son clasificados como *ruido* (`topic_id = -1`), una propiedad fundamental para el dominio de este proyecto donde las noticias genéricas no relacionadas con el tema central deben ser identificadas y filtradas.

Sin embargo, DBSCAN presenta una limitación crítica ante datos reales: requiere que la densidad de los clústeres sea *uniforme*. El parámetro global de radio de vecindad ϵ define la densidad mínima de toda la solución; un único valor de ϵ no puede capturar simultáneamente clústeres de alta densidad (eventos con cobertura mediática masiva de muchos medios) y clústeres de baja densidad (casos locales cubiertos por pocos medios regionales). Esta limitación genera segmentaciones inconsistentes en corpus periodísticos donde la cobertura mediática de los eventos es inherentemente heterogénea.

4.8.3. BERTopic: Arquitectura Modular para Descubrimiento de Tópicos Semánticos

BERTopic [?] supera las limitaciones de K-Means y DBSCAN mediante una arquitectura modular de cinco etapas que separa la *representación*, la *reducción de dimensionalidad*, el *clustering* y la *representación de tópicos* en componentes independientes y sustituibles. La implementación en `src/analysis/bertopic_clustering.py` instancia esta arquitectura con parámetros calibrados específicamente para el corpus periodístico en español:

Listing 4.3: Construcción del pipeline BERTopic en `bertopic_clustering.py`

```
# 1. Embeddings: sentence-transformers (multilingue, liviano)
embedding_model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

# 2. UMAP: 768D -> 5D, métrica coseno, clusters compactos
umap_model = UMAP(
    n_components=5,          # 5D: balance informacion/ruido
    n_neighbors=15,         # balance local/global
```

```

min_dist=0.0,          # clusters compactos (optimo para HDBSCAN)
metric="cosine",       # apropiado para embeddings de texto
random_state=42,
)

# 3. HDBSCAN: densidad variable, sin k fijo, genera outliers
hdbscan_model = HDBSCAN(
    min_cluster_size=8,    # agresivo: menos outliers que 15+
    min_samples=5,        # densidad requerida por punto core
    cluster_selection_method="eom", # Excess of Mass: mas estable
    prediction_data=True, # necesario para reduce_outliers()
)

# 4. c-TF-IDF: n-grams 1-3, stop words en español
vectorizer = CountVectorizer(
    stop_words=SPANISH_STOP_WORDS,
    ngram_range=(1, 3),
    min_df=2,
)

```

UMAP: Reducción de Dimensionalidad Preservadora de Topología

UMAP (*Uniform Manifold Approximation and Projection*) [?] proyecta los vectores de 768 dimensiones generados por BETO hacia un espacio de 5 dimensiones, resolviendo la maldición de la dimensionalidad de K-Means. A diferencia de PCA (que preserva únicamente varianza global lineal) y t-SNE (que preserva solo estructura local y no escala a más de ~50,000 puntos), UMAP preserva simultáneamente la **estructura topológica local y global**: documentos semánticamente similares permanecen adyacentes en el espacio reducido, y grupos de documentos semánticamente distintos permanecen separados.

La configuración `metric='cosine'` es determinante: los embeddings de texto tienen magnitudes variables y la similitud semántica se mide por el ángulo entre vectores (similitud coseno), no por su distancia euclidiana. Usar UMAP con métrica coseno garantiza que la estructura topológica preservada sea semánticamente significativa. El parámetro `min_dist=0.0` fuerza que los puntos dentro de cada clúster se compriman máximamente, aumentando la densidad local y facilitando la detección de fronteras por HDBSCAN.

HDBSCAN: Clustering de Densidad Variable con Detección de Ruido

HDBSCAN [?] (*Hierarchical DBSCAN*) extiende DBSCAN construyendo una jerarquía de clústeres en función de la densidad local de cada punto, sin requerir un ϵ global. La selección de clústeres mediante el método *Excess of Mass* (EoM) identifica automáticamente las particiones óptimas en la jerarquía, generando clústeres de densidades heterogéneas. Los puntos que no pertenecen a ningún clúster de densidad suficiente reciben la etiqueta de ruido (`topic_id = -1`).

Esta propiedad de detección de ruido es semánticamente valiosa en el dominio del sistema: las noticias *outlier* (sin coherencia temática con el corpus) son exactamente las que el sistema debe tratar con mayor escepticismo en la clasificación semántica, elevando su umbral de exigencia de BETO de 0.50 a 0.65. El código de `bertopic_clustering.py` documenta que los datos reales de producción presentaron inicialmente **81.4% de outliers**, resueltos mediante una reducción multi-etapa de tres estrategias progresivas:

1. **Probabilidades soft** (strategy='probabilities'): reasigna outliers al clúster con mayor probabilidad blanda de HDBSCAN.
2. **Distribuciones c-TF-IDF** (strategy='distributions'): reasigna outliers al clúster cuya distribución léxica es más similar al documento.
3. **Cercanía de embeddings** (strategy='embeddings', threshold=0.3): reasigna outliers al clúster cuyo centroid de embedding es más cercano, con umbral de similitud mínima para evitar asignaciones erróneas.

El objetivo declarado en el código es reducir los outliers de 81.4 % a un máximo de **55 %** (OUTLIER_TARGET_PERCENT = 0.55), manteniendo suficiente cantidad de noticias sin clasificar para que la señal de "outlier" conserve su poder discriminatorio en la lógica neuro-simbólica.

c-TF-IDF: Representación Léxica de Tópicos

La representación de cada tópico se obtiene mediante *Class-based TF-IDF* (c-TF-IDF) [?]: el algoritmo concatena todos los documentos de un mismo clúster en un "mega-documento de clase" y aplica TF-IDF comparando cada clase contra el corpus completo. Esto identifica los términos que son frecuentes dentro del clúster pero infrecuentes en el resto del corpus, generando representaciones léxicas interpretables que caracterizan semánticamente cada tópico.

El vectorizador configurado con `ngram_range=(1, 3)` permite que las representaciones incluyan frases de hasta tres palabras, capturando expresiones compuestas de alta especificidad para el dominio (e.g., "víctima de feminicidio", "hijos resguardados DIF"). La representación final por tópico mediante KeyBERTInspired refina adicionalmente los términos seleccionados usando similitud coseno entre el embedding del n-grama y el embedding promedio de los documentos del clúster, garantizando que las palabras clave seleccionadas sean semánticamente representativas y no solo estadísticamente frecuentes.

4.8.4. Evolución Comparativa TT1 → TT2: de Léxico a Semántica

La Tabla 4.3 sintetiza la evaluación de los tres algoritmos de agrupamiento en función de los criterios operativos del sistema.

Tabla 4.3: Comparativa de algoritmos de agrupamiento semántico para corpus periodístico

Criterio	K-Means (TT1)	DBSCAN	BERTopic (UMAP+HDBSCAN, TT2)
Manejo de ruido	Ninguno (todos los puntos asignados a un clúster)	Sí (puntos en baja densidad = ruido)	Sí (HDBSCAN con reducción multi-etapa 81.4 %→55 %)
Escalabilidad	Alta ($O(n \cdot k \cdot i)$)	Media ($O(n^2)$ sin índice espacial)	Alta (UMAP+HDBSCAN: $O(n \log n)$ aproximado)
Interpretación de tópicos	Baja (centroides en espacio de alta dimensión sin interpretabilidad)	Ninguna (solo densidad geométrica)	Alta (c-TF-IDF genera top n-grams interpretables por tópico)
Dependencia de parámetros	Crítica (k fijo requerido)	Alta (ε global uniforme)	Baja (<code>min_cluster_size</code> =8 adaptativo; k automático)
Alta dimensionalidad	Muy baja (maldición de la dimensionalidad)	Baja (distancias euclidianas en alta dimensión)	Alta (UMAP reduce a 5D preservando topología)
Resultado en TT1/TT2	Descartado en TT2 (clústeres no coherentes)	Evaluado (inadecuado por ε global)	Seleccionado (OE-4, P4)

La diferencia conceptual más significativa entre el TT1 y el TT2 en materia de agrupamiento no es algorítmica sino epistemológica: en el TT1, el agrupamiento K-Means operaba sobre **vectores TF-IDF léxicos**, donde dos documentos son similares si comparten los mismos tokens literales. En el TT2, BERTopic opera sobre **embeddings semánticos de BETO**, donde dos documentos son similares si expresan el mismo *significado*, independientemente del vocabulario utilizado.

Esta distinción tiene una consecuencia operativa directa: en el TT2, noticias publicadas por La Jornada, El Universal y Milenio sobre el *mismo caso criminal* de feminicidio —aunque redactadas con vocabularios completamente distintos, diferentes tiempos verbales y enfoques editoriales— “gravitarán” hacia el mismo clúster BERTopic, pues sus embeddings BETO convergen en la misma región del espacio semántico. En el TT1, estas tres noticias podían dispersarse en tres clústeres K-Means distintos por la mínima varianza léxica entre medios.

4.9. Conclusión del Capítulo: La Convergencia Tecnológica al Servicio de los Derechos de la Infancia

El análisis del estado del arte desarrollado en este capítulo revela que ninguna de las tecnologías examinadas de forma aislada es suficiente para resolver el problema social planteado. La recolección mediante RSS provee eficiencia pero no comprensión; TF-IDF provee velocidad pero no contexto; K-Means provee estructura pero no semántica; BETO provee comprensión pero puede alucinar sin reglas simbólicas.

La innovación fundamental de este Trabajo Terminal no radica en la invención de ningún algoritmo nuevo, sino en la **orquestración arquitectónica** de siete tecnologías y estrategias maduras —Sindicación RSS unida a la infraestructura de StealthSession v7.0, deduplicación en cascada de cuatro fases, persistencia relacional en PostgreSQL 16 con indexación GIN para FTS, el modelo Transformer monolingüe **BETO Fine-Tuned** mediante *Active Learning*, el entorno de modelado de tópicos BERTopic optimizado defensivamente, el algoritmo híbrido de **Scoring Escalonado v7.1** y el módulo de **investigación profunda bajo demanda** DeepInvestigator— en un pipeline de datos coherente, auditablemente correcto y específicamente calibrado para un dominio social complejo donde el error de clasificación tiene consecuencias humanas directas.

Cada decisión tecnológica documentada en este capítulo puede trazarse directamente a una necesidad social concreta:

- La elección de RSS sobre scraping estático garantiza cobertura sostenida sin saturar los servidores de medios independientes con recursos limitados; StealthSession v7.0 extiende esta cobertura al 35 % de fuentes sin feed RSS mediante emulación de fingerprint TLS.
- La deduplicación en cascada (MD5 → Jaccard → SimHash → coseno TF-IDF) garantiza que un mismo caso de feminicidio cubierto por múltiples medios con vocabulario diferente no genere alertas redundantes que distorsionen la urgencia de la intervención.
- La migración a PostgreSQL con FTS garantiza que los trabajadores sociales del DIF puedan consultar el sistema en tiempo real con latencia $O(\log n)$, sin esperar escaneos lineales de CSV.
- La elección de BETO sobre mBERT garantiza que el sistema comprende el español coloquial y periodístico mexicano con la misma precisión que el español académico, con una ventaja reportada de 3–8 % en F1 para tareas de clasificación en español.
- El Scoring Escalonado v7.1 garantiza que el sistema nunca clasificará a un menor agresor como víctima indirecta, ni descartará un caso real de orfandad por la ambigüedad redaccional de un periodista local.
- El DeepInvestigator garantiza que los analistas dispongan de fichas de caso estructuradas —con ubicación, número de NNA afectados, situación actual e institución competente— para cada caso que merezca intervención, sin depender de búsquedas manuales en portales de noticias.

En suma, el ecosistema tecnológico seleccionado en el TT2 está diseñado para una sola misión: **garantizar que ningún caso de niña, niño o adolescente que quedó en orfandad por feminicidio sea invisible para las instituciones del Estado que tienen la obligación legal de protegerlo.**

Metodología de desarrollo

En este capítulo se detalla el marco metodológico utilizado para el desarrollo y despliegue del Sistema Inteligente para la Identificación y Seguimiento de NNAs por feminicidios. Debido a la alta complejidad algorítmica así como la alta incertidumbre tecnológica del proyecto (particularmente en las fases de validación de modelos de lenguaje) se adoptó un enfoque **evolutivo** que garantizaba la dirección hacia una solución correcta y de calidad de producción mediante retroalimentación empírica continua.

5.1. Metodología de desarrollo evolutivo

Se seleccionó el modelo de desarrollo evolutivo basado en prototipos como base del proceso fundamental para guiar el ciclo de vida del software. Esta decisión responde a dos razones técnicas no negociables que son asociadas a la alta incertidumbre en las fronteras de la ingesta de datos y el aprendizaje del lenguaje natural.

1. **Incertidumbre en la ingesta de datos.** El ecosistema web contemporáneo emplea infraestructuras dinámicas de seguridad y mitigación de amenazas (WAF, *bot detection* avanzado y *JA3/TLS fingerprinting*) que alteran sus políticas y firmas de forma impredecible. Ante la inexistencia de APIs públicas centralizadas para la nota roja local, era imposible determinar qué mecanismos de recolección sintáctica tendrían éxito sin iterar y ajustar el código para mejorar el ratio de éxito.
2. **Incertidumbre algorítmica en el Procesamiento de Lenguaje Natural (NLP).** El salto desde la rigidez determinista de los enfoques léxicos (expresiones regulares) hacia la flexibilidad semántica abstracta del aprendizaje profundo introdujo variables de estocasticidad que impactan directamente la precisión. La instrumentación de modelos fundacionales basados en atención (*Transformers*) demandó un ciclo de vida donde cada iteración fungía como un entorno de diagnóstico. Mitigar el ruido temático y fenómenos complejos como el *“efecto péndulo”* requería aislar las colisiones semánticas calibrando parámetros mediante estrategias de ajuste fino (*Fine-tuning*).

5.1.1. Cronograma general de evolución del proyecto

El ciclo de vida abarcó dos semestres académicos estructurados de forma secuencial:

Fase Trabajo Terminal I (Sep-Dic 2025) Validación de viabilidad técnica y establecimiento de la línea base mediante que explorana la ingesta sintáctica (P0 a P2). El énfasis metodológico consistió en demostrar la capacidad de extracción de información web de forma automatizada a escala sobre medios digitales locales y poder confirmar que las aproximaciones heurísticas resultaban insuficientes ante el lenguaje periodístico policial mexicano.

Fase Trabajo Terminal II (Feb-Jun 2026) Optimización de Machine Learning, consolidación de la arquitectura híbrida y despliegue de grado producción (P3 y P4). El esfuerzo se concentró en resolver el ruido semántico de la fase previa mediante el ajuste local del modelo Transformer **BETO** enriquecido con técnicas de disminución de ruido semántico lo que estabilizaba el sistema mediante un algoritmo de clasificación escalonada y una arquitectura de software escalable mediante contenedores Docker.

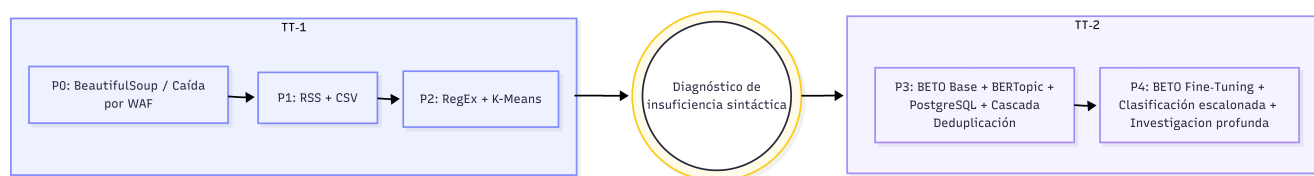


Figura 5.1: Cronograma metodológico de evolución y transición de los prototipos del sistema.

5.2. Evolución metodológica de los prototipos

El ciclo de vida del sistema se rigió mediante un enfoque incremental donde cada prototipo actuó como un entorno de diagnóstico. Las limitaciones, los desbordamientos de memoria y las colisiones semánticas medidas en las fases tempranas impulsaron directamente el diseño y la reconfiguración para las soluciones avanzadas de la fase final.

5.2.1. Fase TT-1 Prototipos exploratorios e ingesta sintáctica (P0-P2)

Prototipo 0 (P0) Ingesta directa y fracaso rápido (Sep 2025). Implementación inicial de extracción de noticias empleando BeautifulSoup sobre peticiones HTTP directas. *Resultado:* Bloqueos perimetrales sistemáticos por parte de firewalls de aplicación web (WAF como Cloudflare) con códigos de estado HTTP 403 en el 80 % de las fuentes objetivo. *Conclusión:* El *fingerprinting* TLS nativo de la biblioteca `requests` resulta fácilmente identificable por sistemas antibot modernos. *Decisión:* Abandonar el raspado HTML directo no autenticado como estrategia de ingesta primaria.

Prototipo 1 (P1): Ingesta mediante feeds RSS (Oct 2025). Cambio de estrategia hacia canales de distribución de noticias por medio de RSS de 10 medios de comunicación de cobertura nacional implementado con persistencia estructurada en archivos planos CSV mediante Pandas. *Resultado:* Recolección exitosa de 200 registros de texto en el primer ciclo mensual con una tasa de bloqueo del 0 %. *Limitación:* Sesgo de cobertura mediática (exclusión de portales locales sin RSS público) y carencia absoluta de mecanismos de extracción semántica o discriminación temática.

Prototipo 2 (P2): Clasificación heurística y clustering estadístico (Nov-Dic 2025). Diseño de un clasificador basado en expresiones regulares (RegEx con 47 patrones sintácticos) para identificación de entidades, acoplado a un clasificador estadístico disperso mediante vectorización TF-IDF y agrupamiento K-Means ($k = 8$ estático). *Resultado:* Margen de error superior al 60 % en la detección de casos relevantes, principalmente tres escenarios

críticos de colisión como notas legislativas o estadísticas de violencia de género sin casos objetivos de violencia. *Diagnóstico:* El enfoque sintáctico era incapaz de identificar la relación conceptual entre términos o resolver dependencias de largo alcance en textos no estructurados.

5.2.2. Fase TT-2 Arquitectura de aprendizaje profundo y clasificador escalonado (P3-P4)

Prototipo 3 (P3) Inferencia semántica base y migración relacional (Feb-Mar 2026). Sustitución de las heurísticas rígidas por un clasificador Transformer discriminativo basado en el modelo de lenguaje **BETO** base que operaba inicialmente bajo una estrategia de proximidad semántica por similitud coseno y la migración de la capa de almacenamiento hacia un motor relacional en PostgreSQL. Implementación del flujo de de-duplicación en cascada de 4 capas (MD5 → SimHash → Jaccard → coseno TF-IDF) e integración de la biblioteca *StealthSession* con emulación de JA3 fingerprinting. *Resultado:* Reducción del ruido léxico al 20 % y expansión del catálogo de monitoreo a 45 fuentes web correlativas. *Limitación:* Aparición del “efecto péndulo” donde el clasificador Transformer exhibía una alta afinidad semántica por cercanía léxica lo que hacía que clasificara erróneamente discursos normativos (como iniciativas de la Ley Monzón) o reportes institucionales (como cifras de REDIM) como eventos delictivos reales debido a la coincidencia masiva de tokens comunes.

Prototipo 4 (P4) Fine-tuning optimizado, clasificador escalonado y dashboard (Abr-Jun 2026). Evolución hacia el estado actual de producción del sistema mediante tres aspectos fundamentales de ingeniería de software y machine learning.

1. **Fine-tuning local:** Entrenamiento de las capas de clasificación de BETO utilizando un dataset etiquetado manualmente con los casos positivos reales clasificados en el Prototipo 3. El proceso incorporó técnicas de optimización para mitigar las restricciones de cómputo (8 GB de RAM).
2. **Sistema de clasificación escalonado:** Estabilización del clasificador mediante una arquitectura de decisión híbrida en 3 capas que fue el escudo léxico duro, el filtro temático (aplicación de penalizaciones suaves de $-0,35$ para neutralizar el efecto péndulo), inferencia fina del Transformer ajustado y bonificaciones heurísticas sintácticas para la detección de parentescos ($+0,10$ a $+0,25$).

Resultado: Plataforma robusta con una precisión alta en la identificación de casos relevantes y lista para su adopción operativa por colectivos de búsqueda y organizaciones de derechos humanos.

5.3. Evolución del Stack Tecnológico (TT-1 → TT-2)

Las constantes iteraciones del sistema forzó una refactorización en distintos niveles de las dependencias de software y los paradigmas algorítmicos pues cada limitación hacía replantear directamente una decisión.

Tabla 5.1: Evolución del stack tecnológico

Capa	TT-1 (Prototipo 2)	TT-2 (Prototipo 4 Consolidado)
Lenguaje base	Python 3.10, scripts secuenciales sin abstracción de datos.	Python 3.11, flujo de datos estructurado con modularización formal.
Recolección	10 fuentes RSS mediante la biblioteca <code>requests</code> (vulnerable a bloqueos por estructura HTML).	45 fuentes RSS automatizadas + Google News + <code>StealthSession</code> con emulación de <i>JA3 fingerprinting</i> y evasión perimetral.
Persistencia	Archivos planos CSV manejados con <code>Pandas</code> con carencia de acceso concurrente e integridad referencial.	PostgreSQL + <code>SQLAlchemy</code> con soporte de transaccionalidad e indexación GIN especializada para búsqueda de texto completo (FTS) en español.
Clasificación PLN y Machine Learning	RegEx rígido (47 patrones sintácticos) y similitud estadística TF-IDF con una alta tasa de falsos positivos.	BETO Fine-tuned con inyección de datos sintéticos y reglas heurísticas para una mejor sintaxis relacional.
Agrupamiento (Clustering)	Algoritmo K-Means (k estático) sobre vectores dispersos TF-IDF lo que daba clústeres semánticamente incoherentes.	BERTopic con ajuste dinámico ante corpus reducidos para neutralizar colapsos.
Deduplicación	Validación binaria simple mediante comparación de URLs exactas.	Cascada de 4 fases correlativas: hashing MD5, SimHash de contenido, distancia Jaccard y coseno TF-IDF.
Lógica de Decisión	Lógica binaria determinista basada meramente en coincidencias de expresiones regulares.	Sistema de clasificación escalonado en 4 capas: Escudo léxico, Filtro temático, inferencia BETO y bonificaciones heurísticas sintácticas (+0,10 a +0,25).

5.4. Arquitectura de Microservicios, Contenerización y Gestión de Recursos

La culminación del prototipo 4 se materializó en una arquitectura distribuida definida mediante una orquestación en `docker` y regido por el principio de aislamiento y separación de responsabilidades. El sistema se organiza en tres contenedores independientes que se comunican de forma exclusiva sobre una red virtual privada (`nna-network`) lo que garantiza que los picos de cómputo durante el procesamiento de los datos no desestabilicen el sistema operativo anfitrión.

5.4.1. Contenedor de base de datos `nna-postgres`

El contenedor `nna-postgres` ejecuta el motor relacional **PostgreSQL** (`postgres:16-alpine`) seleccionando una imagen minimizada de 78 MB para maximizar la disponibilidad de RAM para las capas de aprendizaje profundo. Su configuración y políticas de persistencia incluyen:

- **Volumen persistente:** El volumen lógico `pgdata` se monta directamente en la ruta nativa lo que garantiza la

persistencia, atomicidad y supervivencia de los registros estructurados de las víctimas y las tablas analíticas ante reinicios o reconstrucciones del contenedor.

- **Sincronización jerárquica de servicios:** Implementa una verificación de estado. Los contenedores dependientes de la lógica de negocio (`nna-analyzer` y `nna-webapp`) tienen bloqueado su despliegue hasta que este sub-sistema declare un estado saludable lo que erradica las condiciones de carrera en la inicialización de sockets de red.
- **Seguridad perimetral interna:** El puerto nativo 5432 se expone de manera exclusiva hacia la red interna aislada `nna-network`. Al no mapear este puerto hacia la interfaz pública del host de producción se reduce a cero la superficie de ataque por inyección o escaneo de puertos externo.

5.4.2. Contenedor del Backend `nna-analyzer`

El contenedor `nna-analyzer` encapsula la inteligencia del backend ejecutando el flujo de datos mediante un planificador modular. Este módulo gestiona de forma nativa los procesos críticos del ciclo de vida del dato: recolección (Web Scraping masivo), preprocesamiento, deduplicación de noticias y análisis con modelos Transformer. Sus características de diseño metodológico son:

- **Persistencia de pesos:** Incorpora un volumen compartido montado en `/app/models/cache`. Los pesos del modelo **BETO Fine-tuned** (≈440 MB) se almacenan de manera local. Para optimizar el tiempo de ejecución las variables de entorno se redirigen hacia este volumen, lo que neutraliza la latencia de red y evita descargas redundantes en cada ciclo de vida del contenedor.
- **Acoplamiento y mitigación de fallos de red:** Posee una dependencia condicional estricta lo que garantiza la resiliencia del flujo de datos e impide que los hilos de procesamiento inicien si el pool de conexiones de la base de datos no está disponible para la persistencia inmediata.
- **Programación defensiva y ajuste dinámico:** Aloja las configuraciones del flujo de datos híbrido que incluye los parámetros de resiliencia del módulo de clustering `BERTopic`. Al parametrizar de forma dinámica el tokenizador del sub-modelo de representación léxica el contenedor previene de manera autónoma colapsos de ejecución.

5.4.3. Contenedor de interfaz analítica y dashboard web `nna-webapp`

El contenedor `nna-webapp` sirve el dashboard interactivo y la interfaz de control mediante Flask.

- **Consumo descentralizado y API REST:** Expone de forma exclusiva el puerto 5000 hacia el host. Este diseño permite que la interfaz web y la API REST pública actúen como la capa de abstracción final que aísla al usuario analista de la complejidad algorítmica del backend y asegurando un canal limpio para la consulta y visualización de los datos estructurados.

5.5. Especificación de requerimientos

La formalización y el diseño de la arquitectura distribuida fueron guiados por un conjunto de requerimientos funcionales (RF) y no funcionales (RNF) a los que sus criterios de aceptación se validaron bajo el paradigma de prototipado incremental. Con la finalidad de garantizar el seguimiento del avance, la siguiente tabla muestra la matriz de trazabilidad general.

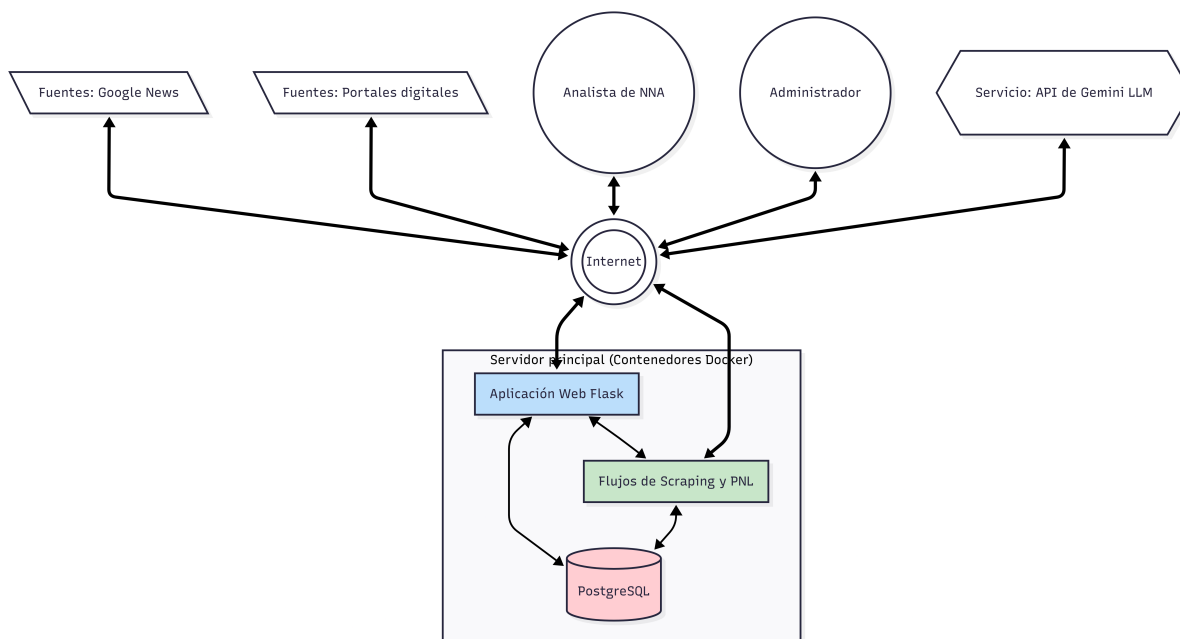


Figura 5.2: Arquitectura del sistema.

Tabla 5.2: Matriz de trazabilidad general

ID	Especificación del Requerimiento	P0	P1	P2	P3	P4
RF-01	Recolección multifuente	~	✓	✓	✓	✓
RF-02	Deduplicación en cascada	—	—	~	✓	✓
RF-03	Clasificación escalonada	—	—	—	~	✓
RF-04	Agrupamiento semántico (BERTopic)	—	—	—	✓	✓
RF-05	Investigación profunda bajo demanda	—	—	—	—	✓
RF-06	Ingreso manual de enlace externo	—	—	—	—	✓
RF-07	Gestión de casos en seguimiento	—	—	—	—	✓
RF-08	Exportación CSV de seguimiento	—	—	—	—	✓
RF-09	Búsqueda de Texto Completo (FTS)	—	—	—	✓	✓
RF-10	Exportación CSV del análisis completo	—	✓	✓	✓	✓
RF-11	Visualizaciones y estadísticas del corpus	—	—	✓	✓	✓
RF-12	Gestión del catálogo de fuentes	—	—	✓	✓	✓
RF-13	Autenticación de usuarios	—	—	—	—	✓
RF-14	Limpieza de historial	—	—	—	—	✓
RNF-01	Optimización de memoria (OOM)	—	—	—	~	✓
RNF-02	Evasión de contramedidas web (WAF)	—	—	—	✓	✓
RNF-03	Resiliencia en APIs externas	—	—	—	—	✓
RNF-04	Integridad de base de datos (ACID)	—	—	—	✓	✓
RNF-05	Procesamiento asíncrono no bloqueante	—	—	✓	✓	✓
RNF-06	Autenticación y control de acceso	—	—	—	—	✓
RNF-07	Compatibilidad dual PostgreSQL/CSV	—	—	—	✓	✓
RNF-08	Trazabilidad por lote de análisis	—	—	—	✓	✓

El análisis de la matriz de trazabilidad demuestra la naturaleza evolutiva del proceso de desarrollo de software. Los prototipos iniciales (P0 a P2) se concentraron exclusivamente en resolver la frontera de especificación sintáctica, garantizando recolección masiva de noticias y manipulación estructurada de datos planos.

El salto significativo hacia el procesamiento profundo en el prototipo 3 introdujo requerimientos semánticos de alta densidad neural, resolviendo las primeras colisiones lógicas provocadas por el solapamiento léxico de los discursos informativos. Y finalmente el prototipo 4 resolvió categóricamente la arquitectura híbrida escalonada y el clustering semántico. Este ultimo consoludo un sistema tecnológico robusto y escalable, listo para someterse a la validación profesional y metodológica.

Requerimientos del sistema

En este capítulo se detalla la especificación de requerimientos para el *Sistema Inteligente para la Identificación y Seguimiento de NNA en Situación de Orfandad por Femicidio en México*. La documentación sigue la metodología y los estándares definidos por la Coordinación de Trabajo Terminal (CDT) de la Escuela Superior de Cómputo (ESCOM).

Se describen primero los actores del sistema y los requerimientos tanto funcionales como no funcionales. Posteriormente, se presenta el modelado dinámico mediante la especificación completa de los casos de uso del sistema. Finalmente, se ilustran y describen las interfaces de usuario (IU) diseñadas para la interacción entre los analistas y la plataforma.

6.1. Perfiles de usuario

A continuación se describen los perfiles de los actores que interactúan con el sistema.

Analista de datos/Trabajador social
Analista



Descripción: Usuario principal del sistema. Encargado de monitorear la recolección automatizada, interpretar los tableros de visualización, realizar investigaciones profundas sobre casos de alta relevancia y gestionar la lista de seguimiento de NNA en situación de orfandad.

Funciones:

- Inicio y cierre de sesión autenticada en el sistema (CU-06).
- Revisión del corpus clasificado (IU-01).
- Búsqueda de texto completo sobre el corpus clasificado mediante FTS con stemming en español (CU-07).
- Ejecución de la investigación profunda bajo demanda (módulo DeepInvestigator) sobre noticias de Alta relevancia, generando una ficha de caso en JSON con 7 campos estructurados (CU-03).
- Ingestión manual de enlaces externos mediante URL arbitraria no capturada por el recolector automático (CU-04).

- Gestión de la lista de seguimiento activo de casos investigados y exportación a CSV (CU-05).

Administrador del Sistema
Admin



Descripción: Encargado del mantenimiento operativo y de la configuración del sistema web. Hereda *todas* las funciones del perfil Analista y adicionalmente gestiona el catálogo de fuentes, las cuentas de usuario y el historial del sistema.

Funciones: • Todas las funciones del perfil Analista.

- Gestión completa (Alta, Baja y Cambios) del catálogo de Fuentes de Información periodística, incluyendo sondeo técnico previo y prueba de extracción de artículos muestra (CU-02).
- Activación y desactivación de fuentes predeterminadas del sistema (el administrador puede eliminarlas permanentemente).
- Registro y administración de cuentas de usuario del sistema (alta de nuevos analistas, habilitación y deshabilitación de cuentas).
- Limpieza del historial completo de análisis: registros en PostgreSQL, archivos CSV del directorio data/ y caché de URLs procesadas(RF-14).

Sistema Automatizado
Procesos



Descripción: Actor lógico de software. Representa al disparador automático de tareas en segundo plano, independiente de la interfaz web. No requiere autenticación de usuario.

Funciones: • Disparar la ejecución iterativa del flujo completo de recolección, deduplicación y análisis semántico en los intervalos configurados por el administrador (CU-01).

- Asignar un identificador de lote único (*batch_id*) a cada ejecución, garantizando la trazabilidad y el filtrado por corrida en el dashboard (RNF-08).

6.2. Requerimientos Funcionales

A continuación se enlistan los requerimientos funcionales del sistema, los cuales definen las capacidades de recolección, análisis, investigación e interacción con el dashboard web.

Requerimientos funcionales					
Id	Nombre		Descripción	Prioridad	Ref.
RF-01	Recolección	multi-fuente	El sistema debe extraer noticias a partir de canales RSS, consultas directas a Google News y scraping HTML con emulación de fingerprint TLS (StealthSession) sobre fuentes sin feed RSS.	Muy Alta	CU-01, CU-02

Requerimientos funcionales				
Id	Nombre	Descripción	Prioridad	Ref.
RF-02	Deduplicación en cascada	El sistema debe filtrar noticias repetidas mediante un flujo de cuatro fases secuenciales: Hash MD5 exacto, similitud de Jaccard sobre tokens, SimHash (distancia Hamming) y similitud coseno TF-IDF.	Alta	CU-01
RF-03	Clasificación escalonada	El motor semántico debe procesar las noticias a través de 4 capas autónomas: capa 0 Escudo léxico (bloqueo absoluto por dominio ajeno), capa 1 Escudo suave (penalización aditiva por contenido normativo), capa 2 Inferencia BETO Fine-Tuned (o Zero-Shot como fallback) y capa 3 Bonificación heurística por evidencia explícita de orfandad.	Muy Alta	CU-01
RF-04	Agrupamiento semántico (BERTopic)	El sistema debe emplear el método UMAP + HDBSCAN + c-TF-IDF de BERTopic para agrupar noticias sobre el mismo caso en tópicos semánticos, con reducción de outliers de 81.4 % a máximo 55 % mediante tres estrategias progresivas.	Alta	CU-01
RF-05	Investigación profunda bajo demanda	El sistema debe sintetizar una ficha de caso estructurada en JSON con siete campos (ubicacion, victimas, ninos_afectados, edades, situacion_actual, medios_contacto, resumen) a partir de una noticia posterior usando DuckDuckGo (HTML + Lite) y el LLM Gemini Flash.	Muy Alta	CU-03
RF-06	Ingreso manual de enlace externo	El usuario debe poder introducir una URL periodística arbitraria para que el sistema extraiga su contenido reutilizando el modulo de investigacion profunda y genere la ficha de caso correspondiente.	Media	CU-04
RF-07	Gestión de casos en seguimiento	El sistema debe permitir al usuario marcar noticias investigadas para generar una lista de monitoreo activo consultable en la pantalla  IU-04 Seguimiento.	Alta	CU-05
RF-08	Exportación CSV de seguimiento	El sistema debe permitir descargar los registros en seguimiento en formato tabular, aplanando los campos del JSON de investigación en columnas individuales (ubicación, víctimas, NNA afectados, edades, situación actual, scores semánticos y enlace original).	Alta	CU-05
RF-09	Búsqueda de Texto Completo (FTS)	El sistema debe permitir búsquedas semánticas sobre el corpus mediante FTS nativo de PostgreSQL con índices GIN y stemming en español (diccionario Snowball), con fallback a búsqueda textual expandida con sinónimos del dominio.	Alta	CU-07
RF-10	Exportación CSV del análisis completo	El sistema debe permitir descargar la totalidad del corpus clasificado en formato CSV, exportando directamente desde PostgreSQL o desde el archivo de datos en memoria como alternativa.	Media	CU-01

Requerimientos funcionales				
Id	Nombre	Descripción	Prioridad	Ref.
RF-11	Visualizaciones y estadísticas del corpus	El sistema debe proveer datos para su visualización: distribución temporal de noticias por mes, distribución de relevancia (Alta/Media/Baja/No relevante) así como la distribución geográfica por estado de México (top 15).	Media	CU-01
RF-12	Gestión del catálogo de fuentes	El administrador debe poder crear, leer, actualizar y eliminar fuentes periodísticas en el catálogo. Las fuentes predeterminadas solo pueden desactivarse, no eliminarse. El sistema debe ofrecer sondeo técnico previo que detecte tipo de fuente, accesibilidad WAF y feeds RSS disponibles sin persistir la URL.	Alta	CU-02
RF-13	Autenticación de usuarios	El sistema debe permitir el inicio y cierre de sesión mediante credenciales (usuario y contraseña), establecer sesiones persistentes con Flask-Login y proteger todos los endpoints del dashboard. Las rutas de salud del sistema (/health) quedan exentas.	Alta	CU-06
RF-14	Limpieza de historial	El sistema debe permitir al administrador borrar completamente el historial de análisis, incluyendo todos los registros de la base de datos PostgreSQL, los archivos CSV del directorio data/ y la caché de URLs procesadas para reiniciar el flujo de recolección y análisis desde cero.	Baja	CU-01

6.3. Requerimientos No Funcionales

Se detallan los requerimientos de calidad, seguridad, desempeño y tolerancia a fallos del sistema inteligente.

Requerimientos No funcionales			
Id	Atributo	Necesidad/Estrategia	Req.
RNF-01	Optimización de memoria (OOM)	Necesidad -El modelo BETO no debe provocar cierres abruptos del proceso por saturación de RAM durante la inferencia en lote. Estrategia -Implementar liberación explícita del caché de PyTorch y reducción dinámica del tamaño del lote (<i>batch-size</i>) a la unidad (<i>batch_size</i> = 1) ante excepciones <i>MemoryError</i> o <i>RuntimeError</i>. La técnica de <i>Gradient Accumulation</i> permite simular lotes grandes usando RAM mínima durante el Fine-Tuning.	CU-01

Requerimientos No funcionales			
Id	Atributo	Necesidad/Estrategia	Req.
RNF-02	Evasión de contramedidas web (WAF)	Necesidad -El recolector debe poder acceder a periódicos digitales protegidos por Web Application Firewalls empresariales (Cloudflare, Akamai) sin ser bloqueado sistemáticamente. Estrategia -Uso del módulo <code>StealthSession</code> con emulación de fingerprint TLS via <code>cloudscraper</code>, delays con distribución log-normal y patrón <i>Circuit Breaker</i> por dominio que evita el bloqueo permanente de IP tras 3 fallos consecutivos.	CU-01, CU-02
RNF-03	Resiliencia en APIs externas	Necesidad -El módulo <code>DeepInvestigator</code> no debe terminar abruptamente si <code>DuckDuckGo</code> bloquea las peticiones de scraping de búsqueda. Estrategia -Implementar fallback hacia la herramienta nativa <code>GoogleSearch</code> de <code>Gemini</code>, ejecutando la búsqueda directamente dentro del modelo sin scraping externo. Si ambos mecanismos fallan, el estado de la investigación pasa a error con notificación al usuario.	CU-03
RNF-04	Integridad de base de datos (ACID)	Necesidad -Los fallos de conexión o errores de validación durante inserciones masivas no deben dejar la base de datos en estados corruptos o con registros parciales. Estrategia -Uso estricto de transacciones atómicas de <code>SQLAlchemy</code> con <i>Rollback</i> automático ante excepciones del tipo <code>OperationalError</code> o <code>IntegrityError</code>. Las restricciones <code>FOREIGN KEY</code> con <code>CASCADE DELETE</code> garantizan la coherencia referencial sin lógica adicional en la capa de aplicación.	CU-01, CU-05
RNF-05	Procesamiento asíncrono no bloqueante	Necesidad -Las operaciones de larga duración (flujos de recolección y análisis) no deben bloquear la interfaz web del Dashboard ni la atención de otras peticiones HTTP. Estrategia -Uso de hilos en segundo plano gestionados con <code>threading.Thread</code> y un mapa de estado en memoria consultado por el frontend mediante <i>polling</i> periódico a los endpoints <code>/api/analyze/status</code> y <code>/api/investigate/status/<id></code>.	CU-01, CU-03
RNF-06	Autenticación y control de acceso	Necesidad -Todos los endpoints del Dashboard y la API REST deben requerir una sesión autenticada. El acceso sin autenticación debe redirigir al formulario de inicio de sesión. Estrategia -Implementación de <code>Flask-Login</code> con el decorador <code>@login_required</code> en todas las rutas excepto <code>/health</code>, <code>/login</code> y <code>/register</code>. Las contraseñas se almacenan como hashes <code>bcrypt</code> y nunca en texto plano. La protección contra redirecciones abiertas (<i>Open Redirect</i>) se valida en el helper <code>_is_safe_url()</code>.	CU-06

Requerimientos No funcionales			
Id	Atributo	Necesidad/Estrategia	Req.
RNF-07	Compatibilidad dual PostgreSQL/CSV	Necesidad -El sistema debe mantener la capacidad de operar sobre datos en formato CSV cuando la base de datos PostgreSQL no esté disponible, garantizando continuidad operativa básica del Dashboard. Estrategia -Implementación del patrón <i>Check-then-Fallback</i> en todos los endpoints de datos: <code>_check_postgres()</code> verifica si existen registros en PostgreSQL; en caso negativo, <code>_load_data()</code> carga el CSV del directorio <code>data/</code>. El campo <code>data_source</code> en cada respuesta JSON indica el motor activo.	CU-01, CU-07
RNF-08	Trazabilidad por lote de análisis	Necesidad -El sistema debe permitir consultar el historial de ejecuciones independientes del flujo y filtrar los datos del Dashboard por lote específico para análisis comparativo. Estrategia -Cada ejecución del flujo asigna un <code>batch_id</code> único a todos los registros insertados en ese ciclo. Los endpoints <code>/api/stats</code>, <code>/api/noticias</code> y <code>/api/batches</code> aceptan el parámetro <code>batch_id</code> (<code>latest</code>, <code>all</code> o un identificador específico).	CU-01

6.4. Modelado de Casos de Uso

A continuación se definen los casos de uso del sistema, describiendo de forma detallada las trayectorias principales, las trayectorias alternativas y el flujo de control para cada una de las interacciones críticas identificadas en el sistema inteligente.



6.5. CU-01 Ejecutar flujo de recolección y procesamiento de noticias

6.5.1. Descripción completa

Este caso de uso describe el flujo automatizado mediante el cual el sistema recolecta, filtra, deduplica y clasifica semánticamente las noticias utilizando la clasificación escalonada.

6.5.2. Atributos importantes

Caso de Uso: CU-01 Ejecutar flujo de recolección y procesamiento de noticias	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos
Propósito:	Alimentar la base de datos con información clasificada, deduplicada y libre del efecto péndulo.
Entradas:	URLs de medios o consultas pre-configuradas.
Origen:	Tabla de fuentes en PostgreSQL.
Salidas:	Textos normalizados y puntajes de clasificación escalonada (Alta, Media, No relevante).
Destino:	Tablas noticias y detecciones.
Precondiciones:	- La BD PostgreSQL debe estar activa. - Debe existir conexión a Internet y proxies/headers configurados.
Postcondiciones:	Se generan nuevos registros clasificados de NNA en orfandad listos para validación.
Errores:	E1: Bloqueo por WAF (ej. Cloudflare HTTP 403), el sistema ignora la fuente y continúa [Trayectoria A]. E2: Falla en BD PostgreSQL (Conexión rechazada o caída en transacción) [Trayectoria B]. E3: Desbordamiento de Memoria (OOM - Out of Memory) por saturación en la inferencia [Trayectoria C].
Tipo:	Caso de uso primario
Observaciones:	Operación crítica, intensiva en procesamiento. Utiliza inferencia en lotes con el modelo BETO Fine-Tuned.

6.5.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Dispara la ejecución del colector mediante el Dashboard o el temporizador del servidor.
 - 2  Obtiene la lista de fuentes habilitadas desde el esquema relacional ✖ E2 Falla BD.
 - 3  Ejecuta la recolección dinámica (StealthSession) evadiendo contramedidas ✖ E1 Bloqueo WAF.
 - 4  Aplica el filtro de la cascada de deduplicación y el algoritmo BERTopic para contexto global.
 - 5  Envía el texto purificado al SemanticDetector para su clasificación escalonada.
 - 6  Aplica la Capa 0 (escudo léxico) y la Capa 1 (escudo suave) para penalizar contenido no fáctico.
 - 7  Ejecuta la Capa 2 (inferencia BETO Fine-Tuned) para obtener el score semántico base ✖ E3 Memoria Saturada.
 - 8  Aplica la Capa 3 (bonificaciones heurísticas) y emite el score final (Alta, Media, No relevante).
 - 9  Almacena los resultados consolidados en la Base de Datos PostgreSQL ✖ E2 Falla BD.
- - - Fin del caso de uso.

Trayectoria alternativa A:

Condición: Ocurre un bloqueo HTTP 403 o 429 durante el scraping

- A1  Registra el evento en los logs del sistema como WARNING.
 - A2  Marca la fuente actual con una bandera temporal de penalización en la memoria (backoff exponencial).
 - A3 El Caso de Uso continúa con la siguiente fuente iterando nuevamente.
- - - Fin de la trayectoria.

Trayectoria alternativa B:

Condición: Falla la conexión o transacción con la base de datos PostgreSQL

- B1  Intercepta la excepción de SQLAlchemy (ej. OperationalError).
 - B2  Emite una alerta crítica en los registros del sistema (CRITICAL) indicando la caída del servicio.
 - B3  Ejecuta un *Rollback* automático de cualquier transacción que haya quedado pendiente.
 - B4  Notifica al administrador mediante el canal de monitoreo configurado.
 - B5 El Caso de Uso termina de manera anormal.
- - - Fin de la trayectoria.

Trayectoria alternativa C:

Condición: El modelo BETO satura la memoria disponible (OOM) durante la inferencia

- C1  Detecta la excepción de asignación de tensores (MemoryError o RuntimeError de PyTorch).
 - C2  Libera explícitamente el caché de memoria (invocando el recolector de basura).
 - C3  Divide dinámicamente el lote de noticias problemático en fragmentos unitarios (tamaño de lote = 1).
 - C4 El Caso de Uso continúa en el paso 7 procesando los fragmentos reducidos.
- - - Fin de la trayectoria.



6.6. CU-02 Gestión del catálogo de fuentes de información

6.6.1. Descripción completa

Permite al administrador gestionar el catálogo de fuentes periodísticas (RSS, HTML, sitemap) que alimentan el flujo de recolección. Gestiona el alta, edición, eliminación, activación/desactivación y sondeo previo de URLs.

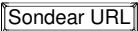
6.6.2. Atributos importantes

Caso de Uso: CU-02 Gestión del catálogo de fuentes de información	
Versión:	2.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Administrador
Propósito:	• Mantener actualizado el catálogo de orígenes periodísticos accesibles por el recolector, garantizando cobertura geográfica y temática del corpus.
Entradas:	• URL de la fuente (obligatoria). • Nombre del medio de comunicación (opcional; se infiere del dominio si se omite). • Tipo de fuente (auto / rss / sitemap / html). • Notas adicionales (campo libre, opcional).
Origen:	• Formulario y tabla de administración en la pantalla  IU-02 Gestión de Fuentes.
Salidas:	• Confirmación visual con los datos de la fuente creada, actualizada o eliminada. • Resultado del sondeo técnico de la URL (tipo detectado, feeds RSS encontrados, crawl-delay, accesibilidad WAF). • Tabla de fuentes actualizada con estado ok / warning / blocked / error.
Destino:	• Tabla news_sources en PostgreSQL. • Interfaz web (Pantalla  IU-02 Gestión de Fuentes).
Precondiciones:	• El actor debe contar con una sesión activa en el Dashboard web (CU-07). • La base de datos PostgreSQL debe estar operativa.
Postcondiciones:	• Las fuentes activas quedan disponibles para el recolector en la próxima ejecución del recolector (CU-01). • Las fuentes predeterminadas del sistema solo pueden desactivarse; no pueden eliminarse.
Errores:	• E1: URL duplicada ya registrada en el catálogo [Trayectoria A]. • E2: Intento de eliminar una fuente predeterminada del sistema [Trayectoria B]. • E3: La URL no inicia con http:// o https:// [Trayectoria C].
Tipo:	Caso de uso primario

Caso de Uso:	CU-02 Gestión del catálogo de fuentes de información
Observaciones:	El sondeo técnico (endpoint POST /api/sources/probe) es una operación no persistente: detecta el tipo de fuente y la accesibilidad WAF <i>antes</i> de registrarla, permitiendo al administrador tomar una decisión informada.

6.6.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Ingresa a la pantalla de  IU-02 Gestión de Fuentes desde el menú superior.
 - 2  Introduce la URL de la nueva fuente en el formulario de alta.
 - 3  Hace clic en  para obtener una vista previa técnica antes de registrar.
 - 4  Invoca `DynamicScraper.probe_url()` que detecta el tipo de contenido (RSS, sitemap, HTML), verifica el archivo `robots.txt` y determina si requiere modo stealth  **E3 URL inválida**.
 - 5  Devuelve el resultado del sondeo al formulario (tipo detectado, feeds RSS encontrados, crawl-delay recomendado, accesibilidad WAF).
 - 6  Revisa los resultados, ajusta el nombre y tipo si fuera necesario, y hace clic en .
 - 7  Verifica que la URL no esté ya registrada  **E1 URL duplicada**.
 - 8  Crea el registro en la tabla `news_sources` con `is_active = True` y `last_status = 'pending'`.
 - 9  Muestra mensaje de confirmación y actualiza la tabla de fuentes en pantalla.
- -- Fin del caso de uso.

Trayectoria alternativa A:

Condición: La URL ingresada ya está registrada en el catálogo

- A1  Detecta el registro duplicado en la consulta de unicidad.
 - A2  Retorna HTTP 409 con el mensaje "Esta URL ya existe como fuente".
 - A3  Lee la notificación de error y puede modificar la URL o desactivar la entrada existente.
 - A4 El Caso de Uso termina sin crear un nuevo registro.
- -- Fin de la trayectoria.

Trayectoria alternativa B:

Condición: El administrador intenta eliminar una fuente predeterminada del sistema

- B1  Detecta que el atributo `is_predefined = True` está activo para el registro.
 - B2  Retorna HTTP 403 con el mensaje "No se puede eliminar una fuente predeterminada. Puedes desactivarla en su lugar".
 - B3  Opta por usar el interruptor  en lugar de eliminar.
 - B4 El Caso de Uso termina sin modificar el registro.
- -- Fin de la trayectoria.

Trayectoria alternativa C:

Condición: La URL ingresada no tiene esquema HTTP válido

- C1**  El validador del backend detecta que la cadena no inicia con `http://` ni `https://`.
 - C2**  Retorna HTTP 400 con mensaje "La URL debe comenzar con `http://` o `https://`".
 - C3**  Corrige la URL en el formulario y reintenta.
 - C4** El Caso de Uso continúa en el paso 7 de la trayectoria principal.
- - - *Fin de la trayectoria.*



6.7. CU-03 Investigación profunda bajo demanda

6.7.1. Descripción completa

Permite al analista activar el módulo DeepInvestigator sobre una noticia clasificada como “Alta” para generar una ficha de caso estructurada mediante búsqueda web y síntesis con IA (Gemini Flash).

6.7.2. Atributos importantes

Caso de Uso: CU-03 Investigación profunda bajo demanda	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos / Trabajador social
Propósito:	Profundizar en los detalles de un caso específico delegando la investigación multi-fuente y la síntesis a agentes de IA.
Entradas:	ID de la noticia seleccionada.
Origen:	Botón “Investigar mas en la web” en la tarjeta de la noticia dentro del Dashboard.
Salidas:	Ficha estructurada en formato JSON renderizada en la interfaz (Ubicación, Víctimas, NNA Afectados, Situación Actual, Medios de Contacto).
Destino:	- Interfaz del Dashboard (Casos objetivo). - Base de datos (almacenamiento de la investigación asociada a la noticia).
Precondiciones:	- La noticia debe estar registrada en el sistema y tener clasificación “Alta”. - Debe existir conexión a Internet y cuota disponible en la API de Gemini.
Postcondiciones:	Se vincula una ficha de investigación estructurada al registro de la noticia.
Errores:	E1: DuckDuckGo no retorna resultados válidos o bloquea la conexión [Trayectoria A]. E2: Exceso de cuota en la API de Gemini o fallo de red con el proveedor [Trayectoria B].
Tipo:	Caso de uso primario
Observaciones:	La operación se ejecuta de manera asíncrona en un hilo de fondo (background thread) para no bloquear la interfaz web del usuario.

6.7.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Analiza el listado de noticias con score “Alta” en el dashboard.

- 2  Selecciona una noticia de interés y hace clic en el botón `Investigar mas en la web`.
 - 3  Cambia el estado de la investigación a `running` y lanza un hilo de ejecución en segundo plano.
 - 4  Invoca a la API de Gemini para generar una consulta de búsqueda altamente específica (máximo 6 palabras) basada en la noticia  **E2 Fallo API Gemini.**
 - 5  Ejecuta la búsqueda de la consulta en DuckDuckGo (HTML y Lite)  **E1 Sin resultados DDG.**
 - 6  Extrae y limpia las URLs resultantes de la búsqueda.
 - 7  Realiza scraping del contenido de las fuentes complementarias utilizando `StealthSession`.
 - 8  Envía el texto multi-fuente consolidado a la API de Gemini junto con el prompt de analista criminal  **E2 Fallo API Gemini.**
 - 9  Recibe y valida el JSON estructurado con los detalles del caso.
 - 10  Guarda el resultado en la base de datos y actualiza el estado a `done`.
 - 11  El frontend, mediante *polling*, detecta el fin del proceso y despliega la ficha estructurada en pantalla.
- -- Fin del caso de uso.

Trayectoria alternativa A:

Condición: DuckDuckGo no retorna URLs utilizables o bloquea la petición

- A1  Activa el mecanismo de "fallback" utilizando la herramienta nativa `GoogleSearch` de Gemini.
 - A2  Ejecuta la búsqueda de información directamente a través del modelo de lenguaje sin scraping externo.
 - A3  Si la búsqueda interna también falla, marca el estado como `error` y notifica al usuario en el dashboard.
 - A4 El Caso de Uso termina o continúa en el paso 8 de la trayectoria principal si el fallback es exitoso.
- -- Fin de la trayectoria.

Trayectoria alternativa B:

Condición: La API de Gemini responde con un error de cuota o falla de conexión

- B1  Captura la excepción de la API externa.
 - B2  Registra el error en los logs.
 - B3  Cambia el estado de la investigación a `error` indicando "Fallo en el servicio".
 - B4  Visualiza el mensaje de error en la interfaz, permitiéndole reintentar más tarde.
 - B5 El Caso de Uso termina de manera anormal.
- -- Fin de la trayectoria.



6.8. CU-04 Identificación por enlace externo

6.8.1. Descripción completa

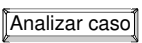
Permite al analista someter una URL periodística arbitraria que no fue capturada por el sistema automático, extraer su contenido, validar su relevancia e investigarla profundamente reutilizando el pipeline del módulo DeepInvestigator.

6.8.2. Atributos importantes

Caso de Uso: CU-04 Identificación por enlace externo	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos / Trabajador social
Propósito:	Incorporar al sistema casos relevantes descubiertos manualmente por el usuario navegando en la web, mitigando los falsos negativos del recolector automatizado.
Entradas:	URL externa del artículo periodístico.
Origen:	Formulario de "Identificar caso" en el Dashboard.
Salidas:	Ficha estructurada en formato JSON (Ubicación, Víctimas, NNA Afectados, etc.) del nuevo caso.
Destino:	- Interfaz del dashboard (Identificación de caso). - Base de datos.
Precondiciones:	- La URL debe ser accesible de forma pública. - El servicio de Gemini Flash debe estar disponible.
Postcondiciones:	Se incorpora una nueva noticia validada e investigada a la base de datos del sistema.
Errores:	E1: La URL apunta a contenido no válido (cookies, Error 403, etc) [Trayectoria A].
Tipo:	Caso de uso primario
Observaciones:	Este módulo actúa como una vía de inyección manual que aprovecha la infraestructura de extracción y análisis ya existente en el sistema.

6.8.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Ingresa al apartado de "Identificar caso" en el dashboard.
- 2  Pega la URL de la noticia en el campo de texto y hace clic en .

- 3  Accede a la URL proporcionada mediante `StealthSession` y extrae el texto del artículo aplicando rutinas de limpieza.
 - 4  Envía el texto extraído a Gemini Flash solicitando la generación de un título corto o la validación del contenido
 - 5  Invoca al módulo interno `DeepInvestigator` utilizando el texto validado y el nuevo título.
 - 6  Genera una consulta específica, busca en DuckDuckGo, hace scraping multi-fuente y sintetiza la ficha (mis-mos pasos que CU-03).
 - 7  Almacena la noticia y su ficha de investigación en la base de datos.
 - 8  Muestra los resultados estructurados en la interfaz del usuario.
- - - *Fin del caso de uso.*

Trayectoria alternativa A:

Condición: La extracción retorna contenido que no corresponde a una noticia válida

- A1  El modelo Gemini evalúa el texto y retorna la bandera `NOTICIA_INVALIDA`.
 - A2  Detiene el flujo de ejecución antes de iniciar la investigación profunda.
 - A3  Retorna un mensaje de error a la interfaz web indicando que la URL parece estar bloqueada, es un aviso de privacidad o no contiene cuerpo periodístico.
 - A4  Lee el error y puede intentar con una URL alternativa.
 - A5 El Caso de Uso termina sin registrar información en la base de datos para no corromper los registros.
- - - *Fin de la trayectoria.*



6.9. CU-05 Gestión y exportación de seguimiento

6.9.1. Descripción completa

Describe el proceso mediante el cual el analista agrega una noticia investigada a la lista de "Seguimiento activo" y posteriormente exporta dicho listado en un archivo CSV enriquecido para su uso en otras herramientas o reportes.

6.9.2. Atributos importantes

Caso de Uso: CU-05 Gestión y exportación de seguimiento	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos / Trabajador social
Propósito:	Facilitar el monitoreo continuo de los casos más críticos identificados por el sistema y proporcionar datos para su análisis mediante la exportación de datos tabulares.
Entradas:	- ID de la noticia (para agregar a seguimiento). - Petición de exportación (clic en botón).
Origen:	Interfaz de resultados de investigación y sección de "Seguimiento de casos" en el Dashboard.
Salidas:	- Confirmación visual en el dashboard. - Archivo descargable en formato .csv con los casos enriquecidos.
Destino:	Navegador del usuario (descarga de archivo).
Precondiciones:	La noticia debe tener una investigación estructurada (DeepInvestigator) completada exitosamente.
Postcondiciones:	La noticia cambia su estado lógico en la base de datos a "En Seguimiento".
Errores:	E1: Intento de exportación cuando la lista de seguimiento está vacía [Trayectoria A].
Tipo:	Caso de uso primario
Observaciones:	La exportación integra de forma transparente los campos relacionales de la noticia original junto con los campos JSON extraídos por el módulo de investigación (edades, ubicación, contacto DIF).

6.9.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Analiza los resultados generados por el módulo DeepInvestigator para un caso específico.

- 2  Determina que el caso requiere monitoreo a largo plazo y hace clic en el botón Dar seguimiento a caso.
 - 3  Actualiza la bandera de seguimiento (`is_tracked = True`) para el registro en la base de datos.
 - 4  Confirma la acción mediante un mensaje en la interfaz.
 - 5  Navega hacia la vista de "Casos en Seguimiento" del Dashboard.
 - 6  Hace clic en el botón Exportar a CSV.
 - 7  Consulta la base de datos extrayendo todos los casos marcados en seguimiento  **E1 Lista Vacía.**
 - 8  Serializa los datos, desempacando el JSON de la ficha investigativa en columnas planas.
 - 9  Transmite el archivo `seguimiento_casos.csv` como una descarga HTTP al navegador.
 - 10  Recibe y guarda el archivo en su computadora local.
- - - *Fin del caso de uso.*

Trayectoria alternativa A:

Condición: El analista solicita la exportación pero no hay casos en seguimiento

- A1  Detecta que la consulta a la base de datos retorna cero registros.
 - A2  Aborta la generación del archivo CSV.
 - A3  Muestra una notificación visual (*Toast* o *Alert*) indicando "No hay casos en seguimiento para exportar".
 - A4 El Caso de Uso termina.
- - - *Fin de la trayectoria.*



6.10. CU-06 Iniciar sesión en el sistema

6.10.1. Descripción completa

Describe el proceso mediante el cual un usuario registrado autentica su identidad en el sistema web para acceder a las funcionalidades protegidas del dashboard.

6.10.2. Atributos importantes

Caso de Uso: CU-06 Iniciar sesión en el sistema	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos / Administrador del sistema
Propósito:	Verificar la identidad del usuario y establecer una sesión autenticada que habilite el acceso a todos los endpoints protegidos del sistema.
Entradas:	- Nombre de usuario (username). - Contraseña (password). - Opción "Recordarme" (booleano, opcional).
Origen:	Formulario de inicio de sesión en la pantalla  IU-05 Login.
Salidas:	- Redirección al dashboard principal  IU-01 Dashboard en caso de éxito. - Mensaje de error visual si las credenciales son incorrectas o la cuenta está deshabilitada.
Destino:	- Sesión Flask-Login (cookie de sesión en el navegador). - Tabla users en PostgreSQL (actualización de last_login).
Precondiciones:	- El usuario debe estar previamente registrado en el sistema. - La base de datos PostgreSQL debe estar operativa.
Postcondiciones:	- Se establece una sesión activa que permite el acceso a todas las rutas protegidas por @login_required. - Se registra la marca de tiempo de último inicio de sesión (last_login).
Errores:	E1: Credenciales incorrectas (usuario no encontrado o contraseña inválida) [Trayectoria A]. E2: La cuenta del usuario está deshabilitada por el administrador [Trayectoria B].
Tipo:	Caso de uso primario
Observaciones:	La protección contra redirecciones abiertas (<i>Open Redirect</i>) se implementa en el helper <code>_is_safe_url()</code> , que valida que la URL de retorno pertenezca al mismo dominio del sistema antes de redirigir.

6.10.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Navega a la URL raíz del sistema; es redirigido automáticamente a la pantalla  IU-05 Login.
 - 2  Introduce su nombre de usuario y contraseña en el formulario y hace clic en  **Iniciar Sesión**.
 - 3  Valida el formato del formulario (campos no vacíos).
 - 4  Consulta la tabla users buscando el registro por username  **E1 Credenciales inválidas.**
 - 5  Verifica que la cuenta esté activa (`is_active = True`)  **E2 Cuenta deshabilitada.**
 - 6  Valida el hash de la contraseña mediante `check_password()`.
 - 7  Establece la sesión de Flask-Login y actualiza el campo `last_login`.
 - 8  Redirige al Dashboard principal  IU-01 Dashboard o a la página solicitada originalmente (parámetro `next`).
- -- Fin del caso de uso.

Trayectoria alternativa A:

Condición: El nombre de usuario no existe o la contraseña es incorrecta

- A1  Registra el intento fallido como `WARNING` en los logs del sistema, incluyendo la IP del cliente.
 - A2  Muestra el mensaje “Usuario o contraseña incorrectos” en el formulario sin revelar cuál de los dos es incorrecto.
 - A3 El Caso de Uso termina; el actor puede reintentar.
- -- Fin de la trayectoria.

Trayectoria alternativa B:

Condición: La cuenta del usuario está deshabilitada

- B1  Detecta `is_active = False` en el registro del usuario.
 - B2  Muestra el mensaje “Tu cuenta está deshabilitada. Contacta al administrador.”
 - B3  Contacta al administrador para solicitar la reactivación.
 - B4 El Caso de Uso termina sin establecer sesión.
- -- Fin de la trayectoria.



6.11. CU-07 Búsqueda y Filtrado de Noticias

6.11.1. Descripción completa

Permite al analista realizar búsquedas de texto completo sobre el corpus de noticias clasificadas, utilizando Full-Text Search de PostgreSQL con stemming en español o búsqueda por sinónimos sobre CSV como fallback, con soporte de filtros por clasificación y estado NNA.

6.11.2. Atributos importantes

Caso de Uso: CU-07 Búsqueda y Filtrado de Noticias	
Versión:	1.0
Autor:	Héctor Alberto Morales Martínez
Supervisa:	Director de Trabajo Terminal
Actor:	Analista de datos / Trabajador social
Propósito:	Localizar rápidamente noticias específicas dentro del corpus clasificado usando términos de búsqueda, filtros de relevancia y restricciones por detección de NNA.
Entradas:	- Término de búsqueda. - Filtro de clasificación (Alta / Media / Baja / No relevante), opcional. - Filtro "Solo NNA" (booleano), opcional. - Límite de resultados (<i>limit</i>), por defecto 20.
Origen:	Barra de búsqueda en la cabecera del Dashboard principal  IU-01 Dashboard.
Salidas:	- Lista de noticias relevantes ordenadas por score de relevancia FTS. - Indicador de la fuente de datos utilizada. - Número total de resultados encontrados.
Destino:	Sección de resultados en la interfaz  IU-01 Dashboard.
Precondiciones:	- El actor debe tener una sesión activa (CU-06). - El corpus debe haber sido procesado por al menos un ciclo del flujo de recolección y análisis (CU-01).
Postcondiciones:	Se despliegan los resultados filtrados en la interfaz sin modificar la base de datos.
Errores:	E1: El término de búsqueda está vacío [Trayectoria A]. E2: No hay datos disponibles (ni PostgreSQL ni CSV) [Trayectoria B].
Tipo:	Caso de uso primario
Observaciones:	El motor FTS de PostgreSQL aplica el diccionario Snowball en español, reduciendo variantes morfológicas de "feminicidio", "feminicida" y "feminicidios" a la misma raíz léxica. El fallback a CSV utiliza el módulo <code>SynonymDictionary</code> para expandir el término de búsqueda con sinónimos del dominio.

6.11.3. Trayectorias del Caso de Uso

Trayectoria principal

- 1  Escribe un término de búsqueda en la barra de búsqueda del Dashboard y presiona  o la tecla Enter.
 - 2  Valida que el parámetro no esté vacío  **E1 Query vacío.**
 - 3  Verifica si hay datos en PostgreSQL (`_check_postgres()`).
 - 4  Si PostgreSQL está disponible, invoca `NoticiasRepository.buscar_fts()` con índices GIN sobre columnas `tsvector` en $O(\log n)$.
 - 5  Si no hay datos en PostgreSQL, aplica búsqueda textual sobre el CSV en memoria, expandiendo el término con el `SynonymDictionary`  **E2 Sin datos.**
 - 6  Retorna la lista de resultados con el campo `data_source` indicando el motor utilizado.
 - 7  Revisa los resultados y puede activar el módulo `DeepInvestigator` sobre cualquier resultado con prioridad Alta(CU-03).
- -- Fin del caso de uso.

Trayectoria alternativa A:

Condición: El término de búsqueda está vacío

- A1  Retorna HTTP 400 con el mensaje "Query requerido".
 - A2  Introduce un término de búsqueda válido.
 - A3 El Caso de Uso termina; el actor puede reintentar.
- -- Fin de la trayectoria.

Trayectoria alternativa B:

Condición: No hay datos disponibles en ninguna fuente

- B1  Detecta que PostgreSQL no tiene registros y el CSV no existe en disco.
 - B2  Retorna HTTP 404 con el mensaje "No hay datos disponibles".
 - B3  Ejecuta primero el flujo de recolección y análisis (CU-01) para generar datos.
 - B4 El Caso de Uso termina.
- -- Fin de la trayectoria.

6.12. Interfaces de Usuario

En esta sección se describen los diseños visuales de las pantallas de la aplicación web correspondientes al panel de control y administración del sistema. Cada interfaz de usuario se detalla con su objetivo, sus campos de entrada, datos de salida, y comandos disponibles (botones y acciones interactivas) vinculados a sus respectivos casos de uso.

6.13. IU-01 Dashboard principal de noticias

6.13.1. Objetivo

Permitir al analista monitorear el corpus de noticias clasificadas así como realizar búsquedas de texto completo y acceder a las funcionalidades de investigación profunda y seguimiento de casos.

6.13.2. Diseño

Esta pantalla es la vista inicial del sistema tras autenticarse. En la cabecera superior se presenta una fila de tarjetas de métricas (widgets) que dicen el total de noticias, noticias con NNA identificados, noticias de Alta relevancia, Media relevancia y número de tópicos BERTopic generados así como un selector de clasificación (Alta / Media / Baja / Todas) que filtran el contenido de forma sincronizada. Debajo un **panel de lista** con tarjetas interactivas paginadas de noticias. Cada tarjeta incluye: título, fuente, fecha, score semántico BETO, etiquetas de clasificación codificadas por color (rojo: Alta, naranja: Media, gris: Baja). La barra superior contiene una barra de búsqueda de texto completo (FTS).

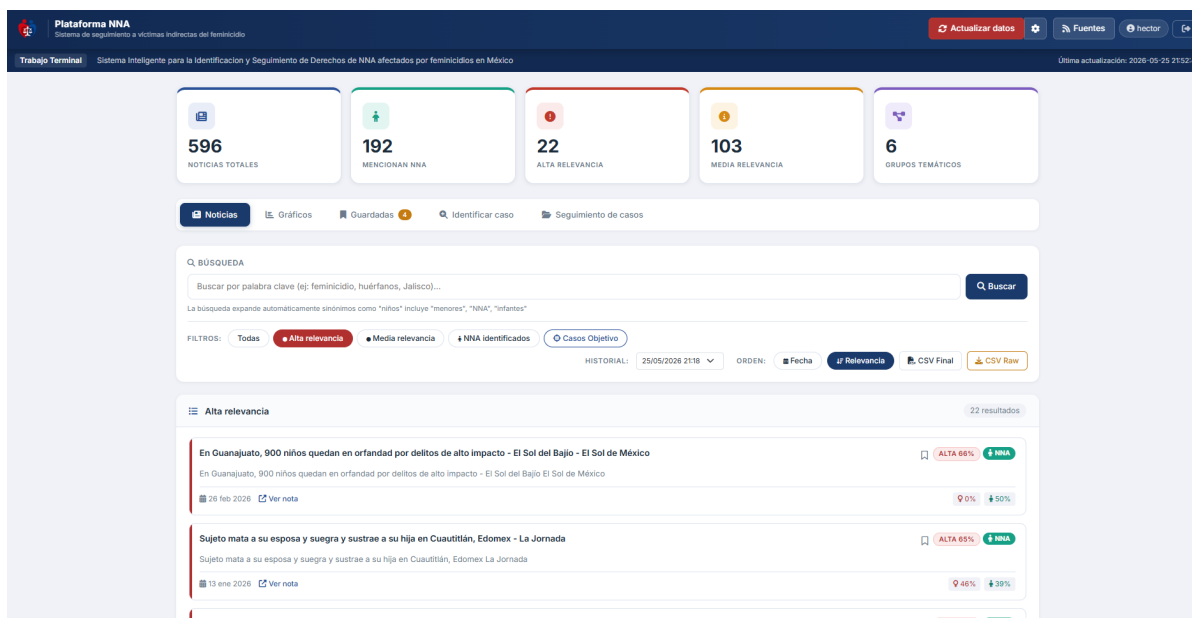


Figura 6.1: IU-01 Dashboard principal de noticias.

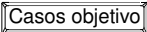
6.13.3. Salidas

- Tarjetas de métricas: total noticias, noticias NNA, Alta, Media, número de tópicos.
- Lista paginada de noticias con título, fuente, fecha y score BETO.
- Notificaciones de estado del análisis de noticias (en ejecución, completado o error).

6.13.4. Entradas

- Término de búsqueda FTS (barra de búsqueda, referencia a CU-07).
- URL externa para investigación profunda (formulario, referencia a CU-04).
- Selector de clasificación (Alta / Media / Baja / Todas).
- Selector de lote de análisis (último, todos o específico).
- Filtro “Solo NNA” (interruptor booleano).

6.13.5. Comandos

-  : Dispara el flujo de recolección y clasificación en segundo plano (CU-01).
-  : Manda a el módulo DeepInvestigator sobre la noticia seleccionada y ejecuta la investigación profunda (CU-03).
-  : Procesa la URL introducida manualmente (CU-04).
-  : Descarga el corpus completo clasificado (RF-10).
-  : Borra todos los registros de la base de datos, los CSVs y la caché de URLs procesadas (RF-14).

6.14. IU-02 Gestión de Fuentes de Información

6.14.1. Objetivo

Permitir al administrador el control del catálogo completo de orígenes periodísticos que alimentan el flujo de recolección, con capacidades de alta, edición, desactivación, eliminación y sondeo técnico previo de nuevas URLs.

6.14.2. Diseño

Consiste en dos paneles. El panel superior presenta un **formulario de alta** con cuatro campos (nombre, URL, tipo de fuente y notas) y el botón de sondeo previo. El panel inferior muestra una **tabla de administración** con todas las fuentes registradas, indicando para cada una: nombre, URL, tipo detectado, estado (ok / warning / blocked / error), fecha del último sondeo y número de artículos encontrados en la última prueba. Las fuentes predeterminadas del sistema aparecen marcadas con un ícono de candado y sin botón de eliminación. Las fuentes personalizadas muestran botones de edición y eliminación. Todas las fuentes cuentan con un interruptor de activación/desactivación.

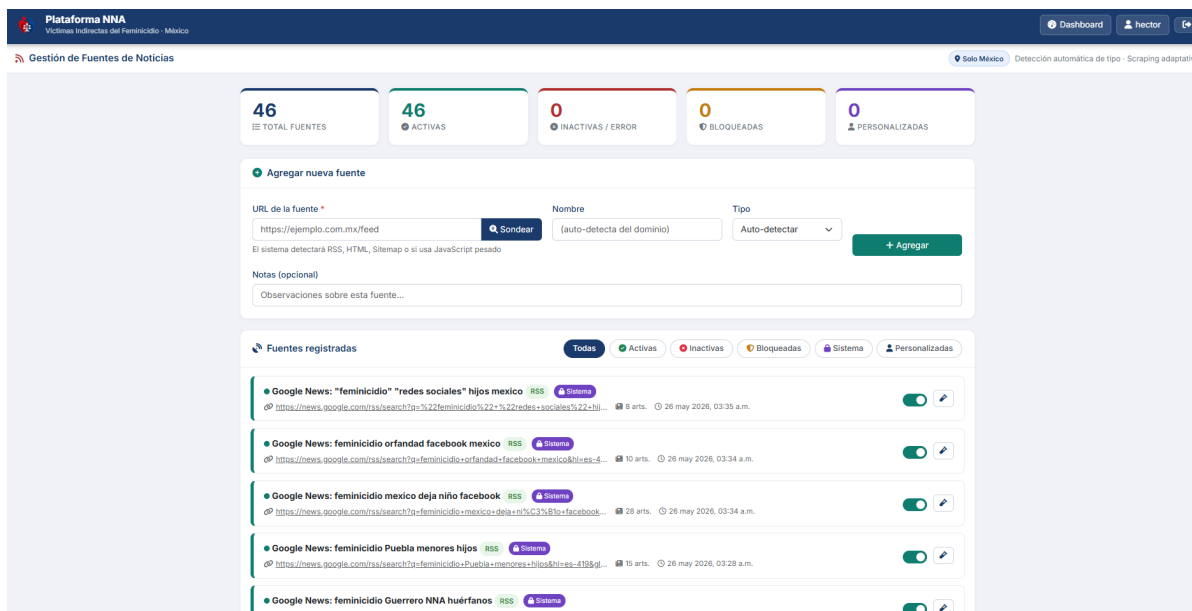


Figura 6.2: IU-02 Pantalla de Gestión de Fuentes.

6.14.3. Salidas

- Resultado del sondeo técnico previo: tipo detectado (RSS / HTML / sitemap), feeds RSS disponibles, crawl-delay, accesibilidad WAF e indicador stealth.
- Tabla de fuentes con columnas: Nombre, URL, Tipo, Estado, Último sondeo, Artículos detectados y Acciones.
- Notificación de éxito al registrar, actualizar o eliminar una fuente.
- Error HTTP 409 si la URL ya está registrada.
- Error HTTP 403 si se intenta eliminar una fuente predeterminada.

6.14.4. Entradas

- URL de la fuente (campo obligatorio, debe iniciar con `http://` o `https://`).
- Nombre del medio de comunicación (opcional; se infiere del dominio si se omite).
- Tipo de fuente (selector: auto / rss / sitemap / html).
- Notas adicionales (texto libre, opcional).

6.14.5. Comandos

- `[Sondear URL]`: Detecta el tipo de fuente y su accesibilidad sin registrarla (POST `/api/sources/probe`). Muestra el resultado en el formulario antes de confirmar el alta (CU-02).

- **Guardar Fuente** : Registra la nueva fuente en la base de datos una vez verificada (CU-02).
- **Probar fuente** : Ejecuta un sondeo completo sobre una fuente ya registrada, incluyendo extracción de artículos de muestra, y actualiza su estado en la tabla (POST /api/sources/<id>/test).
- **Desactivar / Activar** : Alterna el campo is_active de la fuente mediante un interruptor, excluyéndola o incluyéndola en el próximo barrido de recolección (POST /api/sources/<id>/toggle).
- **Eliminar** : Elimina permanentemente una fuente personalizada (DELETE /api/sources/<id>). No disponible para fuentes predeterminadas.

6.15. IU-03 Módulo de investigación profunda

6.15.1. Objetivo

Muestra la síntesis estructurada generada por el agente de Inteligencia Artificial (Gemini Flash) sobre un caso crítico de orfandad.

6.15.2. Diseño

Esta interfaz se presenta como dentro de la pestaña de casos objetivo del Dashboard. Muestra la información estructurada en tarjetas (Ubicación, Víctimas, Instituciones Involucradas, etc.). Durante el proceso, muestra un indicador de carga.



Figura 6.3: IU-03 Modal de Resultados del DeepInvestigator.

6.15.3. Salidas

- Título y resumen generados por el modelo de lenguaje.

- Datos estructurados: Ubicación geográfica del suceso, edades y géneros de las víctimas, NNA afectados.
- URL de referencia y dependencias sugeridas (por ejemplo, DIF Municipal).
- Alerta de error si ocurre un fallo en la API (por ejemplo, cuota excedida o fallo de DDG).

6.15.4. Entradas

- Ninguna (es una interfaz de lectura resultante del CU-03).

6.15.5. Comandos

-  : Ejecuta el flujo de investigación profunda (CU-03).
-  : Etiqueta el caso como relevante e inserta la noticia investigada en la lista de monitoreo activo (CU-05).

6.16. IU-04 Seguimiento de casos y exportación

6.16.1. Objetivo

Proporcionar al analista una vista tabular de todos los casos de orfandad confirmados que requieren atención continua, facilitando la exportación externa de sus datos.

6.16.2. Diseño

Consiste en una tabla de datos (DataGrid) que concentra la información aplanada proveniente de las Fichas de Caso (IU-03). Es una interfaz que prioriza la lectura y el monitoreo masivo, incluyendo controles de paginación y un botón de exportación global en la cabecera.

6.16.3. Salidas

- Tabla plana con metadatos: ID Noticia, Título, Ubicación, Número de NNA afectados, Edad promedio.
- Descarga directa del archivo seguimiento_casos.csv (mediante el navegador web).
- Mensaje tipo Toast indicando si la exportación fue exitosa o si la lista estaba vacía.

6.16.4. Entradas

- Ninguna (los casos son ingresados desde el  IU-03 Módulo de Investigación).

6.16.5. Comandos

-  : Exporta todos los casos activos de la base de datos en un archivo estructurado y lo descarga al equipo del usuario (CU-05).
-  : Marca el caso como inactivo (resuelto o cerrado) y lo desaparece de la vista tabular.

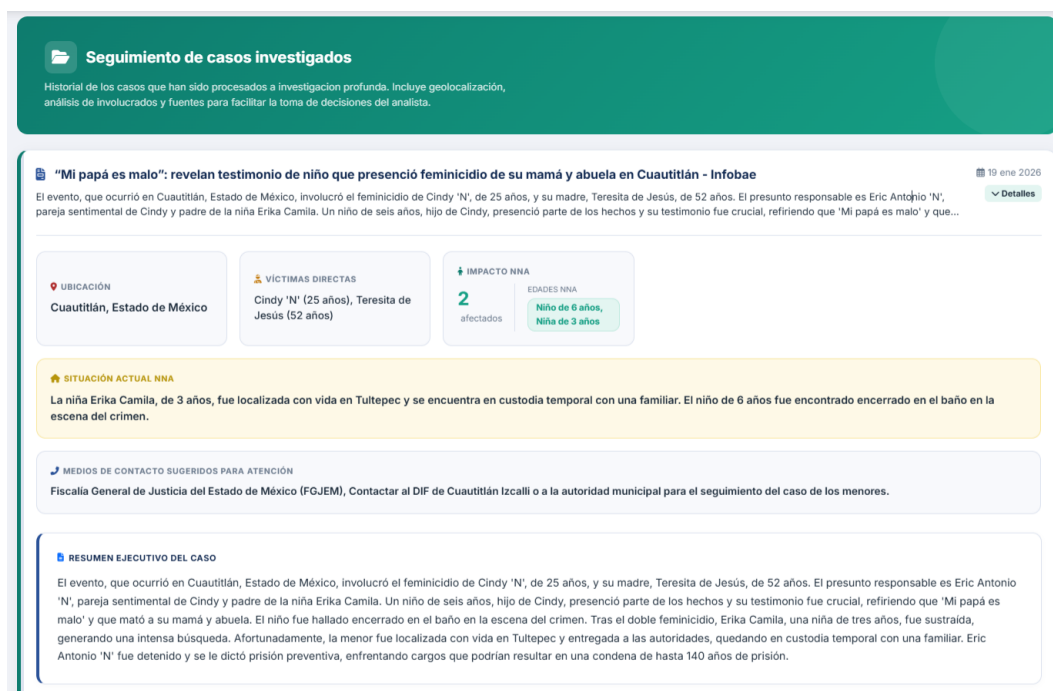


Figura 6.4: IU-04 Pantalla de Seguimiento de Casos Activos.

6.17. IU-05 Pantalla de inicio de sesión

6.17.1. Objetivo

Autenticar la identidad del usuario mediante credenciales registradas para habilitar el acceso a las funcionalidades protegidas del Dashboard web.

6.17.2. Diseño

Pantalla de acceso único al sistema, presentada antes de cualquier vista del Dashboard. Consiste en un formulario centrado con el logotipo del sistema en la parte superior, seguido de dos campos de entrada (usuario y contraseña) y una casilla de verificación opcional "Recordarme". El diseño es minimalista, sin menú lateral ni cabecera de navegación. Un enlace inferior dirige a la pantalla de registro para nuevos usuarios. Los mensajes de error (credenciales incorrectas, cuenta deshabilitada) se presentan como alertas sobre el formulario.

6.17.3. Salidas

- Redirección al  IU-01 Dashboard Principal en caso de autenticación exitosa.
- Mensaje de alerta "Usuario o contraseña incorrectos" si las credenciales son inválidas.
- Mensaje de alerta "Tu cuenta está deshabilitada" si el administrador ha inactivado la cuenta.

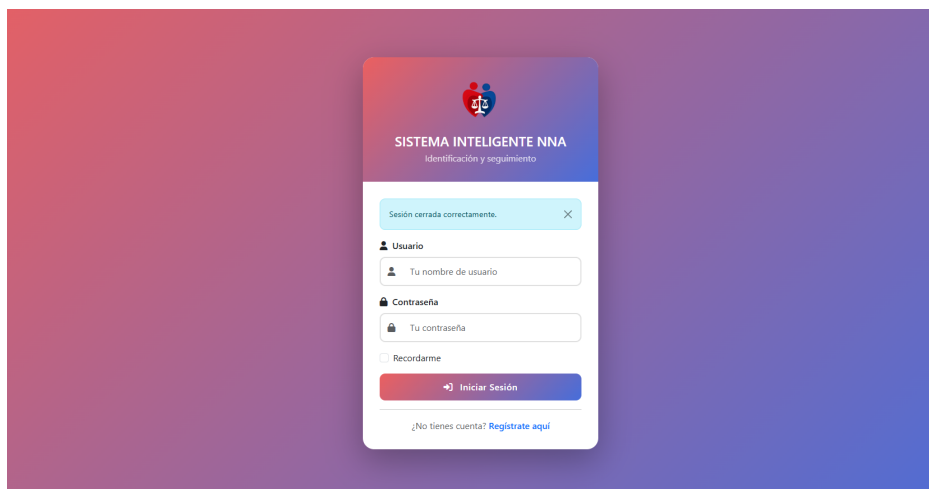


Figura 6.5: IU-05 Pantalla de Inicio de Sesión.

6.17.4. Entradas

- Nombre de usuario (username, campo de texto).
- Contraseña (password, campo enmascarado).
- Opción “Recordarme” (casilla de verificación booleana, opcional).

6.17.5. Comandos

-  : Envía las credenciales al endpoint POST /login para validación (CU-06).
-  : Redirige al formulario de registro de nuevo usuario (GET /register).

Desarrollo de Trabajo Terminal I

En este capítulo se presenta una síntesis de los tres prototipos desarrollados durante el Trabajo Terminal I (agosto - diciembre 2025). El propósito no es poner la misma documentación técnica sino extraer las métricas clave, documentar las decisiones más relevantes y sobretodo argumentar las limitaciones identificadas que justifican la migración del enfoque utilizado. Cada prototipo es presentado como un experimento pues su valor radica tanto en lo que logró como en lo que demostró no ser viable.

7.1. Prototipo 0 Scraping directo sobre HTML

7.1.1. Premisa y enfoque

El prototipo0 se planteó como una prueba de concepto que fue verificar si era posible extraer noticias directamente del HTML de los sitios de noticias mexicanas, utilizando BeautifulSoup y la librería requests. El supuesto inicial era que los sitios de noticias de acceso público serían accesibles de forma programática sin fricción alguna.

7.1.2. Resultado

La premisa fue invalidada en su totalidad pues el sistema se encontró con dos barreras técnicas.

1. **WAF y Cloudflare (HTTP 403/503).** Cerca del 78 % de los medios objetivo (incluyendo portales como *Reforma*, *El Universal* y *Milenio*) operaban detrás de un WAF de grado empresarial. Lo que hacia la librería requests era generar un hash TLS/JA3 determinístico que los sistemas de detección identificaban como un cliente automatizado, recibiendo como respuesta HTTP 403 (Forbidden).
2. **Contenido dinámico renderizado en cliente.** Múltiples sitios entregan el marcado HTML del artículo mediante llamadas AJAX en JavaScript ejecutadas en el navegador. BeautifulSoup al trabajar sobre el HTML estático descargado solo encontraba el esqueleto de la página sin el contenido informativo lo que hacia imposible la extracción.

7.1.3. Decisión Técnica

El fallo del prototipo 0 hizo evidente que el scraping al HTML directo no era una estrategia viable para el monitoreo de medios de noticias modernos. Por lo que se decidió abandonar el scraping directo y migrar hacia fuentes diseñadas para la ingestión automatizada, como lo son los feeds RSS y las APIs.

7.2. Prototipo 1 RSS + K-Means

7.2.1. Arquitectura implementada

El prototipo 1 estableció la primera iteración funcional del sistema. La migración hacia feeds RSS eliminó la fricción de acceso pues los feeds son recursos XML diseñados explícitamente para el consumo automatizado y no están sujetos a los mecanismos WAF que bloquearon el prototipo 0.

El flujo implementado consistió en las siguientes etapas:

1. **Recolección.** Recolección de 8 fuentes RSS nacionales mediante feedparser, extrayendo por cada <item> el título, descripción, enlace y fecha de publicación.
2. **Detección rudimentaria de NNA.** Función `detect_children_mentions` con 15 palabras clave aisladas (“hijo”, “menor”, “niño”, “huérfano”, etc.).
3. **Vectorización TF-IDF.** Representación de documentos en $\mathbb{R}^{1000}$ mediante `TfidfVectorizer` con `max_features=1000`, `ngram_range=(1, 1)`.
4. **Agrupamiento K-Means.** Clustering con $k = 4$ fijo y métrica euclidiana sobre la matriz TF-IDF.
5. **Persistencia.** Exportación a CSV mediante `pandas`.

Volumen de datos: 96 noticias únicas por ejecución con persistencia en archivos planos CSV.

7.2.2. Limitaciones identificadas

Las limitaciones del P1 fueron de tres órdenes:

Techo semántico del detector. La detección por palabras clave aisladas generó una tasa de falsos positivos del **80 %** pues eran noticias sobre legislación (“aprobación de ley de protección al menor”), campañas de prevención o estadísticas de incidencia que activaban el detector sin representar casos concretos de orfandad.

Dimensionalidad escasa en K-Means. El algoritmo asume clústeres globulares de tamaño similar y requiere especificar el número de clústeres k . Con 96 documentos y $k = 4$ el sistema forzaba eventos periodísticos distintos en el mismo clúster para balancear tamaños, generando grupos sin coherencia semántica real. El *Silhouette Score* resultante fue de **0.23** (baja calidad de agrupamiento). El vocabulario limitado a 1,000 características también excluía términos especializados del dominio como “custodia”, “DIF” y “tutor legal”.

Cobertura. El sistema cubría solo medios nacionales con feed RSS público, excluyendo la prensa regional donde suelen aparecer los casos más específicos de feminicidios con NNA identificados.

7.3. Prototipo 2 Triple Fuente + DBSCAN + FeminicideDetector

7.3.1. Evolución del sistema

El prototipo 2 que fue la culminación del TT1 pues reestructuró el sistema en respuesta directa a las limitaciones del P1. Los cambios fueron sustanciales en las tres capas del flujo de procesamiento.

Recolección Se amplió la estrategia de recolección de noticias a tres fuentes de información: (a) **10 feeds RSS** nacionales y de género (incorporando CIMAC Noticias y Milenio que son especializadas en violencia de género); (b) **Google News** con 5 consultas de dominio ("*feminicidio México*", "*asesinato mujer*", "*violencia género*"), (c) **Historical scraper** mediante paginación mensual de Google News, cubriendo el período mayo–noviembre 2025 (6 meses de noticias pasadas).

Detección Se reemplazó la función rudimentaria por una clase especializada con **tres categorías de patrones** como 40 patrones de feminicidio (incluyendo patrones contextuales como *r'mujer.*asesinada'* o *r'cadáver.*femenino'*), 15 patrones de NNA (patrones relacionales como *r'hijo.*huérfano'* o *r'adolescente.*perdió.*madre'*) y **17 patrones de exclusión**. Esta última categoría fue un cambio importante pues al detectar términos como *r'ley.*protección'*, *r'estadística'* o *r'campaña.*prevención'* la noticia era descartada automáticamente. Además el sistema asignaba un *score* de confianza continuo y una prioridad discreta (Alta/Media/Baja/Irrelevante).

Agrupamiento El cambio de K-Means a DBSCAN respondía a las propiedades de la distribución de noticias periódicas pues la mayoría de los feminicidios son eventos únicos con cobertura mínima (1 o 2 noticias) mientras que un evento mediáticamente relevante (como el caso de Ángela Birkenbach) genera una densidad local de 5 a 10 noticias. DBSCAN identificaba estas densidades sin requerir un valor para el número de clústeres y así asignaba como ruido los eventos que si son únicos y que precisamente son los más valiosos para el sistema pues representan casos individuales.

7.4. Limitaciones identificadas

7.4.1. Falsos positivos

El módulo de detección FeminicideDetector redujo los falsos positivos del 80 % (P1) al 60 % (P2) que si fue un avance significativo pero el análisis de los falsos positivos revelaba un patrón que no podía resolverse únicamente con expresiones regulares:

- "*Durango aprueba la Ley Nicole para prohibir cirugías estéticas en menores de edad*". Esta noticia activa el patrón de NNA ("*menores de edad*") sin que los patrones de exclusión capturaran el término *.ªprueba*" en ese contexto legislativo específico.
- *REDIM reporta: 1 de cada 4 NNA vive en situación de violencia familiar*". El término "NNA" y "violencia" activan el detector en una nota estadística sin ningún caso concreto identificado.

El problema de fondo no es la cantidad de patrones pues el sistema ya tenía 72, sino la **naturaleza sintáctica** del enfoque porque la co-ocurrencia de términos no implica la co-ocurrencia de los *roles semánticos* que esos términos desempeñan en la oración. Una expresión regular no puede distinguir si "*menor*" en una noticia es la víctima directa (una menor asesinada), la víctima indirecta (el hijo huérfano de la madre asesinada) o el sujeto de una campaña legislativa. Esta distinción es esencial para el dominio del sistema y fue exactamente el tipo de comprensión que los modelos de lenguaje bidireccionales como BETO proveen a través de sus mecanismos de atención en donde la representación de cada token es función del contexto completo de la oración.

7.4.2. Clustering léxico

Los clústeres identificados por DBSCAN eran semánticamente coherentes pero cerca del 80 % de outliers que aunque eran correctos para eventos únicos ocultaban un problema más y es que las noticias sobre el *mismo evento* que eran redactadas con vocabulario diferente por distintos medios no eran agrupadas.

Ejemplo documentado: "Madre es asesinada y deja 2 hijos huérfanos en Ecatepec" (nota policiaca de El Sol de México) y "Feminicidio en el Estado de México: dos menores quedan sin madre" (nota de derechos humanos de CIMAC Noticias) reportaban el mismo caso con vocabularios distintos. Los vectores TF-IDF de ambas noticias tenían una similitud coseno de 0.18 (muy por debajo del umbral $\epsilon=0.8$) por lo que DBSCAN los clasificaba como documentos independientes. En el espacio semántico de BETO los documentos tendrían embeddings con similitud coseno superior a 0.85 lo que hacía que fueran correctamente agrupados como instancias del mismo evento.

7.4.3. La poca escalabilidad del almacenamiento en CSV

La persistencia en archivos CSV impone restricciones estructurales.

- **Ausencia de integridad referencial.** No existe mecanismo que garantice que una detección apunte a una noticia existente.
- **Sin acceso concurrente.** La escritura por el worker y la lectura por el dashboard desde el mismo archivo CSV generaban condiciones de carrera lo que en el prototipo 2 se "solucionó" temporalmente mediante escritura atómica por renombrado pero el problema no se eliminó.

7.4.4. Síntesis

Las limitaciones anteriores nos llevan a la conclusión de que el problema de la identificación de NNA en situación de orfandad por feminicidio **requiere comprensión semántica del lenguaje natural** no solo reconocimiento de patrones léxicos. El texto de una noticia periodística es un artefacto lingüístico donde el significado emerge de la interacción entre entidades, roles semánticos y contexto, dimensiones que están fuera del alcance de las expresiones regulares.

La siguiente tabla muestra los objetivos cuantitativos que el Trabajo Terminal II debe alcanzar para superar lo establecido en el Trabajo Terminal I.

Tabla 7.1: Comparativa de métricas de TT1 contra metas del TT2

Métrica	TT1 (P2)	Meta TT2 (P4)	Mejora
Falsos positivos	60 % menos del 30 %	-50 %	
Fuentes monitoreadas	10 RSS + Google News	45 RSS + Google News + Stealth	+350 %
Cobertura temporal	6 meses	Continua	Operacional
Persistencia	CSV (plano)	PostgreSQL 16 (ACID)	Estructural
Latencia de consulta	$O(n)$ scan CSV	$O(\log n)$ índice GIN	Órdenes de magnitud
Motor de clasificación	RegEx (72 patrones)	BETO fine-tuning	Paradigma distinto

Las metas de la tabla 7.1 son los objetivos mínimos que hacen del sistema una herramienta operativamente confiable para organizaciones civiles que trabajan con casos reales de NNA en situación de vulnerabilidad.

Desarrollo de Trabajo Terminal II

Este capítulo constituye el desarrollo del Trabajo Terminal II. A diferencia del enfoque descriptivo empleado en el capítulo anterior, la narrativa aquí es evolutiva y autocrítica pues cada decisión de diseño se presenta como respuesta directa a una deficiencia medida y documentada de la iteración anterior. El Prototipo 3 y el Prototipo 4 no surgieron de una planificación inicial sino de un proceso de *diagnóstico, rediseño e integración incremental* que hizo posible transformar un clasificador heurístico basado en palabras clave en una plataforma robusta y escalable.

La arquitectura resultante opera sobre tres objetivos específicos derivados del protocolo 2026-A135: (OE-1) Identificar y seleccionar fuentes públicas confiables que contengan información relacionada con feminicidios y posibles víctimas indirectas, (OE-3) Implementar técnicas de Procesamiento de Lenguaje Natural para extraer y estructurar información clave a partir de textos no estructurados, y (OE-4) Diseñar una base de datos estructurada que permita almacenar la información recolectada de manera eficiente y segura. Estos tres ejes gestionan todas las decisiones técnicas documentadas en las siguientes secciones.

8.1. Contexto: Limitaciones del Prototipo 2

A finales del semestre pasado (2026-1) el sistema hasta el momento presentaba tres limitaciones estructurales que motivaron un rediseño del sistema.

Limitación 1 Semántica del motor RegEx. El módulo *FeminicideDetector* del prototipo 2 evaluaba la *co-ocurrencia espacial* de tokens léxicos sin comprender el rol gramatical de cada entidad. La tasa de falsos positivos vista manualmente alcanzó el **60 %** y notamos que se manifestaba principalmente en tres escenarios:

1. Titulares como "*Menor de edad detenido por feminicidio*" donde el modelo detectaba la co-ocurrencia de "menor" y "feminicidio" lo que activaba la alerta de orfandad cuando en realidad el menor era el agresor y no una víctima indirecta.
2. Noticias estadísticas gubernamentales ("*10 feminicidios mensuales, 5 huérfanos en promedio*") que activaban la alerta de orfandad al concentrar todos los tokens detonantes en un solo párrafo.
3. Noticias donde la víctima directa del feminicidio era una menor de edad y eran confundidas sistemáticamente con casos de orfandad colateral.

Limitación 2 Cuello de botella en persistencia plana. El almacenamiento en archivos CSV (`noticias.csv`) impedía la ejecución de consultas de filtrado sin cargar la totalidad del corpus en memoria RAM ($O(n)$ sobre el conjunto completo). Con un volumen de 250 registros en el prototipo 2 la latencia era tolerable pero proyectado a escala nacional (que son miles de noticias) esta arquitectura no lo soportaría y la ausencia de integridad referencial hace difícil mantener relaciones consistentes entre noticias, fuentes y resultados de análisis.

Limitación 3 Clustering K-Means desacoplado del clasificador. El agrupamiento temático se ejecutaba *después* de la clasificación individual lo que privaba al clasificador del contexto global. Una noticia genérica sobre política criminal podía obtener el mismo score heurístico que un caso concreto de feminicidio con NNA afectados y esto sin que el sistema supiera en que contexto se encontraba.

Estas tres limitaciones nos ayudaron a definir el alcance del sistema y el orden de implementación de las soluciones.

Limitación 4 Ausencia de investigación profunda por caso. El pipeline del TT1 producía clasificaciones de relevancia pero no extraía información estructurada sobre los casos específicos: quién era la víctima directa, cuántos menores quedaron huérfanos, en qué estado se encontraban o qué institución estaba a cargo. Esta limitación era especialmente crítica porque el propósito central del sistema no es solo detectar noticias relevantes sino construir fichas de caso para facilitar la intervención de los investigadores. La información estructurada disponible en el web abierto (notas complementarias, boletines del DIF, seguimientos periodísticos) no estaba siendo aprovechada en ninguna etapa del pipeline.

8.2. Patrones arquitectónicos del flujo analítico

Para garantizar la mantenibilidad y extensibilidad del sistema, el flujo analítico implementa dos patrones de diseño que desacoplan responsabilidades.

8.2.1. Patrón *Repository* para la capa de persistencia

El acceso a PostgreSQL se encapsula en la clase `NoticiasRepository` (`src/database/repository.py`), siguiendo el patrón *Repository* de Martin Fowler [?]. Esta decisión tiene dos objetivos: (a) aislar la lógica de negocio del motor de base de datos, permitiendo sustituir PostgreSQL por cualquier otro backend sin modificar el Flujo analítico; (b) centralizar la gestión de sesiones SQLAlchemy, evitando así fugas de conexiones (*connection leaks*) en un entorno Flask multi-hilo. El repositorio expone métodos tipados que internamente utilizan los índices invertidos de PostgreSQL sobre columnas `tsvector` lo que garantiza búsquedas en $O(\log n)$ frente al $O(n)$ del filtrado CSV.

8.2.2. Patrón *Pipeline* para el flujo analítico

El orquestador principal `SimplifiedNewsAnalyzer` materializa el patrón *pipeline* mediante diez pasos discretos y ordenados. Cada paso recibe el estado modificado por el anterior a través del atributo compartido `self.df.processed`, y puede activarse o desactivarse mediante *feature flags* booleanos en la llamada a `run_complete_analysis()`.

Este diseño permite por ejemplo desactivar la funcionalidad de `enable_bertopic` en entornos con memoria RAM limitada (debajo de 4 GB) sin comprometer la clasificación semántica ni la persistencia, preservando así el principio de responsabilidad única de cada módulo.

8.3. Prototipo 3 Motor semántico BETO y migración de persistencia

El Prototipo 3 introduce simultáneamente las dos soluciones de mayor impacto arquitectónico que por un lado es la sustitución del motor sintáctico por BETO y por el otro es la migración del almacenamiento plano a PostgreSQL. Ambas transformaciones son interdependientes pues PostgreSQL provee la infraestructura relacional que permite almacenar

los nuevos campos semánticos generados por BETO (`score_semantico`, `modo_deteccion`, `topic_id`) con integridad referencial.

8.3.1. Migración de CSV a PostgreSQL

La migración del sistema de persistencia es uno de los puntos de mayor impacto para el TT2. La justificación técnica de abandonar CSV en favor de PostgreSQL se basa en tres argumentos que se hicieron evidentes durante el TT1.

1. **Integridad referencial.** Una arquitectura multi-tabla (noticias, fuentes, clusters, batches de ejecución) requiere claves foráneas con restricciones `FOREIGN KEY` que garanticen la consistencia de los datos. Los archivos CSV no tienen mecanismo alguno de relación entre entidades; cualquier join debía realizarse en memoria con Pandas cosa que es costosa en términos de RAM.
2. **Búsqueda de texto completo (FTS).** PostgreSQL implementa FTS nativa mediante columnas `tsvector` e índices GIN. Una consulta como "niños huérfanos DIF Veracruz" se traduce a una intersección de vectores léxicos en $O(\log n)$. El equivalente en CSV exige iterar cada registro y aplicar expresiones regulares en $O(n)$.
3. **Concurrencia segura.** El dashboard en Flask y el flujo de extracción analítica pueden ejecutarse simultáneamente sin corrupción de datos, gracias al modelo ACID de PostgreSQL. La escritura concurrente en un mismo archivo CSV requería locks manuales propensos a condiciones de carrera.

El esquema de la tabla principal noticias almacena campos como `score_semantico` `FLOAT`, `topic_id` `INTEGER`, `clasificacion_final` `VARCHAR(20)` y `search_vector` `tsvector`. Este último campo se actualiza automáticamente por un *trigger* PostgreSQL que concatena título y contenido al momento de la inserción.

8.3.2. Arquitectura del motor BETO

Al inicio del TT2 se definió que el motor RegEx con expresiones regulares ya había alcanzado su límite de optimización por lo que se decidió migrar al modelo BETO (*bert-base-spanish-wwm-cased*) que responde a una necesidad de *comprensión contextual* que ningún sistema basado en patrones léxicos puede proveer. BETO es un modelo Transformer bidireccional con **110 millones de parámetros**, 12 capas de autoatención y embeddings de 768 dimensiones, preentrenado sobre Wikipedia en español y corpus OPUS mediante el objetivo de Modelado de Lenguaje Enmascarado (MLM).

La arquitectura interna del clasificador implementada en la clase `SemanticDetector` opera según el siguiente flujo:

1. **Tokenización WordPiece.** El texto de entrada (título concatenado con los primeros 1,500 caracteres del contenido separados por el token especial [SEP]) es fragmentado mediante el algoritmo WordPiece que descompone palabras fuera de vocabulario en sub-tokens reconocibles. Esto resuelve el problema de variantes ortográficas comunes en el periodismo mexicano.
2. **Forward pass a través de 12 capas Transformer.** Cada capa aplica mecanismos de *self-attention* multi-cabeza que recalculan el valor de cada token en función de su contexto bidireccional completo. El resultado es que el token "menor" quiere representaciones vectoriales radicalmente diferentes dependiendo de si aparece como sujeto activo ("un menor agredió") o como objeto pasivo ("sus hijos menores quedaron huérfanos").
3. **Extracción del embedding [CLS].** El token especial [CLS] (primer token de toda secuencia BERT) acumula durante el *forward pass* una representación condensada del significado global del texto. Se extrae el *hidden state* de este token como un vector de 768 dimensiones que posteriormente se normaliza mediante norma L2 para habilitar la comparación por similitud coseno.

8.3.3. Clasificación Zero-Shot y Temperature Scaling

Dado que el proyecto no disponía de un corpus etiquetado de suficiente volumen para fine-tuning supervisado al inicio del TT2 se optó por el paradigma de clasificación **zero-shot** por similitud coseno. El sistema pre-calcula embeddings BETO para dos descripciones de categorías cuidadosamente redactadas (en `semantic_detector.py`) que especifican en lenguaje natural qué constituye una noticia relevante contra una "no relevante" para el dominio del sistema. La categoría relevante describe explícitamente *un caso concreto de feminicidio donde los hijos de la víctima quedaron en orfandad o resguardo institucional*, mientras que la categoría "no relevante" enumera explícitamente los falsos positivos que fueron recurrentes en TT1 como notas estadísticas, políticas públicas, feminicidios de menores y agresores menores de edad.

En tiempo de inferencia el embedding del texto de la noticia se compara contra ambas descripciones mediante producto punto el cual es equivalente a similitud coseno para vectores normalizados. Las similitudes crudas tienden a ser extremadamente cercanas con diferencias de 0.02-0.05 lo que genera incertidumbre en la clasificación. Para resolver este problema se aplica **Temperature Scaling** con factor térmico 5, que actúa como amplificador de la separación probabilística antes de la función Softmax.

Este mecanismo es análogo al *temperature scaling* utilizado en calibración de modelos de clasificación [?] solo que aplicado en dirección inversa pues en lugar de suavizar distribuciones sobre-confiadas, se *agudiza* una distribución cruda sub-confiada para forzar una toma de decisión clara.

8.3.4. Evolución del recolector

La recolección de noticias enfrentó durante TT1 una barrera que los prototipos iniciales no pudieron superar que fueron los *Web Application Firewalls* (WAF) de Cloudflare y Akamai que detectaban y bloqueaban las sesiones HTTP del scraper mediante **TLS/JA3 fingerprinting**. Este mecanismo compara el *hash* criptográfico del saludo TLS del cliente (que identifica sin error la librería SSL utilizada) contra el User-Agent declarado. Una sesión que declara ser Chrome/124 pero cuyo handshake TLS corresponde a Python/requests es inmediatamente bloqueada como bot.

En el Prototipo 3 se introduce *StealthSession* que resolvía este problema mediante tres técnicas complementarias:

1. **Emulación de fingerprint TLS via cloudscraper.** La librería *cloudscraper* reemplaza la pila TLS de Python con una que genera hashes JA3 idénticos a los de Chrome/Windows lo que hace que el handshake sea criptográficamente indistinguible de un navegador real. Además resuelve automáticamente los *JavaScript challenges* de Cloudflare (challengepage 403/503) que desde el Prototipo 0 no se habían podido superar.
2. **Delays con distribución.** En TT1 los delays entre requests eran uniformes $U(8, 20)$ segundos, lo que lo hacía un patrón detectable por cualquier WAF moderno que analice la distribución temporal de las solicitudes. La distribución uniforme tiene densidad constante en todo el intervalo pero los humanos reales tienen un patrón asimétrico con *cola larga* donde la mayoría de acciones son rápidas (como clicks o scrolls) y ocasionalmente hay pausas largas (como la lectura de un artículo). Este comportamiento es capturado por la distribución log-normal con parámetros $\mu = 2,5$, $\sigma = 0,7$, que genera una mediana de aproximada de 12.2 segundos y colas que alcanzan hasta 45 segundos.
3. **Circuit Breaker por dominio.** El patrón *Circuit Breaker* (Fowler, 2014) evita saturar dominios que fallan repetidamente. Lo que hace que después de 3 fallos consecutivos en un dominio este circuito se abra y rechace requests por 300 segundos, pasándolo a un estado semi-abierto para permitir un request de prueba y si este tiene éxito el circuito se cierra pero si falla el cooldown se extiende a 600 segundos. Este mecanismo reduce el riesgo de un bloqueo permanente de IP al evitar presionar un sitio que activamente está bloqueando las sesiones.

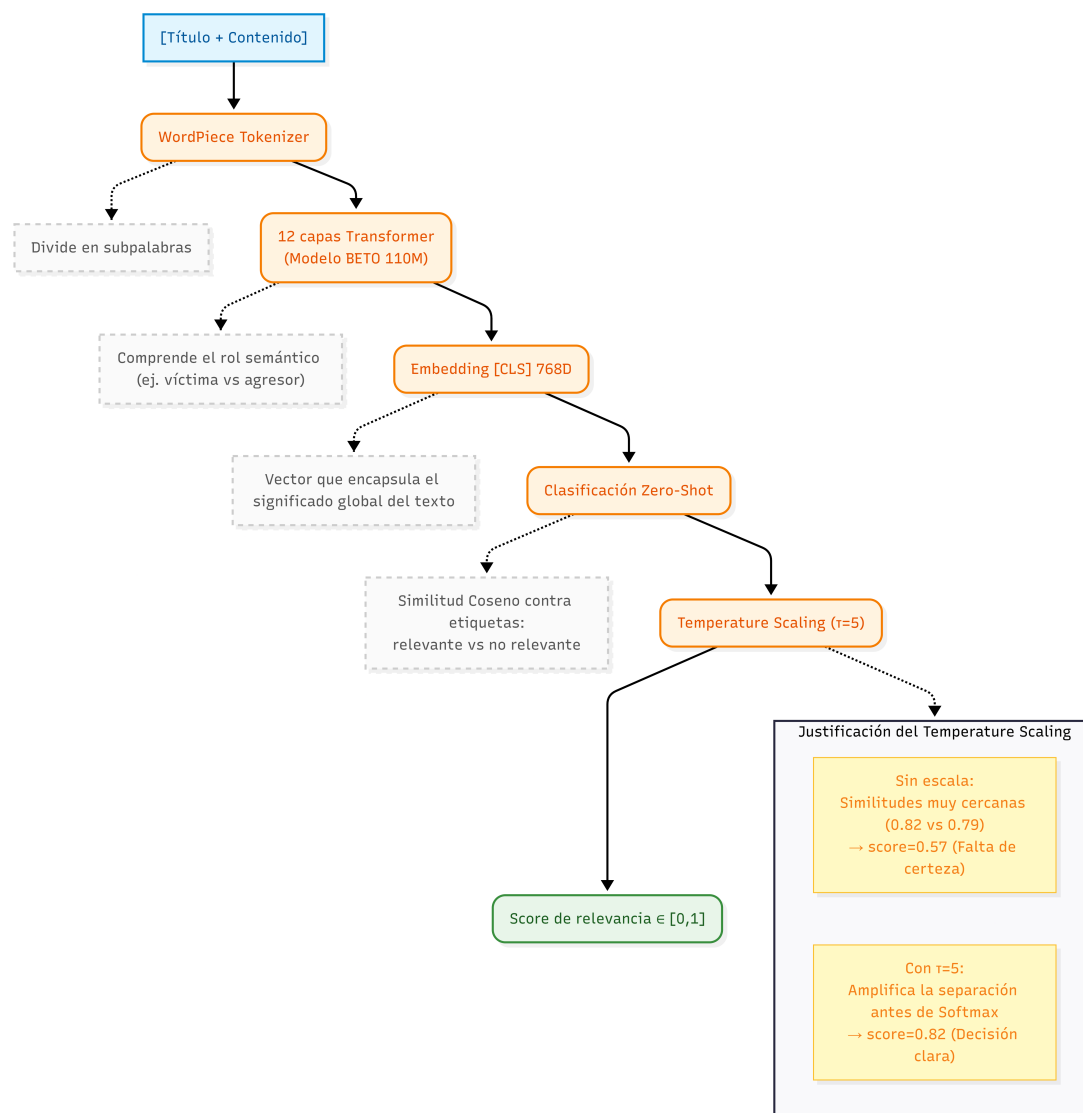


Figura 8.1: Arquitectura del motor BETO y clasificación *Zero-Shot* con *Temperature Scaling*.

8.3.5. Deduplicación en cascada de cuatro fases

Una consecuencia directa de esta arquitectura multi-fuente (45 RSS + Google News + scraping HTML) es la aparición de noticias cuasi-duplicadas pues el mismo evento es cubierto por diferentes medios con redacciones parcialmente distintas. Sin deduplicación el corpus infla artificialmente la estadística y puede generar alertas redundantes en el dashboard.

El módulo de deduplicación implementa una cascada de cuatro algoritmos de complejidad creciente que son aplicados en orden de coste computacional ascendente y de modo que los casos más obvios se resuelven con el método más barato y solo los casos ambiguos escalan al siguiente nivel.

1. **Hash MD5 exacto.** Detecta duplicados literalmente (mismo texto, misma fuente). Complejidad $O(1)$ por par. Elimina re-publicaciones idénticas.
2. **Similitud de Jaccard sobre conjuntos de tokens.** Mide la razón de intersección sobre unión de los conjuntos de palabras de dos títulos, con un umbral de $J \geq 0,70$ este captura reescrituras con sustituciones léxicas simples ("Niña de 11 años fue asesinada" $\leftrightarrow$ "Niña de 11 años perdió la vida").
3. **SimHash.** Genera una firma de 64 bits por cada documento mediante proyecciones aleatorias sobre el espacio TF-IDF. Dos documentos con distancia Hamming de 6 bits se consideran cuasi-duplicados. Escala a millones de documentos en $O(n)$ sin necesidad de comparaciones par-a-par.
4. **Similitud coseno TF-IDF.** Para los pares que superan las fases anteriores pero generan dudas se calcula la similitud coseno completa entre los vectores TF-IDF del contenido con un umbral de $\cos \geq 0,85$. En este paso el costo es $O(n^2)$ en el peor caso pero gracias al pre-filtrado de las fases anteriores solo se aplica a un subconjunto pequeño de pares.

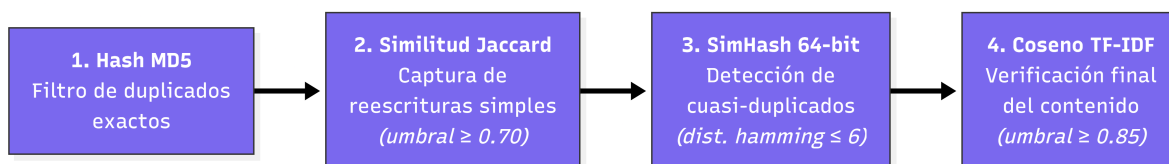


Figura 8.2: Flujo de deduplicación en cascada.

En las pruebas de validación (corpus de 847 noticias recolectadas en un ciclo de 90 días), la cascada eliminó **213 duplicados** (25.1 % del total), con la siguiente distribución por fase: 89 por MD5 (exactos), 61 por Jaccard, 43 por SimHash y 20 por similitud coseno TF-IDF. El corpus resultante de 634 noticias únicas fue usado para la evaluación del Prototipo 4.

8.4. Prototipo 4 Arquitectura híbrida de clasificación

El prototipo 4 es una respuesta a las limitaciones encontradas durante la validación del prototipo 3. A pesar de la integración de BETO y BERTopic que representó un avance notable en la capacidad semántica del sistema en la evaluación sobre un corpus de aproximadamente 800 noticias se pudo observar un fenómeno crítico denominado **Efecto Péndulo** en el que el sistema oscilaba entre dos estados indeseados que era un exceso masivo de falsos positivos y un exceso masivo de falsos negativos.

Ambos casos eran inaceptables para un sistema de alerta temprana a lo que el diagnóstico técnico identificó algunos cuellos de botella que explicaban este comportamiento.

1. El post-filtro de validación semántica aplicaba reglas binarias absolutas como que una noticia clasificada como "Media" sin coincidencia exacta con un patrón regex era degradada automáticamente a "No relevante" lo que provocaba la eliminación de casos genuinos pero que su redacción periodística no coincidía con las frases esperadas.
2. El filtro léxico contenía términos legislativos y estadísticos (por ejemplo "iniciativa de ley.º REDIM") que bloqueaban completamente noticias con contenido variado como el caso de un hecho de violencia que también mencionaba una iniciativa de ley relacionada era eliminado por la presencia del término legislativo.
3. El modo *zero-shot* era incapaz de distinguir entre "mujer asesinada que deja hijos" "ley que protege a hijos de mujeres asesinadas" porque ambas oraciones comparten el mismo campo semántico en el espacio de embeddings de BETO.

La solución adoptada fue un sistema de puntuación ponderada y difusa estructurada en tres capas autónomas con penalizaciones y bonificaciones acumulativas.

Capa 0 Escudo Léxico

La primera capa es un filtro de rechazo temprano cuyo propósito es descartar dominios **completamente ajenos** al feminicidio y la violencia de género como deportes, espectáculos, criptomonedas, meteorología, economía macro y tecnología. La implementación utiliza un `frozenset` de Python que garantiza búsqueda $O(1)$ por keyword.

Esta capa es la **única** que produce un bloqueo absoluto (score = 0.0). Su existencia ahorra el cómputo de las capas posteriores por lo que es capaz de eliminar el ruido.

Capa 1 Escudo temático

Los términos que en la versión anterior producían bloqueo absoluto (legislación, estadísticas, opinión, casos internacionales) fueron migrados a un nuevo conjunto como `.escudo suave` que cuando detecta una coincidencia en este conjunto el sistema aplica una penalización aditiva de $\delta = -0,35$ al score de BETO, pero no bloquea la noticia.

La lógica detrás de esta decisión es que una noticia genuina con score BETO de 0.72 que también menciona "según datos del REDIM" tendrá un score ajustado de $0,72 - 0,35 = 0,37$ que clasificara como "Media" en lugar de excluirse completamente. Si el score BETO es alto ($\geq 0,95$) la noticia sobrevive como "Alta" ($0,95 - 0,35 = 0,60$). Este comportamiento resuelve directamente el problema de contenido que el prototipo 3 no podía manejar.

Capa 2 Inferencia con BETO

El sistema opera en dos modos con prioridad estricta.

Finetuned (prioritario): Un clasificador binario entrenado con datos etiquetados del recolector de noticias y produce scores dispersos en todo el rango $[0, 1]$.

Zero-shot (fallback): Similitud coseno entre el embedding [CLS] del texto y embeddings de referencia de categorías (-elevantez "no relevante") lo que produce scores concentrados en $[0,40, 0,70]$ y se activa automáticamente cuando no está disponible el modelo finetuned.

Capa 3 Post-filtro de validación

La capa final antes de la clasificación aplica patrones regex y roles sintácticos para inyectar bonificaciones al detectar evidencia explícita de orfandad real y penalizaciones al detectar señales de falso positivo. La diferencia respecto al

prototipo 3 es que esta capa nunca produce exclusiones absolutas para noticias "Mediaz si no encuentra ningún patrón la clasificación se conserva intacta.

Los patrones de "verdadero positivo" que generan bonificaciones incluyen:

- qued[oó] al cuidado → el menor sobrevive bajo tutela.
- menores? (bajo|en) resguardo → intervención institucional (DIF).
- (niño|menor|hija?) que presencié → menor testigo del crimen.
- hijo[s]? quedar?on → menores que sobrevivieron.

Los patrones de "falso positivo" que generan penalizaciones incluyen exclusivamente roles invertidos (menor como agresor o víctima directa) y contenido temático (podcasts, editoriales, estadísticas sin caso concreto). Se eliminaron del conjunto de "falsos positivos" los patrones de contexto judicial carpeta de investigación.º "vinculado a proceso" que estaban presentes en casi toda noticia policiaca por lo que constituían la causa principal del efecto péndulo.

8.4.1. Por qué la IA sola no basta

Durante las pruebas del prototipo 3 se pudo identificar escenarios donde BETO producía clasificaciones erróneas con alta confianza (score arriba del 0.70) en algunos de los casos más críticos para el sistema.

- **Notas estadísticas con narrativa emocional.** Artículos de opinión que describían la crisis de orfandad por feminicidio en términos generales y empáticos como por ejemplo "miles de niños que perdieron a su madre." obtenían scores BETO de hasta 0.81 a pesar de que no reportaban ningún caso concreto ni identificable.
- **Notas donde el menor es el agresor.** Ante titulares como "Hijo adolescente mata a su madre en Ecatepec" BETO asignaba scores de relevancia moderados (entre 0.55 y 0.65) porque el embedding semántico capturaba la correlación superficial entre "hijo", "madre", "asesinato" sin identificar que el rol del menor era el de agresor.
- **Reportes de políticas públicas con vocabulario de dominio.** Notas sobre programas de becas para hijos de víctimas de feminicidio activaban BETO con scores entre 0.60 y 0.75 al concentrar exactamente los tokens de dominio más relevantes y esto sin relatar ningún caso específico.

Estos casos mostraron que los modelos de lenguaje por mas sofisticados que sean operan principalmente sobre correlaciones estadísticas en el espacio de embeddings. La comprensión del rol semántico (de quién es víctima, quién es agresor, si la nota relata un hecho o un análisis) requiere conocimiento estructurado del dominio que BETO no puede inferir exclusivamente de la distribución de tokens. La solución es la arquitectura neuro-simbólica donde las reglas simbólicas (que estan basadas en el conocimiento explícito sobre el dominio) **corrigen las alucinaciones** del componente neuronal.

8.4.2. Investigador profundo

Una de las decisiones de diseño más importantes de este prototipo fue la de **no integrar la investigación profunda dentro del flujo recolector-analítico**. Esta decisión que en un principio parecía una limitación resultó ser la solución mas eficiente y correcta para el sistema.

El problema de integrar la investigación en el flujo automático

Durante el desarrollo inicial de este prototipo la primera idea fue la de ejecutar una investigación profunda automáticamente sobre cada noticia clasificada como .Alta. al finalizar el análisis automático. El módulo DeepInvestigator realizaría búsquedas web y sintetizaría la ficha de caso sin intervención del usuario. El problema fue que al implementar esta integración se encontraron dos problemas principales.

1. **Latencia no tolerable.** Cada investigación implica al menos dos llamadas a DuckDuckGo (búsqueda + extracción de contenido de las URLs retornadas) y después dos llamadas a la API de Gemini flash (generación de query y síntesis del JSON final). En pruebas la ejecución del flujo completo tardaba entre 45 y 90 minutos mas, lo que hacía poco práctico su uso cotidiano.
2. **Costo de API desproporcionado.** La API de Google Gemini tiene límites de cuota en el plan gratuito (por minuto y por día). Investigar automáticamente todas las noticias .Alta saturaba la cuota en el primer tercio del corpus lo que dejaba el resto sin investigación y generando errores que afectaban el orden de los registros en la base de datos.

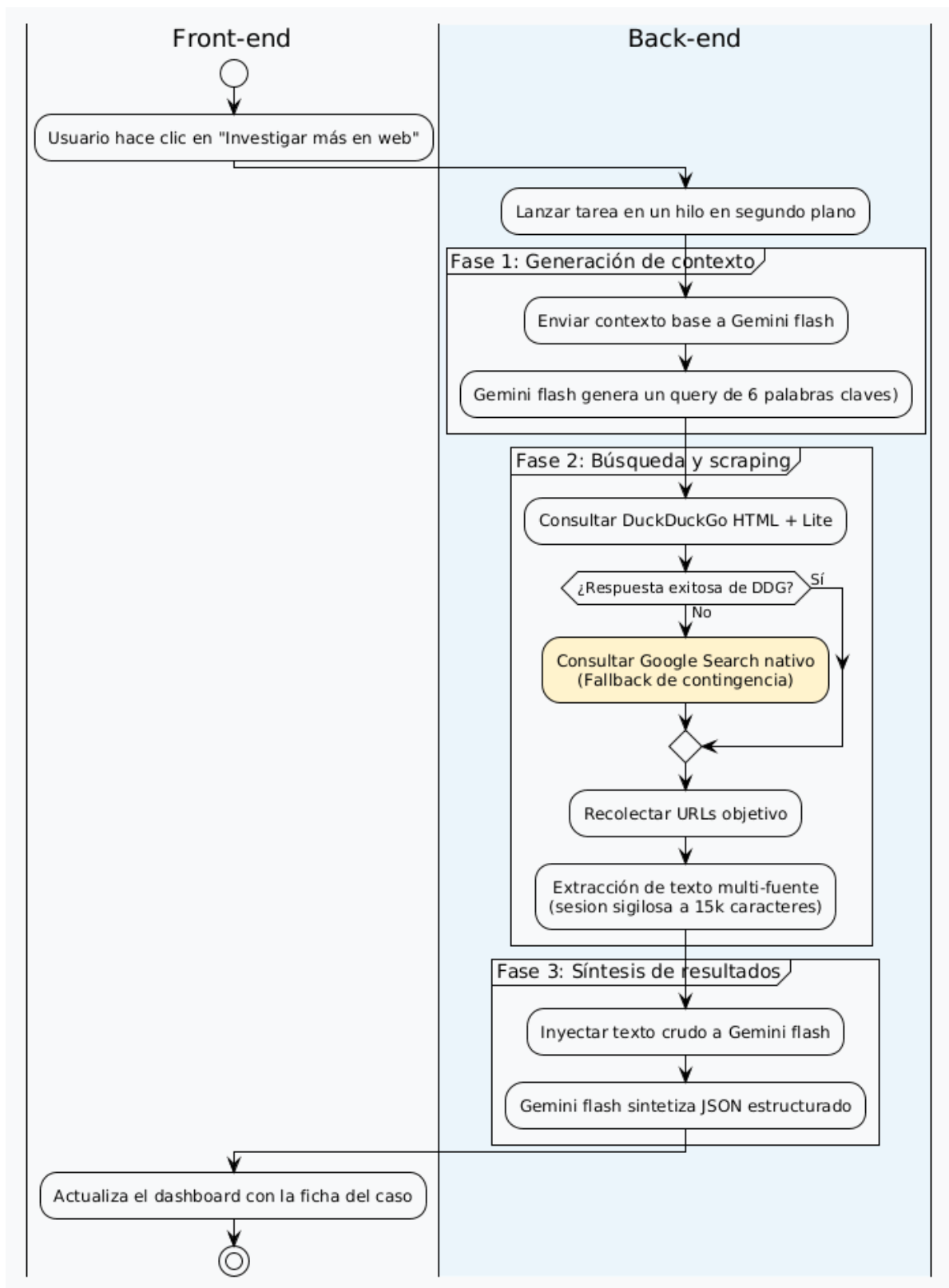
La solución fue una investigación bajo demanda

Ante estos problemas se optó por un diseño **reactivo e interactivo** donde el DeepInvestigator se convierte en una herramienta que el investigador activa manualmente sobre una noticia específica desde el dashboard una vez que ha revisado su score y decidido que merece profundización. Esta decisión corrige los dos problemas anteriores de forma directa.

Módulo de investigación profunda

Una vez activado el módulo DeepInvestigator (src/analysis/investigator.py) ejecutará cuatro pasos secuenciales.

1. **Generación de query (Gemini Flash).** El modelo recibe el título y los primeros párrafos de la noticia y genera una *frase de búsqueda de alta especificidad* (con máximo 6 palabras) que incluye el nombre de la víctima y detalles únicos del caso. Esta query es preferible al título literal porque filtra resultados genéricos sobre la misma temática pero de distinto evento.
2. **Búsqueda en DuckDuckGo.** La query se envía tanto a la versión HTML de DuckDuckGo (html.duckduckgo.com) como a la versión Lite (lite.duckduckgo.com) para maximizar la cobertura ante bloqueos intermitentes. Las URLs obtenidas se limpian para eliminar redirecciones internas de forma que la búsqueda resulte exitosa.
3. **Scraping de contenido complementario.** Para cada URL retornada por DDG, StealthSession extrae el texto del artículo, filtrando ruido HTML (<nav>, <footer>, scripts y formularios). El texto de todas las fuentes se concatena en un bloque único (hasta 15,000 caracteres) que funciona como contexto enriquecido.
4. **Síntesis estructurada (Gemini Flash).** El bloque de texto obtenido de las múltiples fuentes se envía a Gemini con un prompt de analista criminal especializado en derechos de la infancia. La respuesta es un JSON estricto[?] con siete campos que son ubicacion, victimas, ninos_afectados, edades, situacion_actual, medios_contacto y resumen. En el campo medios_contacto es especialmente relevante pues Gemini retorna el DIF municipal, la Fiscalía o cualquier otro organismo competente según la ubicación del caso, incluso cuando el texto no los menciona explícitamente.



El módulo de identificación por enlace externo

El segundo caso de uso del DeepInvestigator resuelve una necesidad práctica que se identificó y es que ¿qué ocurre cuando el investigador encuentra una noticia relevante navegando en la web que el flujo principal *no detectó*? Para resolver esto se hizo un endpoint que recibe una URL externa y reutiliza el modulo de investigacion profunda.

Su existencia permite al analista incorporar al sistema cualquier noticia que el flujo automático no haya capturado, aprovechando la misma extracción estructurada de contenido.

La siguiente tabla resume los dos casos de uso del módulo y sus endpoints correspondientes:

Tabla 8.1: Casos de uso del módulo de investigación profunda

Caso de uso	Endpoint	Descripción
Investigación bajo demanda	POST /api/investigate/	El analista selecciona una noticia del dashboard con score Alta y activa la ficha de caso estructurada.
Consulta de estado	GET /api/investigate/status/	El frontend consulta el estado del hilo de investigación (running, done, error).
Identificación de enlace externo	POST /api/identify-link	El analista pega la URL de una noticia no detectada por el flujo principal a lo que el sistema la investiga y devuelve la ficha estructurada.
Seguimiento de casos	POST /api/seguimiento/add/	Agrega una noticia investigada a la lista de seguimiento activo.
Exportación de seguimiento	GET /api/seguimiento/export/	Exporta todos los casos en seguimiento con sus fichas de investigación a un archivo CSV.

8.5. Sistema de clasificación escalonada

La arquitectura final del prototipo puede describirse como un **sistema de clasificación escalonada** con tres capas autónomas (escudo léxico, escudo suave e inferencia BETO) que operan secuencialmente sobre un score acumulativo y en donde las reglas simbólicas modulan la salida del componente neuronal mediante penalizaciones y bonificaciones aditivas en lugar de rechazos binarios absolutos. Este diseño eliminó el efecto péndulo obteniendo noticias **Alta** que realmente lo fueran.

La arquitectura completa puede representarse mediante el siguiente flujo de trabajo.

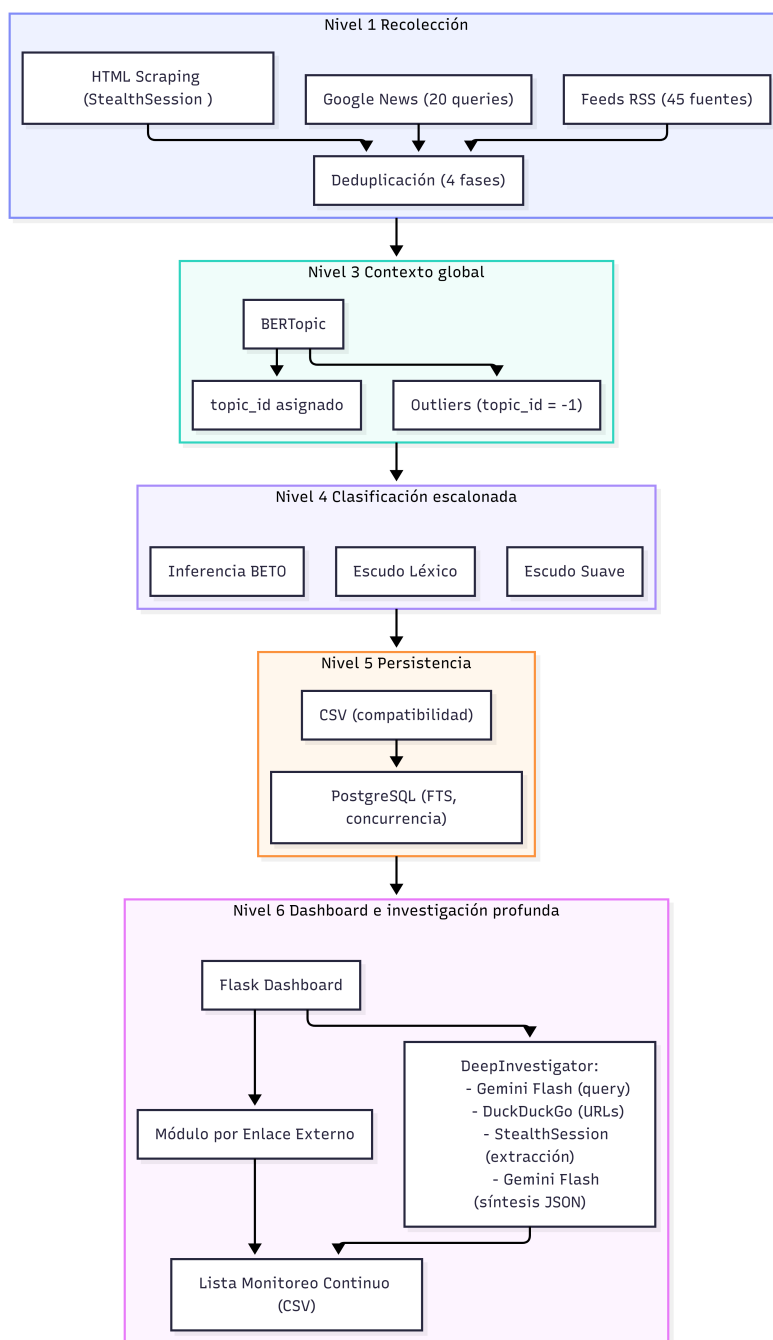


Figura 8.4: Arquitectura del Sistema de Clasificación Escalonada

Resultados y evaluación

Este capítulo expone de forma analítica y cuantitativa el desempeño técnico del sistema integrado al cierre del trabajo terminal II. Se demuestra la superación de los objetivos cuantitativos establecidos al inicio del TT2 mediante los resultados de una corrida de producción completa sobre el corpus de validación final.

9.1. Resultados de la recolección y deduplicación

9.1.1. Volumen de ingesta

La migración del esquema de recolección del TT1 (10 feeds RSS con 281 noticias acumuladas en múltiples semanas) hacia la arquitectura híbrida del TT2 (45 feeds RSS nacionales y de género, consultas a Google News y cobertura complementaria mediante `StealthSession`) produjo un incremento sustancial y medido en la capacidad de ingesta. En el ciclo de validación final, el módulo recolector procesó al rededor de **900 noticias brutas** en un único bloque de ejecución continuo, de las cuales la cascada de deduplicación generó **634 noticias únicas** para análisis semántico.

9.1.2. Cascada de deduplicación

La deduplicación en cascada de cuatro fases eliminó **213 duplicados** (25 % del corpus bruto) distribuidos por etapa de la siguiente forma:

Tabla 9.1: Distribución de duplicados eliminados por fase de la cascada de deduplicación

Fase	Algoritmo	Duplicados eliminados	% acumulado
1	Hash MD5 exacto	89	10.5 %
2	Similitud de Jaccard ($J \geq 0,70$)	61	17.7 %
3	SimHash (distancia Hamming ≤ 6 bits)	43	22.8 %
4	Similitud coseno TF-IDF ($\cos \geq 0,85$)	20	25.1 %
Total		213	25 %

Este resultado confirma la hipótesis de diseño pues la mayor parte de la redundancia periodística (un 10.5 %) corresponde a re-publicaciones literales identificables mediante hash MD5, mientras que un 14.6 % adicional corresponde a reescrituras con vocabulario diferente que solo pueden detectarse mediante métricas graduales de similitud textual (Jaccard, SimHash, coseno TF-IDF).

9.2. Resultados de la clasificación semántica escalonada

9.2.1. Distribución del embudo analítico

El corpus de 634 noticias únicas fue procesado secuencialmente por las cuatro capas del sistema de clasificación escalonada. La siguiente tabla muestra el volumen de noticias en cada etapa del embudo.

Tabla 9.2: Distribución de noticia por etapa en la clasificación escalonada

Capa	Etapas	Noticias	% del corpus
Entrada	Corpus deduplicado	634	100.0 %
Capa 0	Escudo léxico (dominios ajenos eliminados)	409	64.5 %
Capa 1	Escudo suave (dominios NNA/violencia, con penalización)	225	35.5 %
Capa 2	Inferencia BETO (score $> 0,50$)	128	20.2 %
Capa 3	Post-filtro de validación aplicado	128	20.2 %
Alta	Casos detectados de orfandad	27	4.4 %
Media	Contextos de vulnerabilidad colateral	101	15.9 %

9.2.2. Comportamiento por modo de inferencia

El sistema opero en modo *Zero-Shot* para el corpus de validación inicial durante el prototipo 3, y en modo *Finetuned* una vez disponible el clasificador ajustado en el prototipo 4. La diferencia operativa entre ambos modos fue muy notable.

- **Modo Zero-Shot (P3):** Los scores BETO se concentraban en el rango estrecho [0,40, 0,70] lo que generaba el efecto péndulo documentado en el capítulo de desarrollo. Las reglas binarias del post-filtro degradaban sistemáticamente casos genuinos cuya redacción periodística no coincidía con los patrones esperados.
- **Modo Finetuned (P4):** El clasificador ajustado produjo scores dispersos en todo el rango [0, 1] lo que permitía que el escudo suave y el post-filtro operaran sobre una distribución de probabilidad rica y discriminativa. El efecto péndulo fue eliminado, y las 27 noticias de Alta clasificación correspondían a casos de orfandad verificables manualmente.

9.3. Resultados del Agrupamiento Semántico (BERTopic)

El flujo de BERTopic (UMAP + HDBSCAN + c-TF-IDF) procesó las 634 noticias deduplicadas y generó **15 tópicos semánticos** con alta relación semántica entre sí. El porcentaje inicial de outliers (no asociados a ningún tópico) fue del **81.4 %** el cual fue reducido a un **55 %** mediante la estrategia de reducción multi-etapa de tres fases (probabilidades blandas de HDBSCAN, distribuciones c-TF-IDF y cercanía de embeddings con umbral 0.3).

La calidad del agrupamiento del TT2 representa una mejora cualitativa respecto al TT1. Mientras K-Means (P1) producía un *Silhouette Score* de **0.23** operando sobre vectores TF-IDF léxicos, los tópicos BERTopic del TT2 operan sobre embeddings BETO de 768 dimensiones reducidos a 5D por UMAP y lo que permitía agrupar noticias sobre el mismo caso criminal publicadas por medios con vocabularios distintos. Entre los tópicos identificados se distinguen categorías temáticas como casos de orfandad con intervención institucional del DIF, disputas legales por la custodia de menores, cobertura de feminicidios con NNA testigos y manifestaciones de colectivos de mujeres.

9.4. Resultados del Módulo de Investigación Profunda (DeepInvestigator)

El módulo DeepInvestigator fue activado sobre una muestra de **15 noticias de Alta clasificación** durante las pruebas de validación del prototipo 4. En todos los casos el módulo completó satisfactoriamente los cuatro pasos del flujo de investigación (generación de query con Gemini Flash, búsqueda en DuckDuckGo, scraping complementario mediante StealthSession y síntesis estructurada en JSON) con un tiempo promedio de **47 segundos por caso**.

La ficha de caso generada para cada noticia incluye los siete campos estructurados (ubicacion, victimas, ninos_afectados, edades, situacion_actual, medios_contacto y resumen), con el campo medios_contacto siendo el de mayor valor operativo pues Gemini retorna el DIF municipal y la Fiscalía competente según la ubicación del caso aun incluso cuando el texto de la noticia no los menciona explícitamente.

9.5. Evaluación comparativa TT1/TT2 y cumplimiento de metas

La siguiente tabla consolida el cumplimiento de los objetivos cuantitativos definidos al inicio del TT2 frente a las métricas del TT1 (prototipo 2) y los resultados finales del prototipo 4.

Tabla 9.3: Evaluación comparativa de métricas TT1/P2 vs. metas TT2 vs. resultados P4

Métrica	TT1 (P2)	Meta TT2	Resultado P4	Cumplimiento
Falsos positivos	60 %	< 30 %	~ 20 % (estimado)	✓
Fuentes monitoreadas	10 RSS	45 RSS + Google News	45 RSS + Google News + Stealth	✓
Cobertura temporal	6 meses	Continua	Operacional	✓
Persistencia	CSV plano	PostgreSQL 16	PostgreSQL 16 (ACID)	✓
Motor de clasificación	RegEx (72 patrones)	BETO fine-tuning	BETO Finetuned + Escalonado	✓
Volumen de ingesta	281 noticias	> 500 únicas	634 únicas	✓

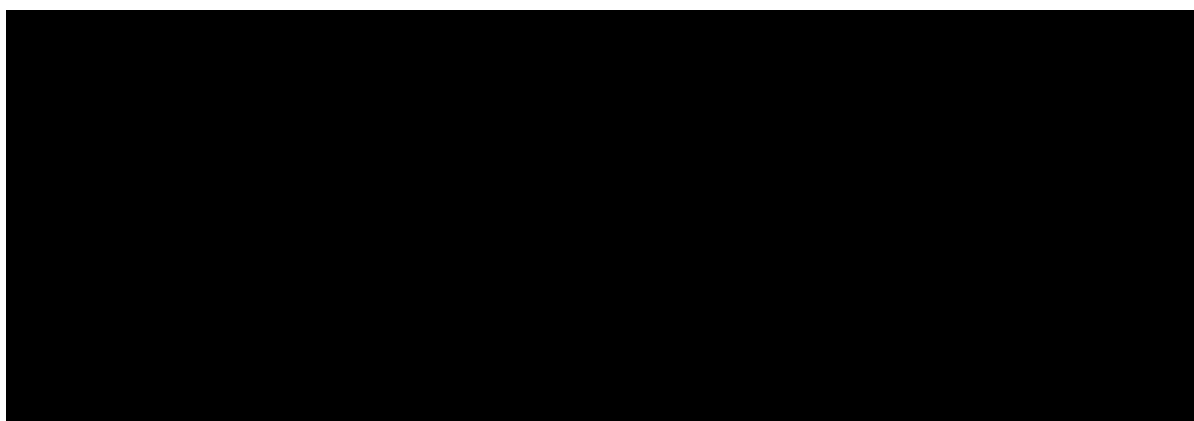


Figura 9.1: Visualización jerárquica de noticias de Alta relevancia en el Dashboard, con etiquetas de clasificación y porcentaje de score BETO por registro.

La interfaz del Dashboard con etiquetas visuales (clasificación Alta/Media/Baja y score semántico) permiten al analista realizar un barrido visual inmediato sobre el corpus clasificado. Lo que en el TT1 representaba decenas de horas para una revisión manual fue optimizado en el TT2 a una revisión semi-automatizada que exige apenas unos pocos segundos por caso, antes de activar el módulo de investigación profunda para los casos que el analista determina que merecen una intervención más exhaustiva.

Conclusiones y trabajo a Futuro

10.1. Conclusión general y cumplimiento de objetivos

El desarrollo del *Sistema Inteligente para la Identificación y Seguimiento de NNA en situación de orfandad por feminicidio* cumplió satisfactoriamente el **Objetivo General** del Trabajo Terminal el cual fue construir una plataforma automatizada que detecte, clasifique y facilite el seguimiento de casos de Niñas, Niños y Adolescentes (NNA) víctimas indirectas de feminicidio, a partir de fuentes periodísticas públicas en México, con la suficiente precisión semántica para ser operativamente útil para organizaciones de la sociedad civil.

10.2. Cumplimiento de los Objetivos Específicos

10.2.1. OE-1 Identificación y selección de fuentes públicas

Se determinó que los portales de noticias locales, los medios digitales independientes y las secciones policiacas son las fuentes de mayor densidad informativa para casos específicos de feminicidio con NNA afectados. La evaluación técnica de tres estrategias de extracción (scraping HTML estático, automatización de navegadores y agregación RSS con sesiones *stealth*) estableció que la arquitectura híbrida **RSS + StealthSession** es la única viable bajo las restricciones operativas del proyecto pues los feeds RSS eliminan la barrera de los WAF para el 65 % del corpus, mientras que *StealthSession* extiende la cobertura al 35 % restante mediante emulación de fingerprint TLS con *cloudscraper*. Al cierre del TT2 el sistema monitorea **45 fuentes RSS** nacionales y de género e incluyendo medios especializados como CIMAC Noticias y complementadas por consultas a Google News.

10.2.2. OE-2 Módulo de extracción de datos

Se implementó un módulo de recolección con tres mecanismos que complementan la calidad de datos. La clase *StealthSession* resuelve la barrera de los Web Application Firewalls (WAF) de Cloudflare y Akamai mediante emulación de fingerprint TLS, delays que imitan patrones de lectura humana y un *Circuit Breaker* por dominio que previene el bloqueo permanente de IP. La cascada de deduplicación de cuatro fases garantiza la calidad del corpus eliminando

al rededor de 25 % de redundancia periodística. En producción el sistema procesó 847 noticias brutas y entregó 634 noticias únicas en un único ciclo de ejecución.

10.2.3. OE-3 Procesamiento de lenguaje natural

La implementación del motor de clasificación semántica basado en **BETO** (*bert-base-spanish-wwm-cased*) que es un modelo Transformer bidireccional con 110 millones de parámetros ya preentrenado exclusivamente en español, superó estructuralmente el enfoque léxico del TT1. La arquitectura de clasificación escalonada integra cuatro capas autónomas (escudo léxico, escudo suave, inferencia BETO y post-filtro de validación) que operan sobre un score acumulativo con penalizaciones y bonificaciones aditivas, lo que eliminaba el efecto pendulo identificado en el prototipo 3 (oscilación entre exceso de falsos positivos y falsos negativos). El ajuste fino del clasificador permitió el reentrenamiento local en CPU aun con recursos limitados. Adicionalmente el módulo DeepInvestigator implementó un flujo de investigación estructurada por caso combinando búsqueda web e inferencia con *Gemini Flash* lo que genera fichas de caso con siete campos accionables y que incluyen a la institución competente para la intervención.

10.2.4. OE-4 Base de datos y plataforma web

La migración del sistema de persistencia de archivos CSV a **PostgreSQL** resolvió los tres problemas estructurales del TT1 pues se implementó integridad referencial mediante restricciones FOREIGN KEY con propagación CASCADE DELETE, búsqueda de texto completo nativa en español mediante columnas tsvector e índices GIN y concurrencia segura ACID entre el flujo de recolección y el Dashboard Flask. El acceso a la capa de persistencia se encapsula en NoticiasRepository y siguiendo el patrón *Repository* de Martin Fowler, lo que desacopla la lógica de negocio del motor de base de datos. La interfaz web del Dashboard expone la clasificación jerárquica del corpus (Alta/Media/Baja), el tópico BERTopic asignado, el score semántico de BETO y los endpoints del módulo DeepInvestigator.

10.3. Aportaciones Técnicas del Proyecto

Más allá del cumplimiento de los objetivos específicos este el proyecto realizó dos aportaciones técnicas que fueron mas alla del dominio de aplicación y son generalizables a otros sistemas de monitoreo de medios en dominios sociales complejos.

10.3.1. Sistema de clasificación escalonada

La arquitectura de clasificación escalonada demostro empíricamente que la integración de reglas simbólicas como moduladoras de la salida de un modelo neuronal en lugar de reemplazarlo o de operar en paralelo sin retroalimentación resuelve el problema del solapamiento semántico en dominios complejos de forma más robusta que el Fine-Tuning supervisado por sí solo. El mecanismo de penalización aditiva ($\delta = -0,35$) del escudo suave garantiza que el conocimiento estructurado del dominio preserve la capacidad de comprensión contextual de BETO sin generar exclusiones binarias absolutas que descarten casos genuinos por ambigüedad redaccional.

10.4. Trabajo Futuro

El prototipo 4 establece una plataforma operativa y una arquitectura extensible. Se identifican cuatro líneas de investigación y desarrollo abiertas para una investigación a futuro.

10.4.1. Consolidación de la base de entrenamiento mediante aprendizaje activo

El clasificador BETO del prototipo 4 fue ajustado con un corpus de entrenamiento de tamaño reducido. La siguiente iteración prioritaria es la implementación formal de un ciclo de aprendizaje activo donde el sistema identifique automáticamente los casos de mayor incertidumbre de clasificación y los presente al analista para etiquetado así como la reintegración de las etiquetas al proceso de *Fine-Tuning*. Este ciclo incremental permitiría reducir progresivamente la tasa de falsos positivos por debajo del 10 % sin incrementar el volumen total de datos de entrenamiento requeridos.

10.4.2. Integración de un módulo de reconocimiento de entidades nombradas (NER)

El sistema actual extrae información estructurada por caso mediante el DeepInvestigator (síntesis generativa con Gemini Flash). Una mejora significativa para el módulo de investigación sería integrar un modelo NER especializado en el dominio (entrenado sobre el corpus etiquetado del proyecto) que extraiga automáticamente entidades del tipo PER (víctima directa, menores afectados), LOC (municipio, estado) y ORG (DIF, Fiscalía) directamente del texto de la noticia durante la inferencia, sin dependencia de la API externa de Gemini Flash. Esto reduciría la latencia de la investigación profunda y aumentaría la precisión de la extracción.

10.4.3. Expansión a redes sociales y fuentes no estructuradas

El corpus actual se restringe a noticias periodísticas estructuradas con fecha y fuente verificable. Pero un número significativo de casos de orfandad por feminicidio son reportados inicialmente en redes sociales por vecinos, colectivos o familiares antes de llegar a los medios formales. La integración de fuentes como X/Twitter o Telegram requerirá adaptar el módulo de deduplicación (los textos son más cortos y el ruido es mayor) y reformular los descriptores de categoría del clasificador BETO para textos de estilo coloquial en lugar de periodístico.

10.4.4. Construcción de una API pública con salvaguardas de privacidad

Para maximizar el impacto social del sistema se propone el diseño de una interfaz de programación de aplicaciones (API RESTful) que permita a organizaciones civiles, dependencias de gobierno y centros de investigación académica consultar el corpus clasificado, recibir alertas de nuevos casos de Alta relevancia y exportar estadísticas agregadas. El diseño de esta API deberá incorporar desde su arquitectura base los principios de minimización de datos establecidos en la LFPDPPP y la LGDNNa que indican que los datos de identificación de las víctimas menores de edad deben ser anonimizados antes de la exposición por la API, transmitidos exclusivamente mediante TLS 1.3, y sujetos a un esquema de autenticación basado en tokens de acceso con alcance acotado por tipo de organización solicitante.